ARTIFICIAL INTELLIGENCE – CASE STUDY

JARVIS: Intelligent Iron Man Armor Recovery and Resource Optimization Using Uniform Cost Search, Simulated Annealing, and Constraint Satisfaction Problems

One integrated engineering mission: restoring a damaged Mark 85 armor through optimal power routing, stochastic nanomaterial optimisation and constraint-based laboratory scheduling

Course	Artificial Intelligence
Submission type	Technical case study, implementation and viva
Submitted by	_____
Register / Roll No.	_____
Instructor	_____
Date	October 2026
Implementation	Python 3 (heapq, math, random, matplotlib, networkx)
Mission ID	MK85-REC-001

Simulation notice. The Iron Man Mark 85 armor, the Arc Reactor, FRIDAY, JARVIS and Stark Laboratory are fictional (Marvel). Every number in this case study – power ratings, losses, thermal rises, nanomaterial quantities, task durations – is invented simulation data designed for an academic exercise. None of the values is a measurement of a real physical system. The algorithms, mathematics, code and results, however, are real: every result reported here was produced by executing the accompanying Python application.

Contents

Abstract
1. Introduction
2. Background: Iron Man Technology Concepts
3. Unified Engineering Problem Statement
4. Objectives
5. System Architecture
6. Problem Formulation and Mathematical Models
7. The Three Algorithms and Why All Three Are Needed
8. Integrated Methodology and Data Flow
9. Pseudocode
10. Python Implementation
11. Experimental Setup and Common Input Dataset
12. Step-by-Step Execution Trace
13. Results
14. Performance Analysis
15. Algorithm Comparison
16. Limitations
17. Real-World Engineering Relevance
18. Conclusion
References
Viva Questions and Answers
Appendix A. Complete Source Code
Appendix B. Shared Mission Dataset (JSON)
Appendix C. Console Output of the Executed Mission

Abstract

During a test session at Stark Tower, FRIDAY detects a cascading failure in the Iron Man Mark 85: the primary power converter (PC1) is offline, the Arc Reactor is derated to 1000 kW, the nanomaterial reserve is limited to 1200 nano-units (NU), and five subsystems are damaged. This case study presents JARVIS, an integrated AI recovery planner that solves this single engineering problem in three cooperating stages. Stage 1 applies Uniform Cost Search (UCS) to a weighted power-distribution graph and finds the minimum-cost route that avoids failed components and respects capacity and cumulative thermal limits (ARC → PC2 → SW1 → SW2 → BUS_B → CLH, cost 44.20). Because the thermal limit is a path constraint, we use a label-setting form of UCS with dominance pruning that remains provably optimal. The route fixes the deliverable power and therefore the repair power budget B = 530 kW. Stage 2 uses Simulated Annealing (SA) to distribute nanomaterial among five protection and repair functions, maximising a non-convex readiness objective subject to the budget, the reserve, mandatory structural protection and the armor thermal limit. Stage 3 converts the route and the allocation into task durations and a residual power cap and solves the laboratory scheduling problem as a Constraint Satisfaction Problem (CSP) using backtracking with Minimum Remaining Values, the degree heuristic and forward checking. JARVIS monitors feasibility between stages. In the executed mission the first plan was rejected because armor charging (120 kW) exceeded the residual power (111.9 kW), and JARVIS replanned by lowering the nanomaterial power fraction from 0.80 to 0.70. The final plan completes in 15 slots (7.5 h) against a 8 h deadline and passes 36/36 independent constraint checks. Experiments show SA outperforming pure hill-climbing on all 30 random seeds and MRV + degree + forward checking reducing CSP search from 3828 to 10 nodes.

Keywords: uniform cost search, resource-constrained shortest path, simulated annealing, constraint satisfaction, MRV, forward checking, integrated planning, replanning.

Simulation notice. The Iron Man Mark 85 armor, the Arc Reactor, FRIDAY, JARVIS and Stark Laboratory are fictional (Marvel). Every number in this case study – power ratings, losses, thermal rises, nanomaterial quantities, task durations – is invented simulation data designed for an academic exercise. None of the values is a measurement of a real physical system. The algorithms, mathematics, code and results, however, are real: every result reported here was produced by executing the accompanying Python application.

1. Introduction

Real engineering recoveries are rarely a single textbook problem. Restoring a damaged system usually means deciding how to re-route energy, how to spend scarce material, and in which order to perform maintenance with shared equipment. These decisions are coupled: the energy route limits the power available for repair, the repair allocation sets how much work must be scheduled, and the schedule must fit within the energy and equipment left over. Treating them independently can produce three locally sensible answers that cannot be executed together.

This case study uses the fictional Iron Man Mark 85 armor as a rich, engaging setting for exactly such a coupled problem. JARVIS, Tony Stark's AI system, has to bring the armor back to a safe operational state after a failure detected by FRIDAY. We show that three classical AI techniques, each matched to the structure of one sub-problem, can be chained into one pipeline in which the output of each stage becomes a hard input of the next:

- **Uniform Cost Search** for the shortest-path structure of the power network (optimal, complete, non-negative additive costs);

- **Simulated Annealing** for the rugged, non-convex resource-allocation landscape (good solutions without exhaustive enumeration, escapes local optima);
- **Constraint Satisfaction** for the scheduling problem whose difficulty lies in many hard, interacting constraints rather than in an objective.

The contribution is not a new algorithm but a complete, executable and validated integration: one shared dataset, one orchestrator with explicit data hand-offs and replanning rules, independent validation of every constraint, and a dashboard reporting one final recovery outcome.

2. Background: Iron Man Technology Concepts

The engineering vocabulary used in this case study is taken from the Iron Man universe and translated into components that an AI planner can reason about. All specifications are fictional.

Table 1. Iron Man technology concepts and their computational models

Concept	Role in this case study	Modelled as
Arc Reactor	Compact power source in the armor chest; derated after the fault	Graph source node ARC, P_avail = 1000 kW
Power converters PC1–PC3	Condition reactor output for the armor bus; PC1 has failed	Graph nodes; status FAILED makes a node unusable
Backup storage (BKP)	Supercapacitor bank providing an alternative feed	Graph node with high loss
Switching units, distribution buses	Route power to the critical load hub	Graph nodes SW1–SW3, BUS_A–C
Critical load hub (CLH)	Feeds flight controller, stabiliser, sensors, life support	UCS goal; critical load 420 kW
Nanotech (Mark 50+ style) armor	Nanoparticles reconfigure into plates, shields, repairs	Reserve of 1200 NU split into 5 categories
FRIDAY	Fault-monitoring AI that raises the alarm	Source of the subsystem status data
JARVIS	Planning AI that coordinates recovery	The orchestrator implemented in Python
Stark Laboratory	Robotic arms, repair stations, bays, charger, test rig, technicians	CSP resources and capacities

3. Unified Engineering Problem Statement

FRIDAY reports the following state of the Mark 85 at the start of the mission:

Table 2. Subsystem status at fault detection (fictional)

ID	Subsystem	Health	Critical load	Status
FLC	Flight Controller	62%	120 kW	DEGRADED
STB	Stabilization System	55%	110 kW	DEGRADED
SEN	Sensor Array	71%	60 kW	MISCALIBRATED
LSC	Life-Support Controller	88%	130 kW	NOMINAL-ON-BACKUP
STR	Structural Shell	47%	0 kW	BREACHED

The Arc Reactor supplies only 1000 kW of its 1500 kW nominal output, the primary converter PC1 is FAILED, and only 1200 NU of nanomaterial remain. The armor must be returned to a safe operational state within 16 laboratory slots of 30 minutes (8 hours).

Unified problem. Find (i) a power route π from the Arc Reactor to the critical load hub, (ii) a nanomaterial allocation x and (iii) a laboratory schedule S such that every mandatory safety constraint holds simultaneously, while minimising power-routing cost, maximising armor readiness for the power that is available, and completing all recovery tasks before the deadline. The three decisions are coupled: π determines the power budget and heat already present in the armor; x consumes part of that budget and determines how much repair, calibration and replenishment work exists; S must perform that work using only the power that π and x leave over.

4. Objectives

1. Model the Arc Reactor network as a weighted graph and compute the minimum-cost route that avoids failed components and satisfies capacity and cumulative thermal limits, using UCS with a priority queue.
2. Derive the repair power budget from the selected route and use it as a hard constraint of the nanomaterial optimisation.
3. Optimise the nanomaterial allocation with Simulated Annealing, demonstrating the acceptance of worse moves, cooling, convergence and escape from local optima.
4. Convert the route and allocation into concrete task durations and a residual power cap, and schedule nine laboratory tasks as a CSP with MRV, degree heuristic and forward checking.
5. Detect infeasibility at any stage and perform controlled replanning.
6. Independently validate the integrated solution and present it on a single recovery dashboard.
7. Provide complete, reproducible Python code, figures, an execution trace and a performance analysis.

5. System Architecture

Figure 1 shows the architecture. FRIDAY's telemetry and Tony Stark's activation command populate one shared mission dataset. The JARVIS orchestrator runs the three engines in sequence, passes their outputs forward as typed result objects, and sends feedback backwards when a later stage proves that an earlier decision cannot be executed. A validation engine recomputes every constraint from scratch and the visualisation layer produces the figures and the recovery dashboard.

Figure 1. JARVIS system architecture for the Mark 85 recovery mission

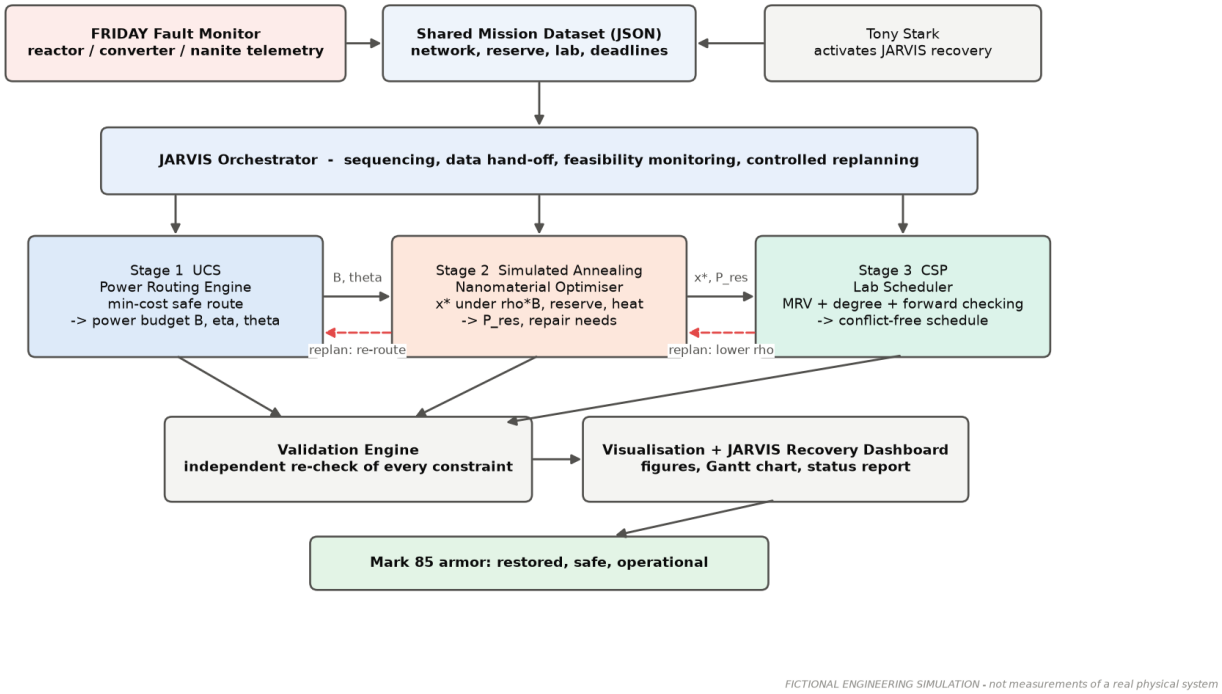


Figure 1. JARVIS system architecture

Table 3. Software modules

Module	Responsibility
data_models.py	Shared dataset loader and typed result objects (UCSResult, SAResult, Requirements, CSPResult)
ucs.py	Stage 1: label-setting Uniform Cost Search with capacity, failure and thermal constraints
annealing.py	Stage 2: allocation problem, objective, feasibility, neighbourhood, SA loop
requirements.py	Bridge: converts UCS + SA outputs into CSP durations and power cap
csp.py	Stage 3: CSP model, bound propagation, prechecks, backtracking + MRV + degree + FC
pipeline.py	Orchestrator: sequencing, replanning policy, makespan tightening, health estimate
validation.py	Independent re-check of all stage and integrated constraints
visualization.py	All figures and the recovery dashboard
experiments.py	Performance study (SA vs hill climbing, CSP heuristics, route strategies)
reporting.py / main.py	Console report, CSV/JSON exports, command-line entry point

6. Problem Formulation and Mathematical Models

6.1 Input parameters (shared dataset)

Table 4. Input parameters

Symbol	Meaning	Value
P_avail	Available Arc Reactor output	1000 kW

P_crit	Critical load of FLC + STB + SEN + LSC	420 kW
P_req	Throughput every route edge must carry	600 kW
Θ_route	Cumulative thermal limit of the route	14.0 °C
Θ_armor	Armor thermal limit (route + nanomaterial heat)	30.0 °C
w_L, w_T, w_S	Edge cost weights (loss, thermal, switching)	1.0, 0.5, 0.4
R	Nanomaterial reserve	1200 NU
ρ_o, Δρ, ρ_min	Initial nano power fraction, replanning step, minimum	0.8, 0.1, 0.4
H, Δt	Scheduling horizon and slot length	16 slots, 30 min
N_tech	Technicians available	2

6.2 Overall engineering objective and mandatory constraints

The overall problem is a staged (hierarchical) optimisation. Optimisation objectives are distinguished from mandatory safety constraints: objectives are preferences JARVIS trades off, constraints may never be violated, and any plan violating one is rejected regardless of its objective value.

$$\text{Stage 1:}\quad \pi^* = \operatorname{argmin}_\{\pi \in \Pi_\text{safe}\}\ C(\pi) \qquad (1)$$

$$\text{Stage 2:}\quad x^* = \operatorname{argmax}_\{x \in X(\pi^*, \rho)\}\ J(x) \qquad (2)$$

$$\text{Stage 3:}\quad \text{find } S \in \mathcal{S}(d(\pi^*, x^*), P_\text{res}(\pi^*, x^*)) \quad (\text{then minimise makespan}) \qquad (3)$$

$$F(\pi, x, S) = g_\text{UCS}(\pi) \wedge g_\text{SA}(x \mid \pi) \wedge g_\text{CSP}(S \mid \pi, x) \wedge g_\text{INT}(\pi, x, S) \qquad (4)$$

Equation (4) is the integrated feasibility condition: a recovery plan is accepted only if F = 1. The lexicographic order (route first, then allocation, then schedule) reflects engineering priority: a safe energy path is a prerequisite for everything else.

Table 5. Optimisation objectives versus mandatory safety constraints

Type	Item	Where enforced
Objective	Minimise route cost C(π) (losses, heat, switching)	UCS
Objective	Maximise readiness J(x) (protection, stability, repair, efficiency)	SA
Objective	Minimise makespan of the recovery schedule	CSP horizon tightening
Mandatory	Never route through FAILED components	UCS expansion
Mandatory	Edge capacity ≥ P_req; route heat ≤ Θ_route	UCS expansion
Mandatory	Minimum structural protection, reserve, power cap ρB, armor heat ≤ Θ_armor	SA feasible set
Mandatory	Precedence, separation, no-overlap, exclusivity, compatibility, power, technicians, deadlines	CSP
Mandatory	Power balance, thermal balance, replenishment, recharge, deadline	Integrated validation

6.3 Stage 1 model: weighted graph and UCS

The network is a directed graph G = (V, E) with |V| = 16 nodes (12 on the power side plus 4 critical loads) and |E| = 18 power edges. Each edge e carries a conversion loss L_e (kW), a thermal rise T_e (°C), a

switching overhead S_e (ms) and a capacity cap_e (kW). The non-negative edge cost and the UCS cumulative cost are:

$$c(e) = w_L \cdot L_e + w_T \cdot T_e + w_S \cdot S_e \geq 0 \qquad (5)$$

$$g(n') = g(n) + c(n, n'), \quad g(ARC) = 0 \qquad (6)$$

$$\theta(n') = \theta(n) + T_(n,n'), \quad \text{valid only if } \theta(n') \leq \Theta_route \text{ and } cap_(n,n') \geq P_req \qquad (7)$$

The power delivered by route π and the budget passed to Stage 2 are:

$$P_del(\pi) = \min(\, P_avail - \Sigma_\{e \in \pi\}\, L_e \,,\ \min_\{e \in \pi\}\, cap_e \,) \qquad (8)$$

$$B(\pi) = P_del(\pi) - P_crit, \quad \eta(\pi) = P_del(\pi) \,/\, P_avail \qquad (9)$$

Because θ is a resource that accumulates along the path, the cheapest partial path to a node may be too hot to complete while a costlier, cooler one succeeds. Classical UCS keeps only one path per node and could therefore miss the constrained optimum. We search over labels (n, g, θ) and discard a label only if an already-expanded label at the same node dominates it (g' ≤ g and θ' ≤ θ). Labels are popped in non-decreasing g, so the first goal label popped is optimal over all constraint-satisfying routes.

Figure 2. Weighted Arc Reactor power-distribution network and UCS optimal route

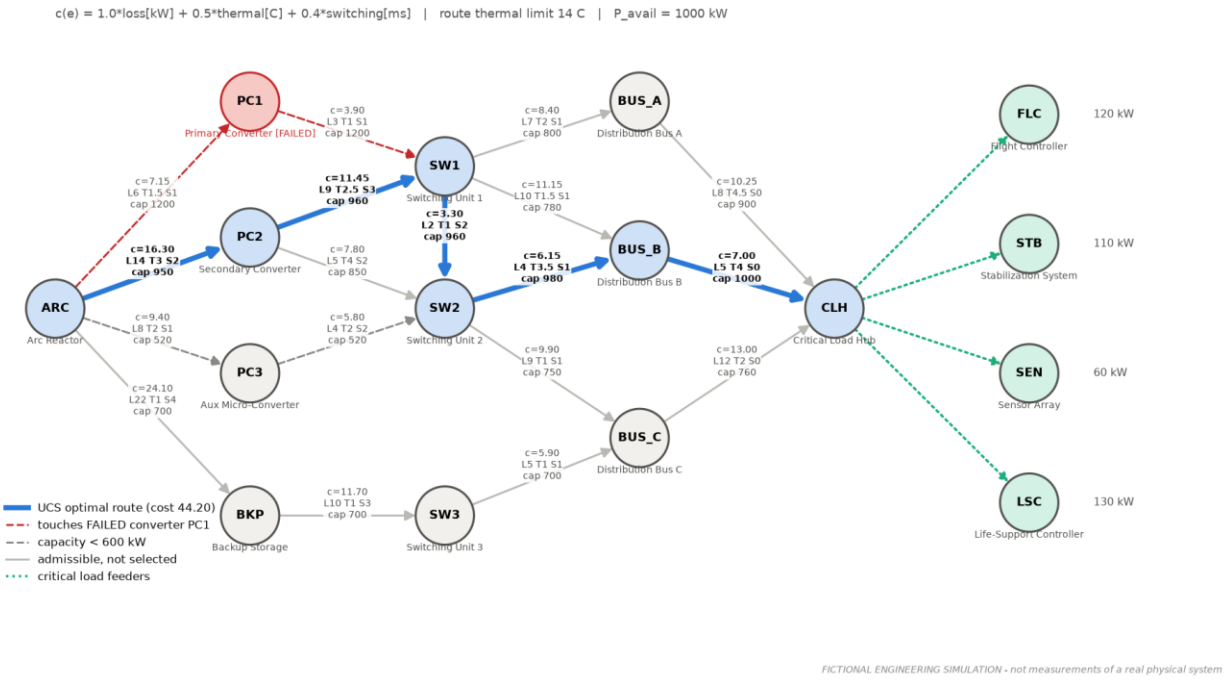


Figure 2. Weighted Arc Reactor power network (graph representation) with the UCS optimal route highlighted

6.4 Stage 2 model: nanomaterial allocation

Decision vector x = (x_SP, x_ES, x_FS, x_SR, x_SN) ∈ $\mathbb{Z}^5$ (NU) for structural protection, emergency shielding, flight stabilisation, structural repair and sensor protection.

$$J(x) = 100 \cdot \Sigma_i\, w_i(1 - e\text{^}(-x_i/s_i)) - \lambda_E \cdot P_nano(x)/B - \lambda_M \cdot M(x)/R - \lambda_W \cdot \Sigma_i\, (x_i \bmod q_i)/q_i \qquad (10)$$

$$P_nano(x) = \Sigma_i\, p_i x_i, \quad \Theta_nano(x) = \Sigma_i\, h_i x_i, \quad M(x) = \Sigma_i\, x_i \qquad (11)$$

The first term rewards protection, stability, repair and sensor protection with diminishing returns (saturation s_i); the second penalises energy consumption; the third rewards material efficiency; the fourth penalises incomplete nanite packets of size q_i (nanoparticles form plates in packets), which makes the landscape rugged with many local optima. The feasible set X(π, ρ) is:

$$m_i \le x_i \le u_i,\ \ M(x) \le R,\ \ P_nano(x) \le \rho \cdot B(\pi),\ \ \theta_route(\pi) + \Theta_nano(x) \le \Theta_armor \qquad (12)$$

$$\text{Metropolis acceptance: } P(\text{accept}) = 1 \text{ if } \Delta E \le 0, \text{ otherwise } \exp(-\Delta E / T),\ \ E = -J \qquad (13)$$

$$\text{Geometric cooling: } T_\{k+1\} = \alpha \cdot T_k \text{ after every } L \text{ iterations} \qquad (14)$$

Table 6. Nanomaterial category parameters

i	Category	min m_i	max u_i	p_i kW/NU	h_i °C/NU	w_i	s_i	q_i
SP	Structural Protection	300	700	0.3	0.01	0.3	260	40
ES	Emergency Shielding	80	400	0.6	0.018	0.2	150	30
FS	Flight Stabilization	60	350	0.7	0.02	0.2	120	25
SR	Structural Repair	100	500	0.45	0.012	0.2	200	50
SN	Sensor Protection	40	250	0.35	0.008	0.1	80	20

Objective weights: $\lambda_E = 12.0$, $\lambda_M = 6.0$, $\lambda_W = 1.2$. SA parameters: $T_o = 4.0$, $\alpha = 0.92$, $L = 40$, $T_min = 0.005$, max iterations = 4000, stall limit = 25 epochs, move size $\delta \in [1, 15]$ NU.

6.5 Stage 3 model: CSP

Variables: one variable per task k ∈ {ARC_DIAG, CONV_REPAIR, NANO_REPL, STRUCT_REST, SENSOR_CAL, FLC_SYNC, CHARGE, THERMAL_INSP, SAFETY_TEST}. Value: the pair (s_k, r_k) of start slot and the concrete resource units used (robotic arm, station, bay, charger, test rig). Domain:

$$D_k = \{\ (s, r) : 0 \le s,\ s + d_k \le \min(H, \text{deadline_k}),\ r \in \Pi_\text{type}\ \text{Compatible_k}(\text{type})\ \} \qquad (15)$$

Equipment compatibility is a unary constraint folded into D_k (for example, structural restoration needs the nano-capable ARM-1, thermal inspection needs the thermally shielded BAY-2). Hard constraints:

$$\text{C1 precedence:} \quad s_j \ge s_i + d_i \qquad \forall\, i \in \text{after}(j) \qquad (16)$$

$$\text{C2 separation:} \quad s_THERM \ge s_CHARGE + d_CHARGE + 1 \ \ (\text{cool-down}) \qquad (17)$$

$$\text{C3 no-overlap:} \quad [s_i, s_i+d_i) \cap [s_j, s_j+d_j) = \emptyset \ \ \text{for safety pairs} \qquad (18)$$

$$\text{C4 exclusivity:} \quad r_i \cap r_j \ne \emptyset \ \Rightarrow\ \text{intervals disjoint} \qquad (19)$$

$$\text{C5 power:} \quad \Sigma_\{k \text{ active at } t\}\ p_k \le P_res = B - P_nano(x^*) \quad \forall t \qquad (20)$$

$$\text{C6 technicians:} \quad \Sigma_\{k \text{ active at } t\}\ tech_k \le N_tech \qquad \forall t \qquad (21)$$

$$\text{C7 deadlines:} \quad s_k + d_k \le \text{deadline_k} \le H \qquad (22)$$

Task durations are functions of the earlier stages, which makes the coupling explicit:

$$d_STRUCT = \lceil x_SR/80 \rceil,\ d_REPL = \lceil M(x)/400 \rceil,\ d_SENSOR = 1 + \lceil x_SN/100 \rceil,\ d_FLC = 1 + \lceil x_FS/150 \rceil \qquad (23)$$

$$d_THERM = 1 + \lceil (\theta_route + \Theta_nano)/15 \rceil,\ \ d_CHARGE = \lceil E_deficit\ /\ (P_charger \cdot \eta(\pi) \cdot \Delta t) \rceil \qquad (24)$$

6.6 Relationship between the stages

The stages are linked by equations (8)–(9) and (20)–(24). Changing the route changes B, θ_route and η; B and θ_route reshape the SA feasible set (12); the SA result fixes P_res and every material-dependent duration; those durations and P_res define the CSP. Because the CSP enforces Σp_k ≤ B − P_nano at

every slot, the integrated power balance P_crit + P_nano + max_t P_lab(t) ≤ P_del holds by construction, and the validation engine re-checks it independently.

7. The Three Algorithms and Why All Three Are Needed

7.1 Uniform Cost Search

UCS expands the frontier node with the lowest path cost g(n), using a binary heap (heapq). With non-negative costs it is complete and optimal. Its responsibility here is to find the safest, cheapest energy path; its output (B, η, θ_route) is the physical envelope for everything that follows. UCS is the right tool because the problem is a pure additive-cost path problem on a small graph, where exact optimality is cheap. Heuristic search (A*) would need an admissible heuristic that brings little benefit on a graph of this size.

7.2 Simulated Annealing

SA is a stochastic local search that accepts worse moves with probability exp(−ΔE/T), allowing it to leave local optima while the temperature is high and to behave like hill climbing as T → 0. Its responsibility is the nanomaterial allocation: a large discrete space (on the order of 10^{12} integer vectors) with a non-convex objective caused by packet quantisation, where exhaustive search is impractical and greedy improvement gets trapped.

7.3 Constraint Satisfaction

A CSP expresses the scheduling problem declaratively as variables, domains and constraints. Backtracking with MRV (choose the most constrained variable), the degree heuristic (break ties by the number of constraints on unassigned variables) and forward checking (remove values that conflict with the new assignment, detect dead ends early) solves it systematically. If a schedule exists it will be found, and if none exists that will be proven. Its responsibility is to produce a conflict-free laboratory plan.

7.4 Why a single algorithm is not adequate

Table 7. Why three algorithms are necessary

If only…	What goes wrong
UCS for everything	The joint state (route × allocation × schedule) is astronomically large; UCS would have to enumerate it with no useful cost ordering for allocation or scheduling.
SA for everything	Hard scheduling and routing constraints would become penalty terms; SA gives no guarantee that the final plan is feasible and no proof of infeasibility.
CSP for everything	Routing cost and readiness are optimisation goals, not constraints; a CSP finds any feasible answer and would need costly branch-and-bound wrappers to optimise them.
Three independent runs	Locally optimal answers can be jointly infeasible. The executed mission shows this: the first SA allocation was optimal for its stage but left too little power for armor charging.

8. Integrated Methodology and Data Flow

JARVIS executes one mission loop (Figure 3). Each arrow carries a concrete quantity rather than a narrative link:

Table 8. Data exchanged between the stages

From → To	Data exchanged	Executed value
Dataset → UCS	graph, P_avail, P_req, Θ_route, failed components	18 edges, PC1 FAILED
UCS → SA	power budget B, route heat θ_route	B = 530.0 kW, θ = 14.0 °C
UCS → CSP	route efficiency η (charging rate)	η = 0.950
SA → CSP	allocation x*, P_res, material used, total heat	P_res = 160.0 kW, M = 841 NU, θ_total = 24.78 °C
CSP → SA (feedback)	infeasibility certificate → lower ρ	ρ 0.80 → 0.70
SA → UCS (feedback)	minimum allocation infeasible → exclude edge, re-route	not needed in nominal run
All → Validation	route, allocation, schedule	36/36 checks pass

Figure 3. End-to-end workflow and data exchanged between UCS, SA and CSP

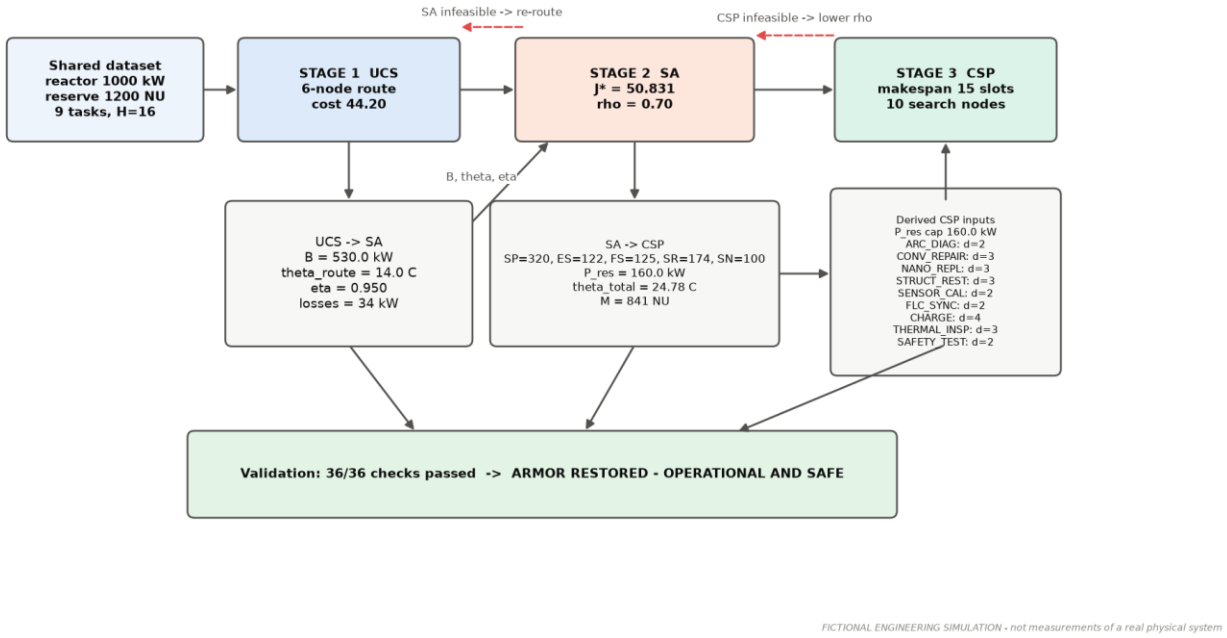


Figure 3. End-to-end workflow and the data exchanged among the three algorithms

8.1 Controlled replanning policy

- **UCS finds no route:** abort and hold the armor in safe mode (no unsafe plan is ever produced).
- **SA cannot meet mandatory minimums under ρ·B:** the route is too weak; exclude its highest-loss edge and re-run UCS for the next-best route.
- **CSP infeasible (precheck or exhausted search):** lower ρ by 0.1 to free residual power for laboratory work and re-run SA and CSP; if ρ would fall below 0.4, change route.
- **Feasible plan found:** tighten the horizon to minimise the makespan, then validate and commit.

Infeasibility is detected cheaply before search through necessary conditions: a single task drawing more than P_res, a critical-path length exceeding a deadline, or total energy Σp_k·d_k exceeding P_res·H. The stress scenario (python main.py --scenario pc2-failure) fails the secondary converter as well. In that run UCS falls back to the backup-storage route (B = 280 kW). SA proves that even the mandatory minimum

allocation needs 239 kW, which is more than the 224 kW cap. JARVIS then excludes the route and finds that no admissible route remains, so it aborts safely instead of returning an unsafe plan.

9. Pseudocode

9.1 Integrated JARVIS mission

```
procedure JARVIS_RECOVERY(dataset D)
    rho <- rho0 ; excluded <- {}
    for attempt = 1 .. maxAttempts do
        route <- UCS(D.network, excluded)                       # Stage 1
        if route = FAIL then return ABORT_SAFE_MODE
        B <- min(P_avail - losses(route), bottleneck(route)) - P_crit
        x <- SIMULATED_ANNEALING(D.nano, B, rho, heat(route))   # Stage 2
        if x = INFEASIBLE then excluded <- excluded + worstEdge(route); continue
        req <- DERIVE_REQUIREMENTS(route, x)                     # durations, P_res
        S <- CSP_BACKTRACK(req, D.lab)                           # Stage 3
        if S = FAIL then
            if rho - step >= rho_min then rho <- rho - step
            else excluded <- excluded + worstEdge(route); rho <- rho0
            continue
        S <- MINIMISE_MAKESPAN(req, S)
        if VALIDATE(route, x, S) then return (route, x, S, RESTORED)
    return FAILED
```

9.2 Stage 1 – label-setting Uniform Cost Search

```
function UCS(G, source, goal, P_req, Theta):
    frontier <- min-heap {(g=0, node=source, path=[source], theta=0)}
    expanded <- map node -> list of (g, theta)
    while frontier not empty:
        (g, n, path, theta) <- POP-MIN(frontier)
        if some (g', t') in expanded[n] with t' <= theta: continue      # dominated
        expanded[n].append((g, theta))
        if n = goal: return path, g                                     # optimal
        for each edge e = (n, m):
            if m FAILED or m in path or cap(e) < P_req: reject
            theta' <- theta + T(e);  if theta' > Theta: reject
            g' <- g + c(e)
            if not dominated(expanded[m], g', theta'): PUSH(frontier, (g', m, path+[m], theta'))
    return FAIL
```

9.3 Stage 2 – Simulated Annealing

```
function SA(problem, T0, alpha, L, Tmin, maxIter, seed):
    x <- mandatory minimums ;  if not feasible(x): return INFEASIBLE
    best <- x ; T <- T0
    while T >= Tmin and iter < maxIter and not stalled:
        repeat L times:
            y <- random feasible neighbour of x (transfer / add / remove delta NU)
            dE <- E(y) - E(x)                       # E = -J
            if dE <= 0 or rand() < exp(-dE / T): x <- y
            if E(x) < E(best): best <- x
        T <- alpha * T
    return best
```

9.4 Stage 3 – Backtracking with MRV, degree heuristic and forward checking

```
function BACKTRACK(assignment, domains):
    if assignment complete: return assignment
    v <- unassigned variable with smallest |domain|  (tie: largest degree)     # MRV + degree
    for value in domains[v] ordered by earliest start:
        if cumulative power / technicians still within limits:
            assign v = value
            for each unassigned u:                                              # forward checking
                remove values of u inconsistent with v = value
                if domain(u) empty: undo, try next value                       # wipeout
            result <- BACKTRACK(assignment, domains)
            if result != FAIL: return result
            restore pruned domains; unassign v
    return FAIL                                                                 # backtrack
```

10. Python Implementation

10.1 Project structure

```
jarvis_recovery/
├── main.py                    # entry point: runs the full mission, exports, figures
├── server.py                  # local web server for the interactive case study page
├── requirements.txt
├── README.md
├── data/
│   └── mission_dataset.json   # ONE shared dataset (fictional)
├── jarvis/
│   ├── __init__.py
│   ├── data_models.py         # shared data models and stage result objects
│   ├── ucs.py                 # Stage 1 – Uniform Cost Search
│   ├── annealing.py           # Stage 2 – Simulated Annealing
│   ├── requirements.py        # bridge UCS + SA -> CSP inputs
│   ├── csp.py                 # Stage 3 – CSP (MRV, degree, forward checking)
│   ├── pipeline.py            # orchestration + controlled replanning
│   ├── validation.py          # independent validation
│   ├── visualization.py       # figures + dashboard
│   ├── experiments.py         # performance study
│   ├── reporting.py           # console report + CSV/JSON exports
│   └── api.py                 # JSON interface used by the web page
├── web/                       # case study page and report page (React)
├── tests/
│   ├── test_mission.py        # unit + integration tests
│   └── test_api.py            # interface and server tests
├── scripts/
│   └── build_case_study.py    # builds this document from the real outputs
├── docs/                      # case study brief, this document (DOCX, PDF)
└── outputs/                   # generated: results.json, CSVs, logs, figures/
```

10.2 Installation and execution commands

```
# Python 3.10+ (tested with Python 3.11.9 on Windows 11)
cd jarvis_recovery
python -m pip install -r requirements.txt

python main.py                          # full mission + performance study + figures
python main.py --quick                  # mission + figures, skip multi-seed study
python main.py --no-plots               # console only
python main.py --seed 7 --out runs/s7   # different SA seed / output folder
python main.py --scenario pc2-failure   # stress test: secondary converter also fails
python -m unittest discover -s tests -v # 19 unit, integration and interface tests
python scripts/build_case_study.py              # regenerate this document from outputs/
```

10.3 Key implementation decisions

- **Priority queue:** heapq with a monotone tie-breaking counter so that equal-cost labels never compare paths.
- **Reproducibility:** SA uses random.Random(seed) with seed 85 from the dataset; the CSP and UCS are deterministic.
- **Feasibility-preserving SA:** infeasible neighbours are rejected and resampled, so every accepted state (including accepted worse states) is a valid allocation.
- **No hardcoded answers:** routes, allocations, durations and schedules are all computed; changing the JSON changes every downstream result, the figures and this document.
- **Independent validation:** validation.py recomputes costs, sums, profiles and overlaps from the raw decision variables and compares the UCS answer with an exhaustive enumeration of all simple routes.

The complete source code is reproduced in Appendix A.

11. Experimental Setup and Common Input Dataset

All experiments were executed with Python 3.11.9 on Windows 11 using matplotlib and networkx. The single shared dataset is data/mission_dataset.json (Appendix B). Mean wall-clock times measured in the run: UCS 0.118 ms, SA 54.1 ms per run, CSP 1.01 ms.

11.1 Power-network edges

Table 9. Weighted power-distribution edges (fictional)

Edge	Loss kW	Thermal °C	Switch ms	Capacity kW	Cost c(e)
ARC→PC1	6	1.5	1	1200	7.15
ARC→PC2	14	3	2	950	16.30
ARC→PC3	8	2	1	520	9.40
ARC→BKP	22	1	4	700	24.10
PC1→SW1	3	1	1	1200	3.90
PC2→SW1	9	2.5	3	960	11.45
PC2→SW2	5	4	2	850	7.80
PC3→SW2	4	2	2	520	5.80
BKP→SW3	10	1	3	700	11.70
SW1→SW2	2	1	2	960	3.30
SW1→BUS_A	7	2	1	800	8.40
SW1→BUS_B	10	1.5	1	780	11.15
SW2→BUS_B	4	3.5	1	980	6.15
SW2→BUS_C	9	1	1	750	9.90
SW3→BUS_C	5	1	1	700	5.90
BUS_A→CLH	8	4.5	0	900	10.25
BUS_B→CLH	5	4	0	1000	7.00
BUS_C→CLH	12	2	0	760	13.00

11.2 Laboratory resources and tasks

Table 10. Laboratory resources

Type	Units (capabilities)
ARM	ARM-1 (nano, standard); ARM-2 (precision, standard)
STATION	RS-E (electrical); RS-N (nanofab)
BAY	BAY-1 (standard, charging-dock); BAY-2 (standard, thermal-shielded)
CHARGER	CHG-1 (armor-charging)
TESTRIG	TEST-RIG (diagnostics)
Technicians	2

Table 11. Recovery tasks

Task	Power kW	Tech	Needs	After	Deadline	Duration rule
ARC_DIAG	40	1	BAY:standard, TESTRIG:diagnostics	–	–	fixed:2
CONV_REPAIR	60	1	ARM:standard, STATION:electrical	ARC_DIAG	8	converter_repair
NANO_REPL	70	0	ARM:nano, STATION:nanofab	STRUCT_REST	–	replenishment
STRUCT_REST	90	1	ARM:nano, BAY:standard	ARC_DIAG	–	struct_restoration
SENSOR_CAL	30	0	ARM:precision, BAY:standard, TESTRIG:diagnostics	STRUCT_REST	–	sensor_calibration
FLC_SYNC	35	1	TESTRIG:diagnostics	CONV_REPAIR, SENSOR_CAL	–	flight_sync
CHARGE	120	0	BAY:charging-dock, CHARGER:armor-charging	CONV_REPAIR	–	charging
THERMAL_INSP	20	1	BAY:thermal-shielded	CHARGE	–	thermal_inspection
SAFETY_TEST	100	2	BAY:thermal-shielded, TESTRIG:diagnostics	FLC_SYNC, THERMAL_INSP, NANO_REPL	16	fixed:2

Safety rules: CHARGE → THERMAL_INSP requires a 1-slot cool-down; STRUCT_REST and CHARGE may not overlap (no nanite fabrication during high-current charging); ARC_DIAG and CHARGE may not overlap (diagnostics require an isolated reactor). Armor battery: 400 kWh at 35% charge, target 90%, charger 120 kW.

12. Step-by-Step Execution Trace

The following traces are taken verbatim from the executed run (outputs/console_log.txt, outputs/sa_iterations.csv, outputs/csp_trace.txt).

12.1 Replanning log of the mission

Table 12. Planning attempts

#	ρ	Route	Allocation (NU)	P_res kW	Outcome	JARVIS action
1	0.80	ARC → PC2 → SW1 → SW2 → BUS_B → CLH	SP=360, ES=151, FS=125, SR=200, SN=120	111.9	CSP infeasible: CHARGE draws 120 kW > available P_res 111.9 kW; empty domain for CHARGE	reduce nanomaterial power fraction rho 0.80 -> 0.70 to free laboratory power; re-run SA and CSP
2	0.70	ARC → PC2 → SW1 → SW2 → BUS_B → CLH	SP=320, ES=122, FS=125, SR=174, SN=100	160.0	feasible	validate and commit

Attempt 1 is a genuine infeasibility discovered by the integration. SA maximised readiness by spending almost all of the 80 % power cap on nanomaterial, which left P_res = 111.9 kW. Armor charging alone

draws 120 kW, so the CSP precheck proved that no schedule could exist. JARVIS lowered ρ to 0.70, and SA re-optimised under the tighter cap. P_res rose to 160.0 kW and the CSP became solvable.

12.2 Stage 1 – UCS expansion trace

Table 13. UCS node expansion order, frontier changes and cumulative costs

Step	Expand	g	θ	Pushed	Rejected	Frontier after (g)
1	ARC	0.0	0.0	PC2 g=16.3 θ=3.0; BKP g=24.1 θ=1.0	PC1: PC1 FAILED (FRIDAY fault); PC3: capacity 520 < required 600 kW	PC2(16.3), BKP(24.1)
2	PC2	16.3	3.0	SW1 g=27.75 θ=5.5; SW2 g=24.1 θ=7.0	–	BKP(24.1), SW2(24.1), SW1(27.75)
3	BKP	24.1	1.0	SW3 g=35.8 θ=2.0	–	SW2(24.1), SW1(27.75), SW3(35.8)
4	SW2	24.1	7.0	BUS_B g=30.25 θ=10.5; BUS_C g=34.0 θ=8.0	–	SW1(27.75), BUS_B(30.25), BUS_C(34.0), SW3(35.8)
5	SW1	27.75	5.5	SW2 g=31.05 θ=6.5; BUS_A g=36.15 θ=7.5; BUS_B g=38.9 θ=7.0	–	BUS_B(30.25), SW2(31.05), BUS_C(34.0), SW3(35.8), BUS_A(36.15), BUS_B(38.9)
6	BUS_B	30.25	10.5	–	CLH: thermal 14.5 > limit 14.0 C	SW2(31.05), BUS_C(34.0), SW3(35.8), BUS_A(36.15), BUS_B(38.9)
7	SW2	31.05	6.5	BUS_B g=37.2 θ=10.0; BUS_C g=40.95 θ=7.5	–	BUS_C(34.0), SW3(35.8), BUS_A(36.15), BUS_B(37.2), BUS_B(38.9), BUS_C(40.95)
8	BUS_C	34.0	8.0	CLH g=47.0 θ=10.0	–	SW3(35.8), BUS_A(36.15), BUS_B(37.2), BUS_B(38.9), BUS_C(40.95), CLH(47.0)
9	SW3	35.8	2.0	BUS_C g=41.7 θ=3.0	–	BUS_A(36.15), BUS_B(37.2), BUS_B(38.9), BUS_C(40.95), BUS_C(41.7), CLH(47.0)
10	BUS_A	36.15	7.5	CLH g=46.4 θ=12.0	–	BUS_B(37.2), BUS_B(38.9), BUS_C(40.95), BUS_C(41.7), CLH(46.4), CLH(47.0)
11	BUS_B	37.2	10.0	CLH g=44.2 θ=14.0	–	BUS_B(38.9), BUS_C(40.95), BUS_C(41.7), CLH(44.2), CLH(46.4), CLH(47.0)

12	BUS_B	38.9	7.0	CLH g=45.9 θ=11.0	–	BUS_C(40.95), BUS_C(41.7), CLH(44.2), CLH(45.9), CLH(46.4), CLH(47.0)
13	BUS_C	40.95	7.5	CLH g=53.95 θ=9.5	–	BUS_C(41.7), CLH(44.2), CLH(45.9), CLH(46.4), CLH(47.0), CLH(53.95)
14	BUS_C	41.7	3.0	CLH g=54.7 θ=5.0	–	CLH(44.2), CLH(45.9), CLH(46.4), CLH(47.0), CLH(53.95), CLH(54.7)
15	CLH GOAL	44.2	14.0	–	–	CLH(45.9), CLH(46.4), CLH(47.0), CLH(53.95), CLH(54.7)

Observations: at step 1 PC1 is rejected because it FAILED and PC3 because its 520 kW capacity cannot carry 600 kW. SW2 is expanded twice (steps 4 and 7) and BUS_B three times. These are distinct, mutually non-dominated labels: the cheaper label via PC2→SW2 reaches BUS_B at θ = 10.5 °C, and its extension to CLH (θ = 14.5 °C) violates the 14 °C limit (step 6). The costlier but cooler label via SW1→SW2 completes at exactly θ = 14.0 °C. The goal is popped at step 15 with g* = 44.20, after 19 labels were generated.

Figure 4. UCS node-expansion order with cumulative cost g(n) (non-decreasing)

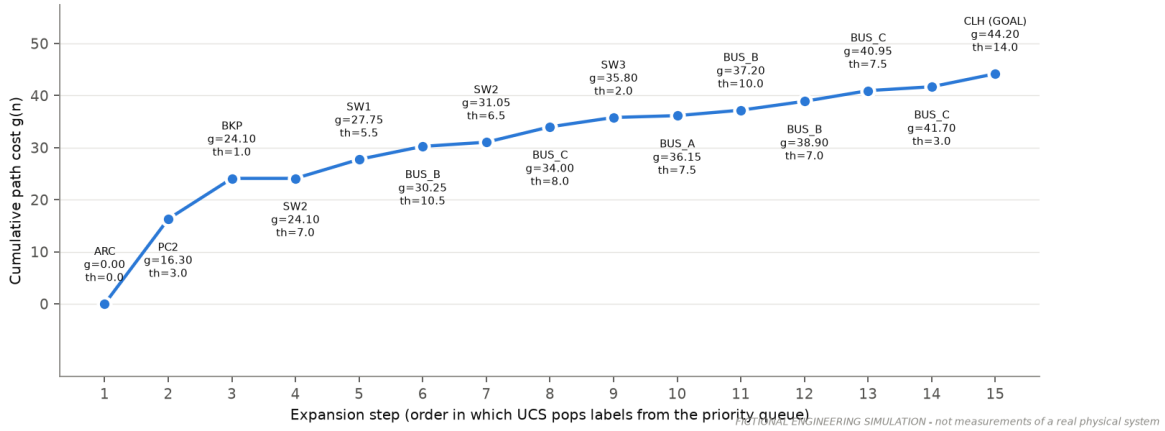


Figure 4. UCS expansion order; g is non-decreasing, as UCS guarantees

Result: route ARC → PC2 → SW1 → SW2 → BUS_B → CLH; losses 34 kW; thermal 14 °C; switching 8 ms; bottleneck 950 kW. P_del = min(1000 − 34, 950) = 950 kW, so B = 950 − 420 = 530 kW and η = 0.950.

12.3 Stage 2 – Simulated Annealing iterations (final attempt)

Table 14. SA iterations 1–18 and every 400th iteration

Iter	T	Move	Cand. J	ΔE	P(acc)	Decision	Current J	Best J	SP/ES/FS/SR/SN
1	4.0000	add 4 NU to SP	38.264	+0.023	0.994	ACCEPT-WORSE	38.264	38.287	304/80/60/100/40
2	4.0000	add 13 NU to SN	38.226	+0.038	0.990	ACCEPT-WORSE	38.226	38.287	304/80/60/100/53

3	4.0000	add 7 NU to SP	38.181	+0.045	0.989	ACCEPT-WORSE	38.181	38.287	311/80/60/100/53
4	4.0000	move 3 NU SN->ES	38.259	-0.078	1.000	ACCEPT-BETTER	38.259	38.287	311/83/60/100/50
5	4.0000	remove 9 NU from SN	38.278	-0.019	1.000	ACCEPT-BETTER	38.278	38.287	311/83/60/100/41
6	4.0000	add 15 NU to SP	39.359	-1.082	1.000	ACCEPT-BETTER (new best)	39.359	39.359	326/83/60/100/41
7	4.0000	move 3 NU SP->FS	39.478	-0.119	1.000	ACCEPT-BETTER (new best)	39.478	39.478	323/83/63/100/41
8	4.0000	add 1 NU to FS	39.508	-0.029	1.000	ACCEPT-BETTER (new best)	39.508	39.508	323/83/64/100/41
9	4.0000	remove 4 NU from SP	38.341	+1.167	0.747	ACCEPT-WORSE	38.341	39.508	319/83/64/100/41
10	4.0000	add 9 NU to ES	39.683	-1.342	1.000	ACCEPT-BETTER (new best)	39.683	39.683	319/92/64/100/41
11	4.0000	remove 3 NU from ES	38.440	+1.243	0.733	REJECT	39.683	39.683	319/92/64/100/41
12	4.0000	add 14 NU to SN	39.624	+0.059	0.985	ACCEPT-WORSE	39.624	39.683	319/92/64/100/55
13	4.0000	move 11 NU ES->FS	39.715	-0.091	1.000	ACCEPT-BETTER (new best)	39.715	39.715	319/81/75/100/55
14	4.0000	remove 2 NU from SN	39.733	-0.019	1.000	ACCEPT-BETTER (new best)	39.733	39.733	319/81/75/100/53
15	4.0000	move 4 NU SP->ES	39.836	-0.103	1.000	ACCEPT-BETTER (new best)	39.836	39.836	315/85/75/100/53
16	4.0000	remove 10 NU from SP	39.904	-0.068	1.000	ACCEPT-BETTER (new best)	39.904	39.904	305/85/75/100/53
17	4.0000	move 11 NU SN->SR	40.164	-0.260	1.000	ACCEPT-BETTER (new best)	40.164	40.164	305/85/75/111/42
18	4.0000	move 2 NU ES->SN	40.129	+0.035	0.991	ACCEPT-WORSE	40.129	40.164	305/83/75/111/44
400	1.8886	move 7 NU SP->ES	46.178	-0.083	1.000	ACCEPT-BETTER	46.178	49.510	390/139/122/149/50
800	0.8204	move 1 NU ES->SN	47.145	+0.023	0.973	ACCEPT-WORSE	47.145	49.510	307/148/130/126/89
1200	0.3564	remove 2 NU from SN	49.268	-0.039	1.000	ACCEPT-BETTER	49.268	49.510	323/123/129/164/67
1600	0.1548	remove 5 NU from SR	49.441	+1.243	0.000	REJECT	50.684	50.766	320/130/126/150/100
2000	0.0672	move 14 NU SR->SP	50.466	+0.230	0.033	REJECT	50.696	50.803	332/122/125/166/100
2400	0.0292	remove 1 NU from ES	49.537	+1.202	0.000	REJECT	50.738	50.822	325/120/128/169/100
2800	0.0127	move 5 NU ES->SR	49.617	+1.196	0.000	REJECT	50.813	50.831	321/121/125/171/100
3200	0.0055	remove 4 NU from ES	49.576	+1.204	0.000	REJECT	50.780	50.831	323/123/126/169/100

The run started from the mandatory-minimum allocation (J_o = 38.287) and terminated because 'temperature 0.0047 < T_min 0.005' after 3240 iterations. It accepted 672 worse moves, rejected 890

infeasible neighbours and improved the best solution 44 times. Rows with ACCEPT-WORSE show the Metropolis rule in action: a move with ΔE > 0 was taken because a random draw fell below exp(−ΔE/T). These moves are what let SA cross the packet-waste barriers that trap hill climbing (Section 14).

12.4 Stage 3 – CSP inputs derived from Stages 1 and 2

Table 15. CSP task durations computed from the route and allocation

Task	Duration (slots)	Derivation from UCS / SA outputs	Power kW	Tech
ARC_DIAG	2	fixed procedure length	40	1
CONV_REPAIR	3	3 slots x 1 failed unit(s) ['PC1']	60	1
NANO_REPL	3	ceil(M=841 NU consumed / 400 NU per slot)	70	0
STRUCT_REST	3	ceil(x_SR=174 / 80 NU per slot)	90	1
SENSOR_CAL	2	1 + ceil(x_SN=100 / 100)	30	0
FLC_SYNC	2	1 + ceil(x_FS=125 / 150)	35	1
CHARGE	4	ceil(deficit 220 kWh / (120 kW x eta=0.950 x 0.5 h))	120	0
THERMAL_INSP	3	1 + ceil(theta_total=24.78 C / 15)	20	1
SAFETY_TEST	2	fixed procedure length	100	2

- Power cap for concurrent lab operations P_res = B - P_nano = 530.0 - 370.0 = 160.0 kW
- Route efficiency eta = 0.950 scales the armor charging rate
- Combined thermal load 24.78 C sets the thermal-inspection length

Initial bound propagation on the precedence network (before search) tightened the domains as follows:

```
bound: CONV_REPAIR start >= 2 (after ARC_DIAG) |D| 12->8
bound: ARC_DIAG start <= 3 (before CONV_REPAIR) |D| 30->8
bound: NANO_REPL start >= 3 (after STRUCT_REST) |D| 14->11
bound: STRUCT_REST start <= 10 (before NANO_REPL) |D| 28->22
bound: STRUCT_REST start >= 2 (after ARC_DIAG) |D| 22->18
bound: SENSOR_CAL start >= 5 (after STRUCT_REST) |D| 30->20
bound: FLC_SYNC start >= 5 (after CONV_REPAIR) |D| 15->10
bound: FLC_SYNC start >= 7 (after SENSOR_CAL) |D| 10->8
bound: SENSOR_CAL start <= 12 (before FLC_SYNC) |D| 20->16
bound: CHARGE start >= 5 (after CONV_REPAIR) |D| 13->8
bound: THERMAL_INSP start >= 10 (after CHARGE) |D| 14->4
bound: CHARGE start <= 8 (before THERMAL_INSP) |D| 8->4
bound: SAFETY_TEST start >= 9 (after FLC_SYNC) |D| 15->6
bound: FLC_SYNC start <= 12 (before SAFETY_TEST) |D| 8->6
bound: SAFETY_TEST start >= 13 (after THERMAL_INSP) |D| 6->2
bound: THERMAL_INSP start <= 11 (before SAFETY_TEST) |D| 4->2
bound: NANO_REPL start <= 11 (before SAFETY_TEST) |D| 11->9
bound: NANO_REPL start >= 5 (after STRUCT_REST) |D| 9->7
bound: STRUCT_REST start <= 8 (before NANO_REPL) |D| 18->14
bound: SENSOR_CAL start <= 10 (before FLC_SYNC) |D| 16->12
bound: CHARGE start <= 6 (before THERMAL_INSP) |D| 4->2
bound: STRUCT_REST start <= 7 (before SENSOR_CAL) |D| 14->12
bound: CONV_REPAIR start <= 3 (before CHARGE) |D| 8->4
bound: ARC_DIAG start <= 1 (before CONV_REPAIR) |D| 8->4
```

Domain sizes entering search: ARC_DIAG 4, CONV_REPAIR 4, NANO_REPL 7, STRUCT_REST 12, SENSOR_CAL 12, FLC_SYNC 6, CHARGE 2, THERMAL_INSP 2, SAFETY_TEST 2.

12.5 Stage 3 – CSP search trace

```
SELECT SAFETY_TEST (MRV |D|=2, degree=6)
ASSIGN SAFETY_TEST=(start 13, end 15, BAY-2/TEST-RIG)
```

```
FC: NANO_REPL 7->6, FLC_SYNC 6->5, THERMAL_INSP 2->1
SELECT THERMAL_INSP (MRV |D|=1, degree=4)
ASSIGN THERMAL_INSP=(start 10, end 13, BAY-2)
  FC: SENSOR_CAL 12->10, CHARGE 2->1
  SELECT CHARGE (MRV |D|=1, degree=4)
  ASSIGN CHARGE=(start 5, end 9, BAY-1/CHG-1)
    FC: CONV_REPAIR 4->2, NANO_REPL 6->2, STRUCT_REST 12->2, SENSOR_CAL 10->6
    SELECT CONV_REPAIR (MRV |D|=2, degree=5)
    ASSIGN CONV_REPAIR=(start 2, end 5, ARM-1/RS-E)
      FC WIPEOUT: domain of STRUCT_REST empty -> undo CONV_REPAIR
    ASSIGN CONV_REPAIR=(start 2, end 5, ARM-2/RS-E)
      FC: ARC_DIAG 4->2
      SELECT ARC_DIAG (MRV |D|=2, degree=3)
      ASSIGN ARC_DIAG=(start 0, end 2, BAY-1/TEST-RIG)
        SELECT STRUCT_REST (MRV |D|=2, degree=2)
        ASSIGN STRUCT_REST=(start 2, end 5, ARM-1/BAY-1)
          SELECT NANO_REPL (MRV |D|=2, degree=0)
          ASSIGN NANO_REPL=(start 9, end 12, ARM-1/RS-N)
            SELECT FLC_SYNC (MRV |D|=5, degree=1)
            ASSIGN FLC_SYNC=(start 7, end 9, TEST-RIG)
              FC: SENSOR_CAL 6->1
              SELECT SENSOR_CAL (MRV |D|=1, degree=0)
              ASSIGN SENSOR_CAL=(start 5, end 7, ARM-2/BAY-2/TEST-RIG)
```

MRV selected SAFETY_TEST first (2 values, degree 6), then THERMAL_INSP and CHARGE, which forward checking had reduced to a single value. Assigning ARM-1 to CONV_REPAIR caused a forward-checking wipeout: STRUCT_REST, which needs the nano-capable ARM-1 in the same window, lost every value. JARVIS undid that assignment and tried ARM-2, and the search then went straight to a solution. Totals: 10 nodes, 0 full backtracks, 1 wipeout, 37 pruned values, 424 constraint checks. Makespan tightening: H = 14: infeasible (critical path forces SAFETY_TEST to end at slot 15 > limit 14; empty domain for THERMAL_INSP; empty domain for SAFETY_TEST). The 15-slot schedule is therefore makespan-optimal.

13. Results

13.1 Stage 1 – optimal power route

The selected route ARC → PC2 → SW1 → SW2 → BUS_B → CLH is highlighted in Figure 2 (Section 6.3). Of the 4 search strategies compared in Section 14.3, only constrained UCS returns a route that is both cost-optimal and safe.

13.2 Stage 2 – nanomaterial allocation

Table 16. Nanomaterial allocation

Category	Minimum	Attempt 1 (ρ=0.80)	Final (ρ=0.70)	Power kW	Heat °C
SP Structural Protection	300	360	320	96.0	3.20
ES Emergency Shielding	80	151	122	73.2	2.20
FS Flight Stabilization	60	125	125	87.5	2.50
SR Structural Repair	100	200	174	78.3	2.09
SN Sensor Protection	40	120	100	35.0	0.80
Total	580	956	841	370.0	10.78

Final objective J* = 50.831, up from 38.287. The allocation uses 370.0 of 371.0 kW, so the power cap is the binding constraint. It uses 841/1200 NU and keeps the armor at 24.78 °C against the 30.0 °C limit. Every category stays above its mandatory minimum.

Figure 5. Optimised nanomaterial allocation for the damaged Mark 85

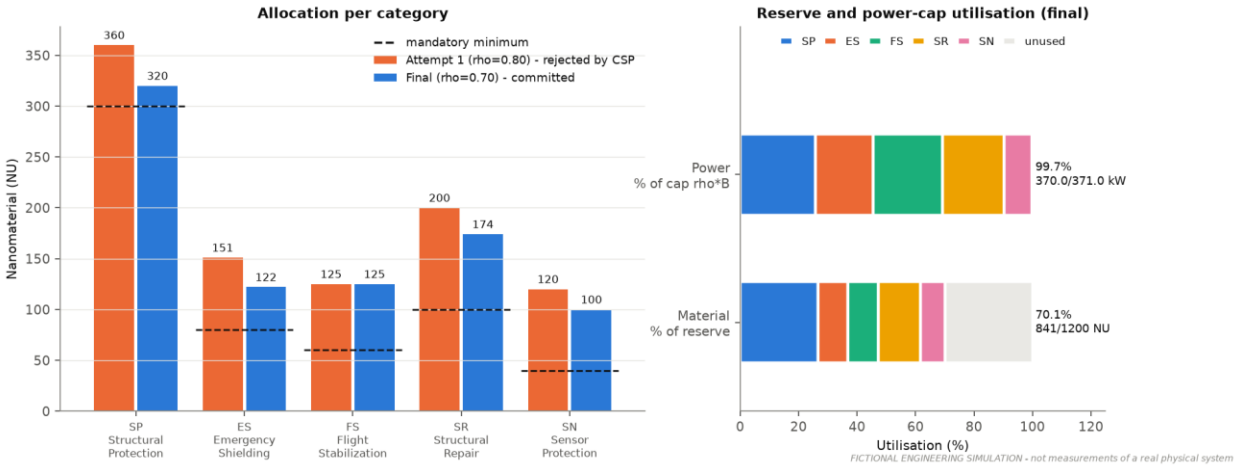


Figure 5. Nanomaterial allocation (rejected first attempt and final plan)

Figure 6. Simulated Annealing convergence and geometric cooling schedule

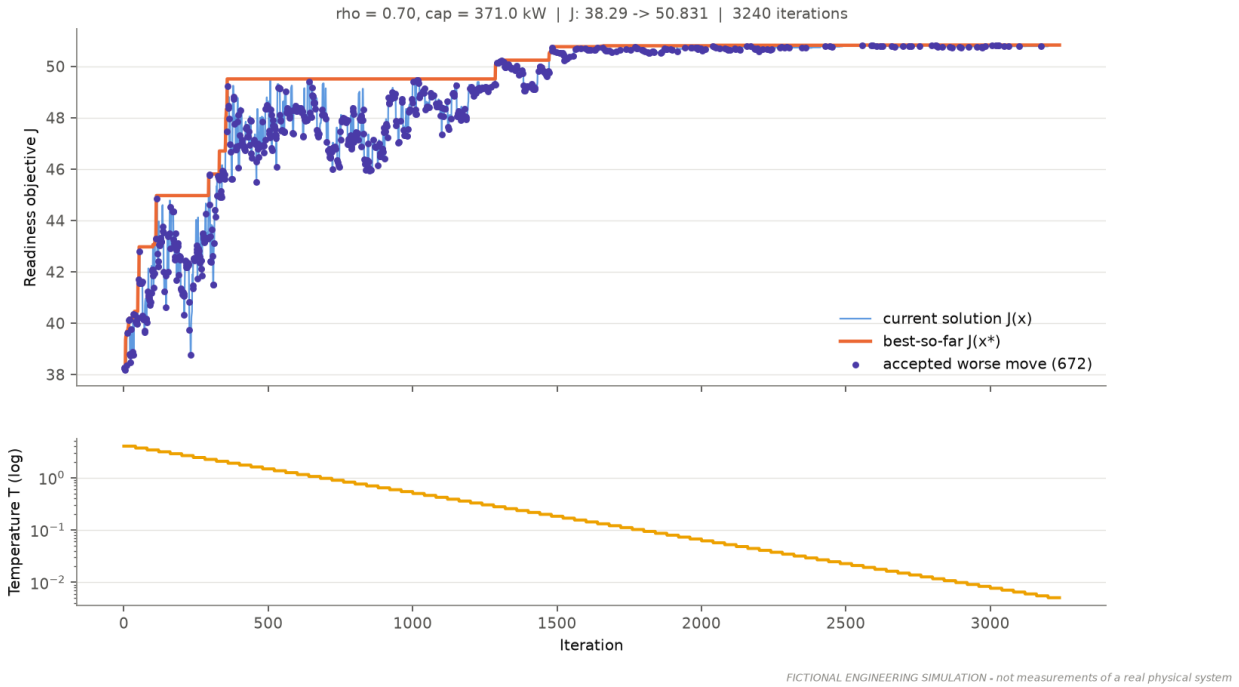


Figure 6. SA convergence: current and best objective; temperature schedule

13.3 Stage 3 – recovery schedule

Table 17. Final conflict-free laboratory schedule

Task	Name	Start	End	Clock (T+h:mm)	Resources	kW	Tech
ARC_DIAG	Arc Reactor Diagnostics	0	2	00:00–01:00	BAY-1, TEST-RIG	40	1
CONV_REPAIR	Power-Converter Repair	2	5	01:00–02:30	ARM-2, RS-E	60	1

STRUCT_REST	Structural Restoration	2	5	01:00–02:30	ARM-1, BAY-1	90	1
CHARGE	Armor Charging	5	9	02:30–04:30	BAY-1, CHG-1	120	0
SENSOR_CAL	Sensor Calibration	5	7	02:30–03:30	ARM-2, BAY-2, TEST-RIG	30	0
FLC_SYNC	Flight-Controller Synchronization	7	9	03:30–04:30	TEST-RIG	35	1
NANO_REPL	Nanomaterial Replenishment	9	12	04:30–06:00	ARM-1, RS-N	70	0
THERMAL_INSP	Thermal Inspection	10	13	05:00–06:30	BAY-2	20	1
SAFETY_TEST	Final Safety Testing	13	15	06:30–07:30	BAY-2, TEST-RIG	100	2

Figure 7. CSP constraint graph for the laboratory recovery schedule

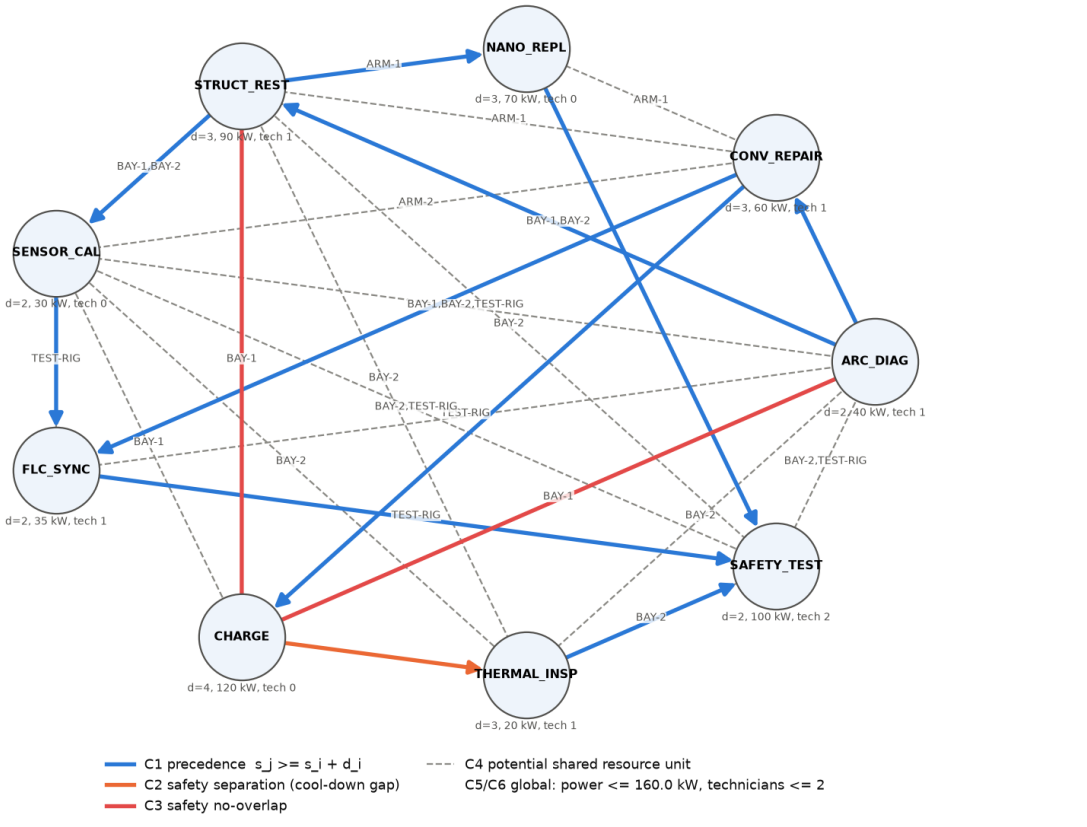


Figure 7. CSP constraint graph

Figure 8. Stark Laboratory recovery schedule (Gantt chart)

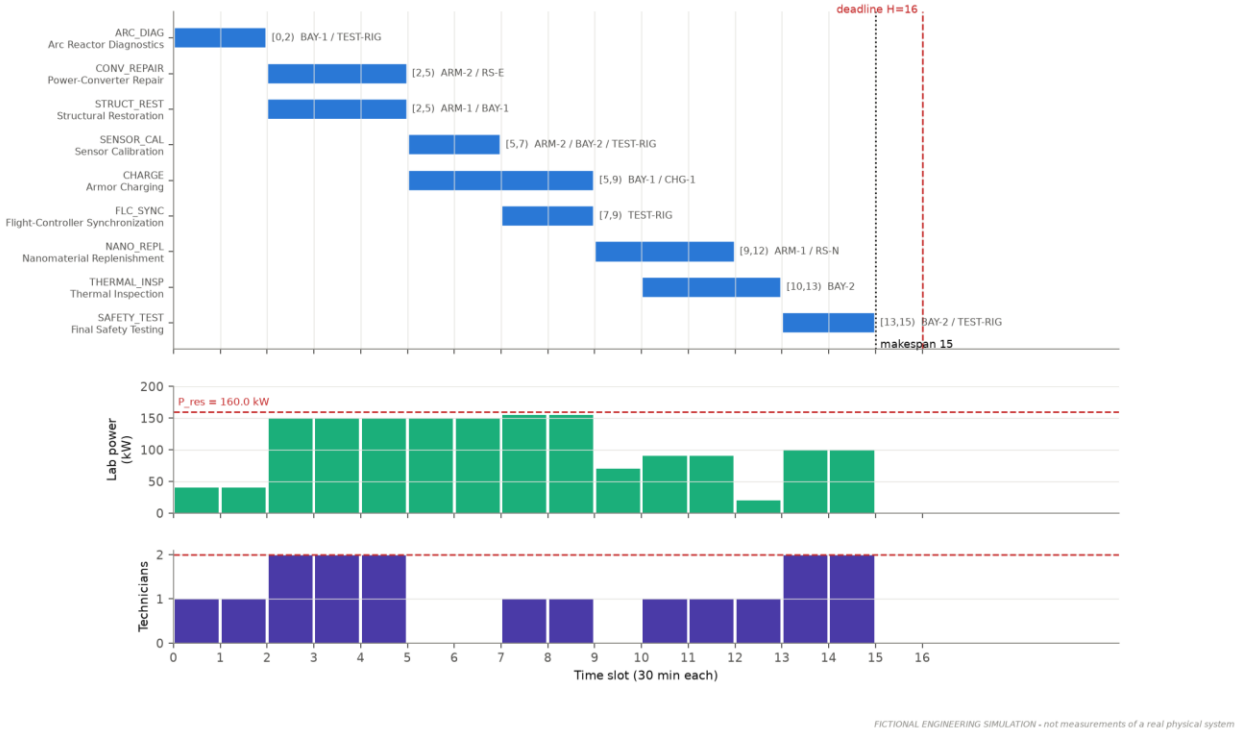


Figure 8. Gantt chart with power and technician profiles against their limits

13.4 Integrated validation and final recovery status

Table 18. Independent constraint validation

Stage	Check	Result	Detail
UCS	route found	PASS	ARC -> PC2 -> SW1 -> SW2 -> BUS_B -> CLH
UCS	starts at source / ends at goal	PASS	ARC ... CLH
UCS	avoids FAILED components	PASS	failed on route: none
UCS	capacity >= required throughput	PASS	bottleneck 950 kW >= 600 kW
UCS	route thermal <= limit	PASS	14.00 C <= 14.00 C
UCS	reported cost = recomputed cost	PASS	44.2000 vs 44.2000
UCS	optimal vs exhaustive enumeration	PASS	UCS 44.20 = best of 6 feasible routes 44.20
UCS	delivered power covers critical load	PASS	delivered 950.0 kW - critical 420 kW = budget 530.0 kW
SA	SP within [300, 700] NU	PASS	x_SP = 320
SA	ES within [80, 400] NU	PASS	x_ES = 122
SA	FS within [60, 350] NU	PASS	x_FS = 125
SA	SR within [100, 500] NU	PASS	x_SR = 174
SA	SN within [40, 250] NU	PASS	x_SN = 100
SA	material <= reserve	PASS	841 <= 1200 NU
SA	nano power <= rho * B	PASS	370.0 <= 371.0 kW (rho=0.70)
SA	armor thermal <= limit	PASS	24.78 <= 30.0 C
SA	reported objective = recomputed	PASS	J = 50.8307

SA	final J >= initial J	PASS	50.831 >= 38.287
CSP	schedule found	PASS	makespan 15 slots
CSP	every task scheduled	PASS	9/9 tasks
CSP	C1 precedence	PASS	all dependencies respected
CSP	C2 safety separation	PASS	cool-down gaps respected
CSP	C3 safety no-overlap	PASS	no forbidden overlaps
CSP	C4 resource exclusivity	PASS	no unit double-booked
CSP	equipment compatibility	PASS	all units capable
CSP	C5 power profile <= P_res	PASS	peak 155 kW <= 160.0 kW
CSP	C6 technicians	PASS	peak 2 <= 2
CSP	C7 deadlines / horizon	PASS	makespan 15 <= 16
CSP	durations match derived requirements	PASS	ok
INTEGRATED	power balance P_crit + P_nano + peak P_lab <= P_delivered	PASS	420 + 370.0 + 155 = 945.0 <= 950.0 kW
INTEGRATED	reactor balance P_delivered + losses <= P_avail	PASS	950 + 34 <= 1000 kW
INTEGRATED	thermal theta_route + theta_nano <= Theta_armor	PASS	14.00 + 10.78 = 24.78 <= 30.0 C
INTEGRATED	mandatory structural protection	PASS	x_SP = 320 >= 300 NU
INTEGRATED	consumed nanomaterial is replenished	PASS	841 NU replenished in slots 9-12
INTEGRATED	armor recharged to target state of charge	PASS	228.0 kWh delivered >= 220.0 kWh deficit
INTEGRATED	mission completes before deadline	PASS	15 slots x 30 min = 7.5 h <= 8.0 h

All 36 mandatory checks pass. Final status: ARMOR RESTORED - OPERATIONAL AND SAFE.

Estimated post-recovery subsystem health (model estimate h' = h + (100 − h)(1 − e^(−x/s)) using the category serving each subsystem): FLC 62% → 86.6%, STB 55% → 84.1%, SEN 71% → 91.7%, LSC 88% → 94.7%, STR 47% → 77.8%.

Figure 9. JARVIS recovery dashboard - Mark 85

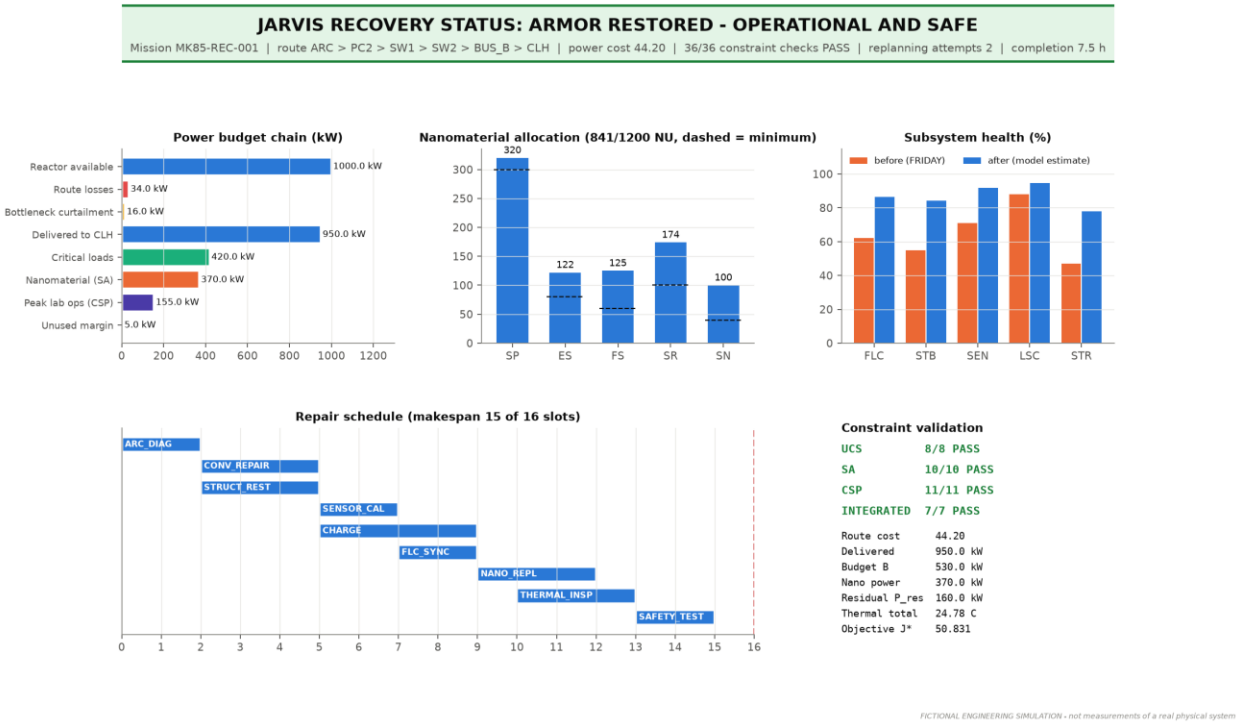


Figure 9. JARVIS recovery dashboard

14. Performance Analysis

14.1 Simulated Annealing versus hill climbing

Table 19. SA vs hill climbing on the final route and power fraction (seeds 0–29)

Method	Runs	Mean J	Std	Best	Worst
Simulated Annealing	30	50.772	0.287	51.199	50.128
Hill climbing (T = 0)	30	47.482	0.194	48.525	47.446

SA was better on 30 of 30 seeds. The two methods use the same neighbourhood and the same feasibility rules and differ only in the acceptance rule. Hill climbing quickly reaches a packet-aligned allocation from which every small move is worse, and stays there. SA accepts those temporary losses at high temperature and continues to better regions. Its run-to-run spread (σ = 0.287) shows the stochastic nature of SA. In return, a run takes about 2747 iterations.

14.2 CSP heuristics

Table 20. CSP search effort on the same instance

Variant	Solved	Makespan	Nodes	Backtracks	Checks	Wipeouts	Time ms
MRV + degree + FC	True	15	10	0	424	1	1.01
MRV + FC	True	15	10	0	370	1	0.78

FC only (static order)	True	15	38	4	1636	25	1.6
MRV + degree, no FC	True	15	341	33	1149	0	2.13
Plain backtracking	True	15	3828	1097	19697	0	19.48

Forward checking is the main source of the improvement because it catches dead ends one level early. MRV then steers the search to the tightly bounded end of the schedule (safety test, thermal inspection, charging) first, so that a valid assignment is found almost without backtracking. On this instance the degree heuristic only breaks ties, so it does not change the node count.

14.3 Route strategies

Table 21. Routes chosen by different strategies

Strategy	Route	Cost	Thermal °C	Satisfies all constraints?
UCS (all constraints)	ARC → PC2 → SW1 → SW2 → BUS_B → CLH	44.2	14.0	yes
UCS thermal-blind	ARC → PC2 → SW2 → BUS_B → CLH	37.25	14.5	NO
UCS thermal- & capacity-blind	ARC → PC3 → SW2 → BUS_B → CLH	28.35	11.5	NO
BFS (fewest hops)	ARC → PC2 → SW1 → BUS_A → CLH	46.4	12.0	yes

Ignoring the thermal constraint yields a cheaper but unsafe route (14.5 °C). Ignoring capacity as well selects PC3, which cannot carry the required 600 kW. Breadth-first search finds a safe route but not the cheapest. Only constrained UCS returns a route that is both safe and cost-optimal; validation also confirms this against the exhaustive enumeration of all simple routes.

Figure 10. Performance analysis of the three stages

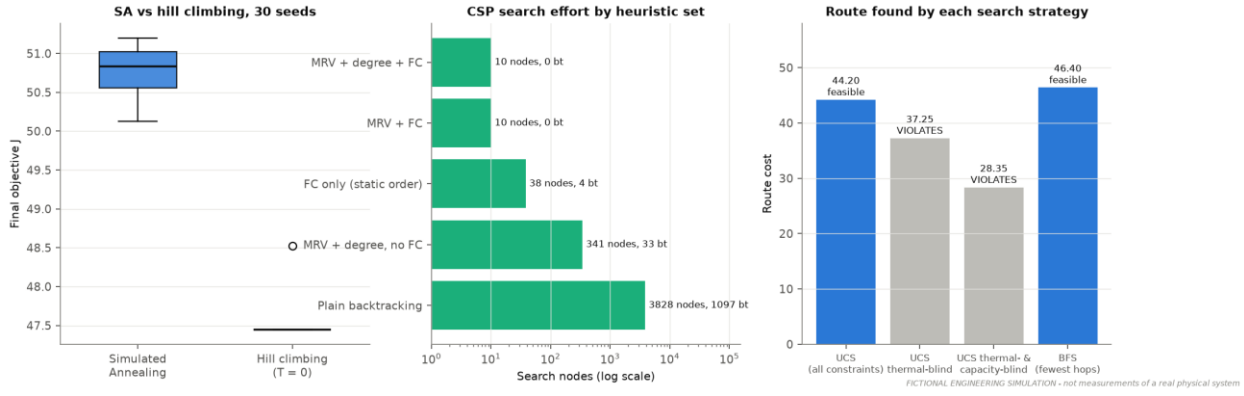


Figure 10. Performance analysis

14.4 Complexity

Table 22. Complexity of the three stages

Stage	Time complexity	Space	Guarantee

UCS (labels)	O(Λ log Λ), Λ = labels generated (classic UCS: O((|V|+|E|) log |V|))	O(Λ)	complete and optimal (non-negative costs, dominance pruning)
SA	O(K · n) objective evaluations, K iterations, n = 5 categories	O(n)	none (probabilistic); asymptotic convergence only for logarithmic cooling
CSP backtracking	O(d^k) worst case, k = 9 variables, d = max domain size	O(k·d)	complete: finds a schedule or proves none exists

15. Algorithm Comparison

Table 23. Comparison of the three algorithms

Aspect	UCS	Simulated Annealing	CSP backtracking
Problem class	Shortest path	Combinatorial optimisation	Constraint satisfaction
Role in JARVIS	Safe energy route, power budget	Nanomaterial allocation	Laboratory schedule
Search type	Systematic, uninformed, best-first on g	Stochastic local search	Systematic depth-first
Optimal?	Yes	No guarantee (near-optimal)	Feasibility (makespan via tightening)
Complete?	Yes	No	Yes
Handles hard constraints	Pruning during expansion	Feasible-set restriction	Natively
Uses randomness	No	Yes (seeded)	No
Key data structure	Binary heap	Current/best state, temperature	Domains, trail of pruned values
Output to next stage	B, θ_route, η	x*, P_res, M, θ_total	Schedule → validation

16. Limitations

- All data are fictional and the physics is simplified: losses are additive constants, heat is a linear per-unit model, and power is treated as a steady-state quantity.
- The staged decomposition is not globally optimal. A cheaper route with less capacity, or a smaller allocation, might give a better overall plan. JARVIS explores this only through its feedback rules.
- UCS minimises a weighted cost. It does not maximise the delivered power, so the weights encode an engineering trade-off that would need calibration in practice.
- SA is stochastic. Different seeds give slightly different allocations (Section 14.1), and the cooling parameters were tuned by hand.
- The CSP uses discrete 30-minute slots and fixed durations. It does not model uncertainty, task pre-emption or stochastic failures during the recovery.
- The replanning policy is rule-based. A learning or negotiation-based coordinator could explore the joint space more systematically.
- Post-recovery health is a simple model estimate shown for presentation, not a validated prediction.

17. Real-World Engineering Relevance

The same structure appears in real systems, with real data and certification replacing the fiction:

- **Power systems and micro-grids:** service restoration after a fault is a constrained path and reconfiguration problem; feeder capacity and thermal limits are hard constraints.
- **Spacecraft and aircraft health management:** re-routing power around failed converters and re-budgeting energy for heaters, payloads and repairs.
- **Electric-vehicle and battery systems:** thermal-aware energy allocation among subsystems.
- **Maintenance, repair and overhaul (MRO) scheduling:** hangars, robotic cells, technicians and test rigs are classical CSP / scheduling resources.
- **Robotics and additive manufacturing:** allocating limited material to competing structural goals.

The integration principle carries over directly: solve each sub-problem with the method that matches its structure, pass explicit quantities between stages, validate independently, and replan when a later stage proves an earlier decision unexecutable.

18. Conclusion

This case study solved one engineering problem, the recovery of a damaged Mark 85 armor, with one dataset and three cooperating AI algorithms. UCS produced a provably optimal safe power route (ARC → PC2 → SW1 → SW2 → BUS_B → CLH, cost 44.20) under capacity, failure and cumulative thermal constraints, and it fixed a 530 kW repair budget. Simulated Annealing used that budget to find a strong nanomaterial allocation (J* = 50.831) on a rugged landscape where hill climbing stagnates. The CSP turned the route and allocation into a conflict-free, makespan-optimal 15-slot laboratory schedule with very little search, thanks to MRV and forward checking.

The most instructive event was the replanning in attempt 1. Each stage was locally sound, yet the combination could not be executed, because the optimal allocation consumed the power needed for charging. JARVIS detected this with a cheap infeasibility certificate and resolved it by trading a little readiness for schedulability. This is why the three algorithms must be integrated rather than run in isolation. The final plan passes all 36 independent checks, and the armor is reported as armor restored - operational and safe, within the bounds of this fictional simulation.

References

[1] Russell, S. and Norvig, P. (2020). Artificial Intelligence: A Modern Approach, 4th ed. Pearson. (Ch. 3 uninformed search/UCS; Ch. 4 local search and simulated annealing; Ch. 6 constraint satisfaction.)

[2] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1, 269–271.

[3] Felner, A. (2011). Position paper: Dijkstra's algorithm versus Uniform Cost Search or a case against Dijkstra's algorithm. Proc. 4th Annual Symposium on Combinatorial Search (SoCS).

[4] Irnich, S. and Desaulniers, G. (2005). Shortest path problems with resource constraints. In Column Generation, Springer, 33–65.

[5] Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H. and Teller, E. (1953). Equation of state calculations by fast computing machines. Journal of Chemical Physics, 21(6), 1087–1092.

[6] Kirkpatrick, S., Gelatt, C. D. and Vecchi, M. P. (1983). Optimization by simulated annealing. Science, 220(4598), 671–680.

[7] Geman, S. and Geman, D. (1984). Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images. IEEE Trans. PAMI, 6(6), 721–741.

[8] Haralick, R. M. and Elliott, G. L. (1980). Increasing tree search efficiency for constraint satisfaction problems. Artificial Intelligence, 14(3), 263–313.

[9] Brélaz, D. (1979). New methods to color the vertices of a graph. Communications of the ACM, 22(4), 251–256.

[10] Mackworth, A. K. (1977). Consistency in networks of relations. Artificial Intelligence, 8(1), 99–118.

[11] Dechter, R. (2003). Constraint Processing. Morgan Kaufmann.

[12] Hagberg, A., Schult, D. and Swart, P. (2008). Exploring network structure, dynamics, and function using NetworkX. Proc. 7th Python in Science Conference (SciPy 2008), 11–15.

[13] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95.

[14] Marvel Studios (2008–2019). Iron Man film series and Avengers: Endgame. Fictional source of the Iron Man setting; no technical data were taken from it.

Viva Questions and Answers

Q1. Why is UCS optimal here, and why did you not use plain Dijkstra with one label per node?

Answer: All edge costs are non-negative and labels are popped in non-decreasing g, so the first goal label popped is optimal. One label per node is insufficient because the cumulative thermal limit is a path constraint. In our run the cheapest label at BUS_B (θ = 10.5 °C) cannot reach CLH, while a costlier and cooler label can. Keeping non-dominated (g, θ) labels preserves optimality.

Q2. What is the difference between UCS and BFS in this problem?

Answer: BFS minimises the number of hops. It returned ARC→PC2→SW1→BUS_A→CLH with cost 46.40, which is safe but more expensive than the UCS route (44.20). UCS minimises the sum of the weighted edge costs.

Q3. How does Stage 1 influence Stage 2 numerically?

Answer: B = min(P_avail − losses, bottleneck) − P_crit = 530 kW becomes the power cap ρB of the SA feasible set, and θ_route = 14.0 °C consumes part of the armor thermal limit.

Q4. Write the SA acceptance probability and explain the role of temperature.

Answer: $P = \exp(-\Delta E/T)$ for $\Delta E > 0$, and 1 otherwise. At high T, worse moves are accepted often, so the search explores. As T decreases geometrically ($T \leftarrow 0.92\ T$), the search behaves more and more like hill climbing and exploits the best region found.

Q5. Why does hill climbing fail on the allocation problem?

Answer: The packet-waste term makes small moves away from packet-aligned allocations worse, which creates many local optima. Hill climbing never accepts a worse move and gets stuck. SA accepts them temporarily and was better on 30/30 seeds.

Q6. How are constraints handled in your SA? Why not penalties?

Answer: Infeasible neighbours are rejected and resampled, so every visited state is a safe allocation. Penalties could leave the final answer infeasible, which is unacceptable for mandatory safety limits.

Q7. Is SA guaranteed to find the global optimum?

Answer: Only asymptotically, with a logarithmic cooling schedule and infinite time. With geometric cooling it is a heuristic. That is why we report its spread over 30 seeds.

Q8. Define the CSP variables, domains and constraints.

Answer: Variables: the 9 tasks. Value: (start slot, concrete resource units). Domains: starts within the deadline combined with compatible equipment. Constraints: precedence, cool-down separation, safety no-overlap, resource exclusivity, cumulative power ≤ P_res, technicians ≤ 2, deadlines.

Q9. What do MRV, the degree heuristic and forward checking do?

Answer: MRV picks the variable with the fewest remaining values (fail-first). The degree heuristic breaks ties by the number of constraints on unassigned variables. Forward checking removes neighbour values that are inconsistent with a new assignment and backtracks as soon as a domain empties.

Q10. Show a backtracking event from your trace.

Answer: CONV_REPAIR was first assigned ARM-1. Forward checking emptied the domain of STRUCT_REST, which also needs ARM-1 in that window, so the assignment was undone and ARM-2 was tried next.

Q11. How do SA outputs become CSP inputs?

Answer: Durations are computed from the allocation: d_STRUCT = ⌈x_SR/80⌉, d_REPL = ⌈M/400⌉, d_SENSOR = 1 + ⌈x_SN/100⌉, d_FLC = 1 + ⌈x_FS/150⌉ and d_THERM = 1 + ⌈θ_total/15⌉. The power cap is P_res = B − P_nano.

Q12. What happened in planning attempt 1?

Answer: P_res was 111.9 kW while charging needs 120 kW. The CSP precheck proved infeasibility without any search, and JARVIS lowered ρ from 0.80 to 0.70 and re-optimised.

Q13. How do you know the final schedule is makespan-optimal?

Answer: JARVIS re-solved with H = 14. The critical path DIAG(2) + CONV(3) + CHARGE(4) + gap(1) + THERM(3) + SAFETY(2) = 15 slots already exceeds 14, so 15 is a lower bound and it is achieved.

Q14. What is the integrated feasibility condition?

Answer: F = g_UCS ∧ g_SA ∧ g_CSP ∧ g_INT. It includes the power balance P_crit + P_nano + peak P_lab ≤ P_del, the thermal balance, replenishment of the consumed material, recharge to the target state of charge and the deadline.

Q15. Why not solve everything with one CSP or one SA?

Answer: A CSP has no native notion of cost or readiness, and SA cannot guarantee hard feasibility or prove infeasibility. A single joint search space would be intractable. Each algorithm is used where its guarantees matter.

Q16. How did you make results reproducible and avoid fabricated outputs?

Answer: A seeded random.Random(85) is used, UCS and the CSP are deterministic, and every table and figure is generated from outputs/results.json, which main.py writes. The validation module recomputes everything, and 10 unit tests check optimality, feasibility and replanning.

Q17. What happens if PC2 also fails?

Answer: UCS falls back to the backup-storage route, which gives B = 280 kW. The mandatory minimum allocation needs 239 kW, more than the 224 kW cap, so JARVIS excludes the route. No admissible route remains, and it aborts safely with the armor held in safe mode.

Q18. What is the time complexity of each stage?

Answer: Label-setting UCS is O(Λ log Λ) in the number of labels. SA is O(K·n) for K iterations. CSP backtracking is O(d^k) in the worst case, but heuristics cut it to 10 nodes here, against 3828 for plain backtracking.

Q19. Which parts of the study are real and which are fictional?

Answer: The armor, the reactor and all numerical data are fictional. The algorithms, equations, code, execution traces and measured performance are real outputs of the program.

Appendix A. Complete Source Code

A.1 main.py

```
from __future__ import annotations

import argparse
import io
import sys
from contextlib import redirect_stdout
from dataclasses import replace
from pathlib import Path

from jarvis import reporting, visualization as viz
from jarvis.data_models import load_dataset
from jarvis.experiments import run_experiments
from jarvis.pipeline import run_mission

ROOT = Path(__file__).resolve().parent
def _rel(p) -> str:
    p = Path(p).resolve()
    try:
        return p.relative_to(ROOT).as_posix()
    except ValueError:
        return p.name
class Tee(io.TextIOBase):
    def __init__(self, *streams):
        self.streams = streams

    def write(self, s):
        for st in self.streams:
            st.write(s)
        return len(s)

    def flush(self):
        for st in self.streams:
            st.flush()
def main(argv=None) -> int:
    ap = argparse.ArgumentParser(description="JARVIS Mark 85 recovery mission")
    ap.add_argument("--dataset", default=str(ROOT / "data" / "mission_dataset.json"))
    ap.add_argument("--out", default=str(ROOT / "outputs"))
    ap.add_argument("--seed", type=int, default=None, help="override the dataset random seed")
    ap.add_argument("--no-plots", action="store_true")
    ap.add_argument("--quick", action="store_true", help="skip the multi-seed performance study")
    ap.add_argument("--scenario", choices=["nominal", "pc2-failure"], default="nominal",
                    help="pc2-failure: stress test in which the secondary converter also fails")
    args = ap.parse_args(argv)

    outdir = Path(args.out)
    figdir = outdir / "figures"
    outdir.mkdir(parents=True, exist_ok=True)
    log_buffer = io.StringIO()
    tee = Tee(sys.stdout, log_buffer)

    with redirect_stdout(tee):
        ds = load_dataset(args.dataset)
        if args.scenario == "pc2-failure":
            ds.network.nodes["PC2"] = replace(ds.network.nodes["PC2"], status="FAILED")
            print("STRESS SCENARIO: FRIDAY reports secondary converter PC2 FAILED as well")
        print(reporting.mission_header(ds))
        out = run_mission(ds, seed=args.seed)
        if out.ucs is None or out.csp is None or not out.csp.solved:
            print(reporting.attempts_report(out))
            print(f"\nMISSION FAILED: {out.status}")
            (outdir / "console_log.txt").write_text(log_buffer.getvalue(), encoding="utf-8")
            return 2
        print(reporting.ucs_report(ds, out.ucs))
        print(reporting.sa_report(ds, out.sa))
        print(reporting.csp_report(out))
        print(reporting.attempts_report(out))
        print(reporting.validation_report(out))

        perf = {}
        if not args.quick:
            print(reporting.rule("PERFORMANCE STUDY"))
            perf = run_experiments(ds, out)
            sv = perf["sa_vs_hc"]
            print(f"SA  over {len(sv['sa'])} seeds: mean J={sv['sa_mean']:.3f} sd={sv['sa_std']:.3f} "
                  f"best={sv['sa_best']:.3f} worst={sv['sa_worst']:.3f}")
            print(f"HC  over {len(sv['hc'])} seeds: mean J={sv['hc_mean']:.3f} sd={sv['hc_std']:.3f} "
                  f"best={sv['hc_best']:.3f} worst={sv['hc_worst']:.3f}   SA better in {sv['sa_wins']} seeds")
            for r in perf["csp_variants"]:
                print(f"CSP {r['variant']:<24} solved={r['solved']} nodes={r['nodes']:>6} "
                      f"backtracks={r['backtracks']:>6} checks={r['checks']:>7} time={r['time_ms']} ms")
            for r in perf["route_methods"]:
                print(f"ROUTE {r['method'].replace(chr(10), ' '):<30} {' > '.join(r['path'])} cost={r['cost']} "
                      f"thermal={r['thermal']} feasible={r['feasible']}")
            print(f"timing (ms): {perf['timing_ms']}")

        reporting.export(out, ds, perf, outdir)
        if not args.no_plots:
```

```
            figdir.mkdir(parents=True, exist_ok=True)
            figs = [viz.architecture(figdir / "fig01_architecture.png"),
                    viz.power_network(ds, out.ucs, figdir / "fig02_power_network.png"),
                    viz.ucs_expansion(out.ucs, ds.network, figdir / "fig03_ucs_expansion.png"),
                    viz.nano_allocation(ds, out.attempts, out.sa, figdir / "fig04_nano_allocation.png"),
                    viz.sa_convergence(out.sa, figdir / "fig05_sa_convergence.png"),
                    viz.csp_constraint_graph(out.csp_model, figdir / "fig06_csp_constraint_graph.png"),
                    viz.gantt(out.csp, out.req, ds, figdir / "fig07_gantt.png"),
                    viz.workflow(out, figdir / "fig08_workflow.png"),
                    viz.dashboard(ds, out, figdir / "fig09_dashboard.png")]
            if perf:
                figs.append(viz.performance(perf, figdir / "fig10_performance.png"))
            print(reporting.rule("FIGURES"))
            for f in figs:
                print("  " + _rel(f))
        print(f"\nExports written to {_rel(outdir)}")
    (outdir / "console_log.txt").write_text(log_buffer.getvalue(), encoding="utf-8")
    return 0 if out.success else 1
if __name__ == "__main__":
    sys.exit(main())
```

A.2 jarvis/__init__.py

```
__version__ = "1.0.0"
```

A.3 jarvis/data_models.py

```
from __future__ import annotations

import json
from dataclasses import dataclass, field, asdict
from pathlib import Path
from typing import Dict, List, Optional, Tuple
@dataclass(frozen=True)
class Node:
    id: str
    label: str
    type: str
    status: str

    @property
    def failed(self) -> bool:
        return self.status == "FAILED"
@dataclass(frozen=True)
class Edge:
    u: str
    v: str
    loss_kW: float
    thermal_C: float
    switch_ms: float
    capacity_kW: float

    def cost(self, w: Dict[str, float]) -> float:
        return (w["loss"] * self.loss_kW
                + w["thermal"] * self.thermal_C
                + w["switching"] * self.switch_ms)
@dataclass
class PowerNetwork:
    source: str
    goal: str
    required_throughput_kW: float
    thermal_limit_C: float
    cost_weights: Dict[str, float]
    nodes: Dict[str, Node]
    edges: List[Edge]
    load_edges: List[Tuple[str, str]]

    def out_edges(self, u: str) -> List[Edge]:
        return [e for e in self.edges if e.u == u]

    def edge(self, u: str, v: str) -> Edge:
        for e in self.edges:
            if e.u == u and e.v == v:
                return e
        raise KeyError(f"No edge {u}->{v}")
@dataclass(frozen=True)
class NanoCategory:
    id: str
    name: str
    min_NU: int
    max_NU: int
    power_kW_per_NU: float
    heat_C_per_NU: float
    benefit_weight: float
    saturation_NU: float
    packet_NU: int
@dataclass
class NanomaterialSpec:
    reserve_NU: int
    armor_thermal_limit_C: float
    initial_power_fraction: float
```

```
    objective_weights: Dict[str, float]
    categories: List[NanoCategory]
@dataclass
class AnnealingParams:
    initial_temperature: float
    cooling_rate: float
    min_temperature: float
    iterations_per_temperature: int
    max_iterations: int
    stall_epochs: int
    step_min_NU: int
    step_max_NU: int
@dataclass(frozen=True)
class ResourceUnit:
    id: str
    rtype: str
    capabilities: Tuple[str, ...]
@dataclass
class TaskSpec:
    id: str
    name: str
    duration_rule: str
    power_kW: float
    technicians: int
    needs: Dict[str, str]
    after: List[str]
    deadline: Optional[int]
@dataclass
class LabSpec:
    resources: Dict[str, List[ResourceUnit]]
    technicians: int
    tasks: List[TaskSpec]
    separations: List[dict]
    no_overlap: List[dict]
@dataclass
class MissionDataset:
    mission: dict
    reactor: dict
    subsystems: List[dict]
    network: PowerNetwork
    nano: NanomaterialSpec
    annealing: AnnealingParams
    rules: dict
    lab: LabSpec
    source_path: str = ""

    @property
    def critical_load_kW(self) -> float:
        return float(sum(s["critical_load_kW"] for s in self.subsystems))
def load_dataset(path: str | Path) -> MissionDataset:
    path = Path(path)
    raw = json.loads(path.read_text(encoding="utf-8"))
    pn = raw["power_network"]
    network = PowerNetwork(
        source=pn["source"], goal=pn["goal"],
        required_throughput_kW=float(pn["required_throughput_kW"]),
        thermal_limit_C=float(pn["thermal_limit_C"]),
        cost_weights=pn["cost_weights"],
        nodes={n["id"]: Node(**n) for n in pn["nodes"]},
        edges=[Edge(**e) for e in pn["edges"]],
        load_edges=[(e["u"], e["v"]) for e in pn["load_edges"]],
    )
    nm = raw["nanomaterial"]
    nano = NanomaterialSpec(
        reserve_NU=nm["reserve_NU"],
        armor_thermal_limit_C=float(nm["armor_thermal_limit_C"]),
        initial_power_fraction=float(nm["initial_power_fraction"]),
        objective_weights=nm["objective_weights"],
        categories=[NanoCategory(**c) for c in nm["categories"]],
    )
    lab_raw = raw["laboratory"]
    resources = {
        rtype: [ResourceUnit(u["id"], rtype, tuple(u["capabilities"])) for u in units]
        for rtype, units in lab_raw["resources"].items()
    }
    lab = LabSpec(
        resources=resources,
        technicians=lab_raw["technicians"],
        tasks=[TaskSpec(**t) for t in lab_raw["tasks"]],
        separations=lab_raw["separations"],
        no_overlap=lab_raw["no_overlap"],
    )
    return MissionDataset(
        mission=raw["mission"], reactor=raw["reactor"], subsystems=raw["subsystems"],
        network=network, nano=nano, annealing=AnnealingParams(**raw["annealing"]),
        rules=raw["requirement_rules"], lab=lab, source_path=str(path),
    )
@dataclass
class UCSResult:
    found: bool
    path: List[str]
    path_cost: float
    total_loss_kW: float
```

```
    total_thermal_C: float
    total_switch_ms: float
    bottleneck_capacity_kW: float
    delivered_power_kW: float
    critical_load_kW: float
    power_budget_kW: float
    route_efficiency: float
    expansions: int
    generated: int
    trace: List[dict] = field(default_factory=list)
    pruned: List[dict] = field(default_factory=list)
    excluded_edges: List[Tuple[str, str]] = field(default_factory=list)
@dataclass
class SAResult:
    feasible: bool
    allocation: Dict[str, int]
    objective: float
    energy: float
    nano_power_kW: float
    nano_heat_C: float
    material_used_NU: int
    power_cap_kW: float
    power_fraction: float
    residual_power_kW: float
    total_thermal_C: float
    iterations: int
    accepted_worse: int
    rejected_infeasible: int
    improvements: int
    history: List[dict] = field(default_factory=list)
    initial_allocation: Dict[str, int] = field(default_factory=dict)
    initial_objective: float = 0.0
    termination: str = ""
@dataclass
class TaskRequirement:
    id: str
    name: str
    duration: int
    power_kW: float
    technicians: int
    needs: Dict[str, str]
    after: List[str]
    deadline: Optional[int]
    derivation: str
@dataclass
class Requirements:
    tasks: List[TaskRequirement]
    power_cap_kW: float
    horizon: int
    technicians: int
    notes: List[str]
@dataclass
class CSPResult:
    solved: bool
    schedule: Dict[str, dict]
    makespan: int
    nodes: int
    backtracks: int
    constraint_checks: int
    fc_prunes: int
    wipeouts: int
    trace: List[str] = field(default_factory=list)
    failure_reason: str = ""
    elapsed_s: float = 0.0
def to_jsonable(obj):
    if hasattr(obj, "__dataclass_fields__"):
        return {k: to_jsonable(v) for k, v in asdict(obj).items()}
    if isinstance(obj, dict):
        return {str(k): to_jsonable(v) for k, v in obj.items()}
    if isinstance(obj, (list, tuple)):
        return [to_jsonable(v) for v in obj]
    return obj
```

A.4 jarvis/ucs.py

```
from __future__ import annotations

import heapq
import itertools
from typing import Iterable, List, Optional, Set, Tuple

from .data_models import PowerNetwork, UCSResult
def _dominated(labels: List[Tuple[float, float]], g: float, theta: float) -> bool:
    return any(g2 <= g + 1e-9 and t2 <= theta + 1e-9 for g2, t2 in labels)
def uniform_cost_search(net: PowerNetwork,
                        available_kW: float,
                        critical_load_kW: float,
                        excluded_edges: Iterable[Tuple[str, str]] = (),
                        enforce_thermal: bool = True,
                        enforce_capacity: bool = True,
                        record_trace: bool = True) -> UCSResult:
    w = net.cost_weights
```

```
    excluded: Set[Tuple[str, str]] = set(excluded_edges)
    tie = itertools.count()
    frontier = [(0.0, next(tie), net.source, (net.source,), 0.0, 0.0, 0.0, float("inf"))]
    expanded = {}
    trace, pruned = [], []
    expansions = generated = 0

    while frontier:
        g, _, node, path, theta, loss, sw, bott = heapq.heappop(frontier)
        if _dominated(expanded.get(node, []), g, theta):
            if record_trace:
                pruned.append({"node": node, "g": round(g, 2), "theta": theta,
                               "reason": "dominated label (already expanded cheaper & cooler)"})
            continue
        expanded.setdefault(node, []).append((g, theta))
        expansions += 1

        step = {"step": expansions, "expand": node, "g": round(g, 2), "theta": round(theta, 2),
                "path": list(path), "pushed": [], "rejected": []}

        if node == net.goal:
            if record_trace:
                step["frontier_after"] = _snapshot(frontier)
                step["goal"] = True
                trace.append(step)
            delivered = min(available_kW - loss, bott)
            return UCSResult(
                found=True, path=list(path), path_cost=round(g, 4),
                total_loss_kW=loss, total_thermal_C=round(theta, 4), total_switch_ms=sw,
                bottleneck_capacity_kW=bott, delivered_power_kW=delivered,
                critical_load_kW=critical_load_kW,
                power_budget_kW=delivered - critical_load_kW,
                route_efficiency=delivered / available_kW,
                expansions=expansions, generated=generated,
                trace=trace, pruned=pruned, excluded_edges=sorted(excluded))

        for e in net.out_edges(node):
            reason = None
            if (e.u, e.v) in excluded:
                reason = "edge excluded by replanning"
            elif net.nodes[e.v].failed:
                reason = f"{e.v} FAILED (FRIDAY fault)"
            elif e.v in path:
                reason = "cycle"
            elif enforce_capacity and e.capacity_kW < net.required_throughput_kW:
                reason = f"capacity {e.capacity_kW:.0f} < required {net.required_throughput_kW:.0f} kW"
            new_theta = theta + e.thermal_C
            if reason is None and enforce_thermal and new_theta > net.thermal_limit_C + 1e-9:
                reason = f"thermal {new_theta:.1f} > limit {net.thermal_limit_C:.1f} C"
            new_g = g + e.cost(w)
            if reason is None and _dominated(expanded.get(e.v, []), new_g, new_theta):
                reason = "dominated by expanded label"
            if reason:
                step["rejected"].append({"to": e.v, "reason": reason})
                continue
            generated += 1
            heapq.heappush(frontier, (new_g, next(tie), e.v, path + (e.v,), new_theta,
                                      loss + e.loss_kW, sw + e.switch_ms,
                                      min(bott, e.capacity_kW)))
            step["pushed"].append({"to": e.v, "edge_cost": round(e.cost(w), 2),
                                   "g": round(new_g, 2), "theta": round(new_theta, 2)})
        if record_trace:
            step["frontier_after"] = _snapshot(frontier)
            trace.append(step)

    return UCSResult(found=False, path=[], path_cost=float("inf"), total_loss_kW=0,
                     total_thermal_C=0, total_switch_ms=0, bottleneck_capacity_kW=0,
                     delivered_power_kW=0, critical_load_kW=critical_load_kW,
                     power_budget_kW=-critical_load_kW, route_efficiency=0,
                     expansions=expansions, generated=generated, trace=trace,
                     pruned=pruned, excluded_edges=sorted(excluded))
def _snapshot(frontier) -> List[dict]:
    return [{"node": item[2], "g": round(item[0], 2), "theta": round(item[4], 2)}
            for item in sorted(frontier)]
def enumerate_simple_paths(net: PowerNetwork) -> List[dict]:
    w = net.cost_weights
    out = []

    def dfs(node, path, edges):
        if node == net.goal:
            out.append({
                "path": list(path),
                "cost": round(sum(e.cost(w) for e in edges), 4),
                "loss": sum(e.loss_kW for e in edges),
                "thermal": sum(e.thermal_C for e in edges),
                "bottleneck": min(e.capacity_kW for e in edges),
                "uses_failed": any(net.nodes[p].failed for p in path),
            })
            return
        for e in net.out_edges(node):
            if e.v not in path:
                dfs(e.v, path + [e.v], edges + [e])

    dfs(net.source, [net.source], [])
    for r in out:
```

```
        r["feasible"] = (not r["uses_failed"]
                         and r["bottleneck"] >= net.required_throughput_kW
                         and r["thermal"] <= net.thermal_limit_C + 1e-9)
    return sorted(out, key=lambda r: r["cost"])
```

A.5 jarvis/annealing.py

```
from __future__ import annotations

import math
import random
from typing import Dict, List, Optional, Tuple

from .data_models import AnnealingParams, NanomaterialSpec, SAResult
class NanoAllocationProblem:
    def __init__(self, spec: NanomaterialSpec, power_budget_kW: float,
                 power_fraction: float, route_thermal_C: float):
        self.spec = spec
        self.cats = spec.categories
        self.ids = [c.id for c in self.cats]
        self.B = power_budget_kW
        self.rho = power_fraction
        self.power_cap = power_fraction * power_budget_kW
        self.route_thermal = route_thermal_C
        w = spec.objective_weights
        self.lam_E, self.lam_M, self.lam_W = w["energy"], w["material"], w["packet_waste"]

    def power(self, x: Dict[str, int]) -> float:
        return sum(c.power_kW_per_NU * x[c.id] for c in self.cats)

    def heat(self, x: Dict[str, int]) -> float:
        return sum(c.heat_C_per_NU * x[c.id] for c in self.cats)

    def material(self, x: Dict[str, int]) -> int:
        return sum(x.values())

    def benefit_terms(self, x: Dict[str, int]) -> Dict[str, float]:
        return {c.id: 100.0 * c.benefit_weight * (1.0 - math.exp(-x[c.id] / c.saturation_NU))
                for c in self.cats}

    def objective(self, x: Dict[str, int]) -> float:
        benefit = sum(self.benefit_terms(x).values())
        energy_pen = self.lam_E * self.power(x) / self.B
        material_pen = self.lam_M * self.material(x) / self.spec.reserve_NU
        waste_pen = self.lam_W * sum((x[c.id] % c.packet_NU) / c.packet_NU for c in self.cats)
        return benefit - energy_pen - material_pen - waste_pen

    def violations(self, x: Dict[str, int]) -> List[str]:
        v = []
        for c in self.cats:
            if x[c.id] < c.min_NU:
                v.append(f"{c.id} below mandatory minimum ({x[c.id]} < {c.min_NU})")
            if x[c.id] > c.max_NU:
                v.append(f"{c.id} above maximum ({x[c.id]} > {c.max_NU})")
        if self.material(x) > self.spec.reserve_NU:
            v.append(f"material {self.material(x)} > reserve {self.spec.reserve_NU}")
        if self.power(x) > self.power_cap + 1e-9:
            v.append(f"power {self.power(x):.1f} > cap {self.power_cap:.1f} kW")
        total_heat = self.route_thermal + self.heat(x)
        if total_heat > self.spec.armor_thermal_limit_C + 1e-9:
            v.append(f"thermal {total_heat:.2f} > limit {self.spec.armor_thermal_limit_C} C")
        return v

    def feasible(self, x: Dict[str, int]) -> bool:
        return not self.violations(x)

    def initial_solution(self) -> Dict[str, int]:
        return {c.id: c.min_NU for c in self.cats}

    def neighbor(self, x: Dict[str, int], rng: random.Random,
                 step_min: int, step_max: int) -> Tuple[Dict[str, int], str]:
        y = dict(x)
        delta = rng.randint(step_min, step_max)
        kind = rng.choices(["transfer", "add", "remove"], weights=[0.6, 0.2, 0.2])[0]
        i = rng.choice(self.ids)
        if kind == "transfer":
            j = rng.choice([k for k in self.ids if k != i])
            y[i] -= delta
            y[j] += delta
            desc = f"move {delta} NU {i}->{j}"
        elif kind == "add":
            y[i] += delta
            desc = f"add {delta} NU to {i}"
        else:
            y[i] -= delta
            desc = f"remove {delta} NU from {i}"
        return y, desc
def simulated_annealing(problem: NanoAllocationProblem, params: AnnealingParams,
                        seed: int, greedy: bool = False,
                        max_neighbor_tries: int = 100) -> SAResult:
    rng = random.Random(seed)
    x0 = problem.initial_solution()
    if not problem.feasible(x0):
```

```
        return SAResult(feasible=False, allocation=x0, objective=float("-inf"),
                        energy=float("inf"), nano_power_kW=problem.power(x0),
                        nano_heat_C=problem.heat(x0), material_used_NU=problem.material(x0),
                        power_cap_kW=problem.power_cap, power_fraction=problem.rho,
                        residual_power_kW=problem.B - problem.power(x0),
                        total_thermal_C=problem.route_thermal + problem.heat(x0),
                        iterations=0, accepted_worse=0, rejected_infeasible=0, improvements=0,
                        initial_allocation=x0, initial_objective=problem.objective(x0),
                        termination="INFEASIBLE: mandatory minimum allocation violates "
                                    + "; ".join(problem.violations(x0)))

    cur, cur_E = dict(x0), -problem.objective(x0)
    best, best_E = dict(cur), cur_E
    T = params.initial_temperature
    history = [{"iter": 0, "T": T, "move": "initial (mandatory minimums)", "cand_J": -cur_E,
                "cur_J": -cur_E, "best_J": -best_E, "dE": 0.0, "p_accept": 1.0,
                "decision": "START", "alloc": dict(cur)}]
    it = accepted_worse = rejected = improvements = 0
    epochs_since_best = 0
    termination = ""

    while True:
        if it >= params.max_iterations:
            termination = f"max iterations ({params.max_iterations}) reached"
            break
        if not greedy and T < params.min_temperature:
            termination = f"temperature {T:.4f} < T_min {params.min_temperature}"
            break
        if epochs_since_best >= params.stall_epochs:
            termination = f"no best-improvement for {params.stall_epochs} temperature epochs"
            break

        improved_this_epoch = False
        for _ in range(params.iterations_per_temperature):
            it += 1
            cand = None
            for _try in range(max_neighbor_tries):
                y, desc = problem.neighbor(cur, rng, params.step_min_NU, params.step_max_NU)
                if problem.feasible(y):
                    cand = y
                    break
                rejected += 1
            if cand is None:
                history.append({"iter": it, "T": T, "move": "no feasible neighbour",
                                "cand_J": None, "cur_J": -cur_E, "best_J": -best_E,
                                "dE": None, "p_accept": 0.0, "decision": "STAY", "alloc": dict(cur)})
                continue
            cand_E = -problem.objective(cand)
            dE = cand_E - cur_E
            if dE <= 0:
                p, accept = 1.0, True
            elif greedy:
                p, accept = 0.0, False
            else:
                p = math.exp(-dE / T)
                accept = rng.random() < p
            if accept:
                decision = "ACCEPT-BETTER" if dE <= 0 else "ACCEPT-WORSE"
                if dE > 0:
                    accepted_worse += 1
                cur, cur_E = cand, cand_E
                if cur_E < best_E - 1e-12:
                    best, best_E = dict(cur), cur_E
                    improvements += 1
                    improved_this_epoch = True
                    decision += " (new best)"
            else:
                decision = "REJECT"
            history.append({"iter": it, "T": T, "move": desc, "cand_J": -cand_E,
                            "cur_J": -cur_E, "best_J": -best_E, "dE": dE,
                            "p_accept": p, "decision": decision, "alloc": dict(cur)})
            if it >= params.max_iterations:
                break
        epochs_since_best = 0 if improved_this_epoch else epochs_since_best + 1
        if not greedy:
            T *= params.cooling_rate

    return SAResult(
        feasible=True, allocation=best, objective=-best_E, energy=best_E,
        nano_power_kW=problem.power(best), nano_heat_C=problem.heat(best),
        material_used_NU=problem.material(best), power_cap_kW=problem.power_cap,
        power_fraction=problem.rho, residual_power_kW=problem.B - problem.power(best),
        total_thermal_C=problem.route_thermal + problem.heat(best),
        iterations=it, accepted_worse=accepted_worse, rejected_infeasible=rejected,
        improvements=improvements, history=history, initial_allocation=x0,
        initial_objective=problem.objective(x0), termination=termination)
```

A.6 jarvis/requirements.py

```
from __future__ import annotations

import math

from .data_models import MissionDataset, Requirements, SAResult, TaskRequirement, UCSResult
```

```
def derive_requirements(ds: MissionDataset, ucs: UCSResult, sa: SAResult) -> Requirements:
    r = ds.rules
    x = sa.allocation
    failed_units = sorted(n.id for n in ds.network.nodes.values() if n.failed)
    battery = r["armor_battery_capacity_kWh"]
    deficit_kWh = battery * (r["target_state_of_charge"] - r["armor_state_of_charge"])
    slot_h = ds.mission["slot_minutes"] / 60.0
    charge_per_slot = r["charger_rating_kW"] * ucs.route_efficiency * slot_h

    rules = {
        "struct_restoration": (
            math.ceil(x["SR"] / r["struct_restoration_NU_per_slot"]),
            f"ceil(x_SR={x['SR']} / {r['struct_restoration_NU_per_slot']} NU per slot)"),
        "replenishment": (
            math.ceil(sa.material_used_NU / r["replenishment_NU_per_slot"]),
            f"ceil(M={sa.material_used_NU} NU consumed / {r['replenishment_NU_per_slot']} NU per slot)"),
        "sensor_calibration": (
            1 + math.ceil(x["SN"] / r["sensor_calibration_NU_per_slot"]),
            f"1 + ceil(x_SN={x['SN']} / {r['sensor_calibration_NU_per_slot']})"),
        "flight_sync": (
            1 + math.ceil(x["FS"] / r["flight_sync_NU_per_slot"]),
            f"1 + ceil(x_FS={x['FS']} / {r['flight_sync_NU_per_slot']})"),
        "thermal_inspection": (
            1 + math.ceil(sa.total_thermal_C / r["thermal_inspection_C_per_slot"]),
            f"1 + ceil(theta_total={sa.total_thermal_C:.2f} C / {r['thermal_inspection_C_per_slot']})"),
        "charging": (
            math.ceil(deficit_kWh / charge_per_slot),
            f"ceil(deficit {deficit_kWh:.0f} kWh / ({r['charger_rating_kW']} kW x eta={ucs.route_efficiency:.3f} x {slot_h} h))"),
        "converter_repair": (
            r["converter_repair_slots_per_failed_unit"] * len(failed_units),
            f"{r['converter_repair_slots_per_failed_unit']} slots x {len(failed_units)} failed unit(s) {failed_units}"),
    }

    tasks = []
    for t in ds.lab.tasks:
        if t.duration_rule.startswith("fixed:"):
            d, why = int(t.duration_rule.split(":")[1]), "fixed procedure length"
        else:
            d, why = rules[t.duration_rule]
        tasks.append(TaskRequirement(id=t.id, name=t.name, duration=max(1, int(d)),
                                     power_kW=float(t.power_kW), technicians=t.technicians,
                                     needs=dict(t.needs), after=list(t.after),
                                     deadline=t.deadline, derivation=why))
    notes = [
        f"Power cap for concurrent lab operations P_res = B - P_nano = "
        f"{ucs.power_budget_kW:.1f} - {sa.nano_power_kW:.1f} = {sa.residual_power_kW:.1f} kW",
        f"Route efficiency eta = {ucs.route_efficiency:.3f} scales the armor charging rate",
        f"Combined thermal load {sa.total_thermal_C:.2f} C sets the thermal-inspection length",
    ]
    return Requirements(tasks=tasks, power_cap_kW=sa.residual_power_kW,
                        horizon=ds.mission["horizon_slots"], technicians=ds.lab.technicians,
                        notes=notes)
```

A.7 jarvis/csp.py

```
from __future__ import annotations

import itertools
import time
from typing import Dict, List, Optional, Tuple

from .data_models import CSPResult, LabSpec, Requirements, TaskRequirement

Value = Tuple[int, Tuple[str, ...]]
class RecoveryCSP:
    def __init__(self, req: Requirements, lab: LabSpec, horizon: Optional[int] = None):
        self.req = req
        self.lab = lab
        self.H = horizon if horizon is not None else req.horizon
        self.tasks: Dict[str, TaskRequirement] = {t.id: t for t in req.tasks}
        self.order = [t.id for t in req.tasks]
        self.cap = req.power_cap_kW
        self.ntech = req.technicians
        self.prec: Dict[Tuple[str, str], int] = {}
        for t in req.tasks:
            for p in t.after:
                self.prec[(p, t.id)] = 0
        for s in lab.separations:
            self.prec[(s["first"], s["second"])] = max(self.prec.get((s["first"], s["second"]), 0),
                                                       s["min_gap_slots"])
        self.no_overlap = {frozenset((n["a"], n["b"])) for n in lab.no_overlap}
        self.domains: Dict[str, List[Value]] = {k: self._unary_domain(self.tasks[k]) for k in self.order}
        self.neighbors = self._build_neighbors()

    def compatible_units(self, task: TaskRequirement) -> Dict[str, List[str]]:
        out = {}
        for rtype, cap in task.needs.items():
            out[rtype] = [u.id for u in self.lab.resources[rtype] if cap in u.capabilities]
        return out

    def _unary_domain(self, t: TaskRequirement) -> List[Value]:
        if t.power_kW > self.cap or t.technicians > self.ntech:
            return []
        last_end = self.H if t.deadline is None else min(self.H, t.deadline)
```

```
        units = self.compatible_units(t)
        combos = list(itertools.product(*[units[r] for r in sorted(units)])) if units else [()]
        return [(s, tuple(c)) for s in range(0, last_end - t.duration + 1) for c in combos]

    def _build_neighbors(self) -> Dict[str, set]:
        nb = {k: set() for k in self.order}
        for (a, b) in self.prec:
            nb[a].add(b); nb[b].add(a)
        for pair in self.no_overlap:
            a, b = tuple(pair)
            nb[a].add(b); nb[b].add(a)
        units = {k: set(u for v in self.domains[k] for u in v[1]) for k in self.order}
        for a, b in itertools.combinations(self.order, 2):
            if units[a] & units[b]:
                nb[a].add(b); nb[b].add(a)
        return nb

    def constraint_edges(self) -> List[Tuple[str, str, str]]:
        edges = []
        for (a, b), gap in self.prec.items():
            edges.append((a, b, "separation" if gap > 0 else "precedence"))
        for pair in self.no_overlap:
            a, b = sorted(pair)
            edges.append((a, b, "no-overlap"))
        units = {k: set(u for v in self.domains[k] for u in v[1]) for k in self.order}
        for a, b in itertools.combinations(self.order, 2):
            shared = units[a] & units[b]
            if shared:
                edges.append((a, b, "resource:" + ",".join(sorted(shared))))
        return edges

    def propagate_bounds(self) -> List[str]:
        log = []
        changed = True
        while changed:
            changed = False
            for (a, b), gap in self.prec.items():
                da, db = self.domains[a], self.domains[b]
                if not da or not db:
                    continue
                est_b = min(v[0] for v in da) + self.tasks[a].duration + gap
                lst_a = max(v[0] for v in db) - gap - self.tasks[a].duration
                nb_ = [v for v in db if v[0] >= est_b]
                na_ = [v for v in da if v[0] <= lst_a]
                if len(nb_) != len(db):
                    log.append(f"bound: {b} start >= {est_b} (after {a}) |D| {len(db)}->{len(nb_)}")
                    self.domains[b] = nb_; changed = True
                if len(na_) != len(da):
                    log.append(f"bound: {a} start <= {lst_a} (before {b}) |D| {len(da)}->{len(na_)}")
                    self.domains[a] = na_; changed = True
        return log

    def prechecks(self) -> List[str]:
        problems = []
        for k in self.order:
            t = self.tasks[k]
            if t.power_kW > self.cap:
                problems.append(f"{k} draws {t.power_kW:.0f} kW > available P_res {self.cap:.1f} kW")
            if t.technicians > self.ntech:
                problems.append(f"{k} needs {t.technicians} technicians > {self.ntech}")
        est = {k: 0 for k in self.order}
        for _ in range(len(self.order)):
            for (a, b), gap in self.prec.items():
                est[b] = max(est[b], est[a] + self.tasks[a].duration + gap)
        for k in self.order:
            t = self.tasks[k]
            end = est[k] + t.duration
            limit = self.H if t.deadline is None else min(self.H, t.deadline)
            if end > limit:
                problems.append(f"critical path forces {k} to end at slot {end} > limit {limit}")
        energy = sum(t.power_kW * t.duration for t in self.tasks.values())
        if energy > self.cap * self.H + 1e-9:
            problems.append(f"energy bound: sum p*d = {energy:.0f} > P_res*H = {self.cap * self.H:.0f}")
        for k in self.order:
            if not self.domains[k]:
                problems.append(f"empty domain for {k}")
        return problems

    def _binary_ok(self, a: str, va: Value, b: str, vb: Value) -> bool:
        sa, ra = va
        sb, rb = vb
        ea, eb = sa + self.tasks[a].duration, sb + self.tasks[b].duration
        g = self.prec.get((a, b))
        if g is not None and sb < ea + g:
            return False
        g = self.prec.get((b, a))
        if g is not None and sa < eb + g:
            return False
        overlap = sa < eb and sb < ea
        if overlap and frozenset((a, b)) in self.no_overlap:
            return False
        if overlap and set(ra) & set(rb):
            return False
        return True
```

```
    def _cumulative_ok(self, k: str, v: Value, power: List[float], tech: List[int]) -> bool:
        t = self.tasks[k]
        for slot in range(v[0], v[0] + t.duration):
            if power[slot] + t.power_kW > self.cap + 1e-9:
                return False
            if tech[slot] + t.technicians > self.ntech:
                return False
        return True

    def solve(self, use_mrv=True, use_degree=True, use_fc=True,
              node_limit=300_000, trace_limit=4000) -> CSPResult:
        t0 = time.perf_counter()
        stats = {"nodes": 0, "backtracks": 0, "checks": 0, "prunes": 0, "wipeouts": 0}
        trace: List[str] = []

        def log(msg, depth):
            if len(trace) < trace_limit:
                trace.append("  " * depth + msg)

        pre = self.prechecks()
        if pre:
            for p in pre:
                log("PRECHECK FAIL: " + p, 0)
            return CSPResult(False, {}, 0, 0, 0, 0, 0, 0, trace, "; ".join(pre),
                             time.perf_counter() - t0)

        domains = {k: list(v) for k, v in self.domains.items()}
        assignment: Dict[str, Value] = {}
        power = [0.0] * (self.H + 1)
        tech = [0] * (self.H + 1)
        limit_hit = [False]

        def select_var() -> str:
            unassigned = [k for k in self.order if k not in assignment]
            if not use_mrv:
                return unassigned[0]
            def key(k):
                deg = sum(1 for n in self.neighbors[k] if n not in assignment) if use_degree else 0
                return (len(domains[k]), -deg, self.order.index(k))
            return min(unassigned, key=key)

        def consistent(k: str, v: Value) -> bool:
            for other, ov in assignment.items():
                stats["checks"] += 1
                if not self._binary_ok(k, v, other, ov):
                    return False
            stats["checks"] += 1
            return self._cumulative_ok(k, v, power, tech)

        def apply(k, v, sign):
            t = self.tasks[k]
            for slot in range(v[0], v[0] + t.duration):
                power[slot] += sign * t.power_kW
                tech[slot] += sign * t.technicians

        def fmt(k, v):
            return f"{k}=(start {v[0]}, end {v[0] + self.tasks[k].duration}, {'/'.join(v[1]) or '-'})"

        def backtrack(depth: int) -> bool:
            if len(assignment) == len(self.order):
                return True
            if stats["nodes"] >= node_limit:
                limit_hit[0] = True
                return False
            k = select_var()
            deg = sum(1 for n in self.neighbors[k] if n not in assignment)
            log(f"SELECT {k} (MRV |D|={len(domains[k])}, degree={deg})", depth)
            for v in list(domains[k]):
                stats["nodes"] += 1
                if not use_fc and not consistent(k, v):
                    continue
                if use_fc:
                    stats["checks"] += 1
                    if not self._cumulative_ok(k, v, power, tech):
                        continue
                assignment[k] = v
                apply(k, v, +1)
                log(f"ASSIGN {fmt(k, v)}", depth)
                removed = {}
                wipeout = None
                if use_fc:
                    for u in self.order:
                        if u in assignment:
                            continue
                        keep = []
                        for vu in domains[u]:
                            stats["checks"] += 2
                            if self._binary_ok(k, v, u, vu) and self._cumulative_ok(u, vu, power, tech):
                                keep.append(vu)
                        if len(keep) != len(domains[u]):
                            removed[u] = domains[u]
                            stats["prunes"] += len(domains[u]) - len(keep)
                            domains[u] = keep
                        if not keep:
                            wipeout = u
```

```
                        break
                if removed and wipeout is None:
                    log("FC: " + ", ".join(f"{u} {len(removed[u])}->{len(domains[u])}"
                                           for u in removed), depth + 1)
            if wipeout is None and backtrack(depth + 1):
                return True
            if wipeout is not None:
                stats["wipeouts"] += 1
                log(f"FC WIPEOUT: domain of {wipeout} empty -> undo {k}", depth + 1)
            for u, old in removed.items():
                domains[u] = old
            apply(k, v, -1)
            del assignment[k]
            if limit_hit[0]:
                return False
        stats["backtracks"] += 1
        log(f"BACKTRACK from {k}: no consistent value left", depth)
        return False

    ok = backtrack(0)
    elapsed = time.perf_counter() - t0
    if not ok:
        reason = ("node limit reached" if limit_hit[0]
                  else "search exhausted: no assignment satisfies all constraints")
        return CSPResult(False, {}, 0, stats["nodes"], stats["backtracks"], stats["checks"],
                         stats["prunes"], stats["wipeouts"], trace, reason, elapsed)
    schedule = {}
    for k in self.order:
        s, units = assignment[k]
        t = self.tasks[k]
        needs = sorted(t.needs)
        schedule[k] = {"start": s, "end": s + t.duration, "duration": t.duration,
                       "resources": dict(zip(needs, units)), "power_kW": t.power_kW,
                       "technicians": t.technicians, "name": t.name}
    makespan = max(v["end"] for v in schedule.values())
    return CSPResult(True, schedule, makespan, stats["nodes"], stats["backtracks"],
                     stats["checks"], stats["prunes"], stats["wipeouts"], trace, "", elapsed)
def solve_recovery_csp(req: Requirements, lab: LabSpec, horizon: Optional[int] = None,
                       **kw) -> Tuple[CSPResult, "RecoveryCSP", List[str]]:
    csp = RecoveryCSP(req, lab, horizon)
    bounds_log = csp.propagate_bounds()
    return csp.solve(**kw), csp, bounds_log
```

A.8 jarvis/pipeline.py

```
from __future__ import annotations

import math
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Tuple

from .annealing import NanoAllocationProblem, simulated_annealing
from .csp import RecoveryCSP, solve_recovery_csp
from .data_models import (CSPResult, MissionDataset, Requirements, SAResult, UCSResult)
from .requirements import derive_requirements
from .ucs import uniform_cost_search
from .validation import validate_csp, validate_integrated, validate_sa, validate_ucs
@dataclass
class Attempt:
    number: int
    power_fraction: float
    excluded_edges: List[Tuple[str, str]]
    ucs: Optional[UCSResult] = None
    sa: Optional[SAResult] = None
    req: Optional[Requirements] = None
    csp: Optional[CSPResult] = None
    outcome: str = ""
    action: str = ""
@dataclass
class MissionOutcome:
    success: bool
    attempts: List[Attempt]
    ucs: Optional[UCSResult]
    sa: Optional[SAResult]
    req: Optional[Requirements]
    csp: Optional[CSPResult]
    csp_first: Optional[CSPResult]
    csp_model: Optional[RecoveryCSP]
    bounds_log: List[str]
    makespan_search: List[dict]
    checks: List[Dict]
    health_after: Dict[str, float] = field(default_factory=dict)
    status: str = ""
def _worst_edge(ucs: UCSResult, ds: MissionDataset) -> Tuple[str, str]:
    edges = [ds.network.edge(a, b) for a, b in zip(ucs.path, ucs.path[1:])]
    e = max(edges, key=lambda e: e.loss_kW)
    return (e.u, e.v)
def minimize_makespan(req: Requirements, ds: MissionDataset, first: CSPResult,
                      node_limit: int = 200_000):
    best, log = first, []
    H = first.makespan - 1
    while H > 0:
        res, model, _ = solve_recovery_csp(req, ds.lab, horizon=H, node_limit=node_limit)
```

```
        log.append({"horizon": H, "solved": res.solved, "nodes": res.nodes,
                    "reason": res.failure_reason if not res.solved else f"makespan {res.makespan}"})
        if not res.solved:
            break
        best = res
        H = res.makespan - 1
    return best, log
def estimate_health(ds: MissionDataset, sa: SAResult) -> Dict[str, float]:
    serve = {"STR": "SR", "FLC": "FS", "STB": "FS", "SEN": "SN", "LSC": "ES"}
    cats = {c.id: c for c in ds.nano.categories}
    out = {}
    for s in ds.subsystems:
        c = cats[serve[s["id"]]]
        phi = 1 - math.exp(-sa.allocation[c.id] / c.saturation_NU)
        out[s["id"]] = round(s["health_pct"] + (100 - s["health_pct"]) * phi, 1)
    return out
def run_mission(ds: MissionDataset, seed: Optional[int] = None, verbose: bool = True) -> MissionOutcome:
    seed = ds.mission["seed"] if seed is None else seed
    rules = ds.rules
    rho = ds.nano.initial_power_fraction
    excluded: List[Tuple[str, str]] = []
    attempts: List[Attempt] = []
    say = print if verbose else (lambda *a, **k: None)

    for n in range(1, ds.mission["max_replanning_attempts"] + 1):
        att = Attempt(n, rho, list(excluded))
        attempts.append(att)
        say(f"\n=== JARVIS planning attempt {n}: rho={rho:.2f}, excluded edges={excluded or 'none'} ===")

        ucs = uniform_cost_search(ds.network, ds.reactor["available_output_kW"],
                                  ds.critical_load_kW, excluded_edges=excluded)
        att.ucs = ucs
        if not ucs.found or ucs.power_budget_kW <= 0:
            att.outcome = "UCS: no admissible route" if not ucs.found else "UCS: route budget <= 0"
            if not ucs.found:
                att.action = "ABORT: no safe route exists - manual intervention required"
                say(att.outcome, "->", att.action)
                break
            excluded.append(_worst_edge(ucs, ds))
            att.action = f"exclude edge {excluded[-1]} and re-route"
            say(att.outcome, "->", att.action)
            continue
        say(f"[UCS] route {' -> '.join(ucs.path)} cost={ucs.path_cost:.2f} "
            f"delivered={ucs.delivered_power_kW:.1f} kW budget B={ucs.power_budget_kW:.1f} kW")

        problem = NanoAllocationProblem(ds.nano, ucs.power_budget_kW, rho, ucs.total_thermal_C)
        sa = simulated_annealing(problem, ds.annealing, seed)
        att.sa = sa
        if not sa.feasible:
            att.outcome = "SA: " + sa.termination
            excluded.append(_worst_edge(ucs, ds))
            att.action = f"route too weak: exclude edge {excluded[-1]} and re-route"
            say(att.outcome, "->", att.action)
            continue
        say(f"[SA ] allocation {sa.allocation} J={sa.objective:.3f} P_nano={sa.nano_power_kW:.1f} kW "
            f"P_res={sa.residual_power_kW:.1f} kW")

        req = derive_requirements(ds, ucs, sa)
        att.req = req
        csp_res, model, bounds_log = solve_recovery_csp(req, ds.lab)
        att.csp = csp_res
        if not csp_res.solved:
            att.outcome = "CSP infeasible: " + csp_res.failure_reason
            new_rho = round(rho - rules["replan_power_fraction_step"], 4)
            if new_rho >= rules["min_power_fraction"] - 1e-9:
                att.action = (f"reduce nanomaterial power fraction rho {rho:.2f} -> {new_rho:.2f} "
                              f"to free laboratory power; re-run SA and CSP")
                rho = new_rho
            else:
                excluded.append(_worst_edge(ucs, ds))
                att.action = f"rho at minimum: exclude edge {excluded[-1]}, reset rho, re-route"
                rho = ds.nano.initial_power_fraction
            say("[CSP]", att.outcome, "->", att.action)
            continue
        say(f"[CSP] schedule found: makespan={csp_res.makespan} slots, nodes={csp_res.nodes}, "
            f"backtracks={csp_res.backtracks}")

        best, ms_log = minimize_makespan(req, ds, csp_res)
        att.outcome = "feasible"
        att.action = "validate and commit"
        checks = (validate_ucs(ds, ucs) + validate_sa(ds, ucs, sa) +
                  validate_csp(ds, req, best) + validate_integrated(ds, ucs, sa, req, best))
        success = all(c["passed"] for c in checks)
        health = estimate_health(ds, sa)
        status = ("ARMOR RESTORED - OPERATIONAL AND SAFE" if success
                  else "RECOVERY PLAN REJECTED BY VALIDATION")
        return MissionOutcome(success, attempts, ucs, sa, req, best, csp_res, model, bounds_log,
                              ms_log, checks, health, status)

    last = attempts[-1]
    status = ("RECOVERY ABORTED - no safe power route; armor held in safe mode for manual intervention"
              if last.action.startswith("ABORT") else "RECOVERY FAILED - replanning budget exhausted")
    return MissionOutcome(False, attempts, last.ucs, last.sa, last.req, last.csp, last.csp, None,
                          [], [], [], {}, status)
```

A.9 jarvis/validation.py

```
from __future__ import annotations

import math
from typing import Dict, List

from .annealing import NanoAllocationProblem
from .data_models import CSPResult, MissionDataset, Requirements, SAResult, UCSResult
from .ucs import enumerate_simple_paths
def _check(stage: str, name: str, ok: bool, detail: str) -> Dict:
    return {"stage": stage, "check": name, "passed": bool(ok), "detail": detail}
def validate_ucs(ds: MissionDataset, u: UCSResult) -> List[Dict]:
    net, out = ds.network, []
    out.append(_check("UCS", "route found", u.found, " -> ".join(u.path)))
    if not u.found:
        return out
    edges = [net.edge(a, b) for a, b in zip(u.path, u.path[1:])]
    out.append(_check("UCS", "starts at source / ends at goal",
                      u.path[0] == net.source and u.path[-1] == net.goal,
                      f"{u.path[0]} ... {u.path[-1]}"))
    failed = [p for p in u.path if net.nodes[p].failed]
    out.append(_check("UCS", "avoids FAILED components", not failed, f"failed on route: {failed or 'none'}"))
    mincap = min(e.capacity_kW for e in edges)
    out.append(_check("UCS", "capacity >= required throughput", mincap >= net.required_throughput_kW,
                      f"bottleneck {mincap:.0f} kW >= {net.required_throughput_kW:.0f} kW"))
    th = sum(e.thermal_C for e in edges)
    out.append(_check("UCS", "route thermal <= limit", th <= net.thermal_limit_C + 1e-9,
                      f"{th:.2f} C <= {net.thermal_limit_C:.2f} C"))
    cost = sum(e.cost(net.cost_weights) for e in edges)
    out.append(_check("UCS", "reported cost = recomputed cost", math.isclose(cost, u.path_cost, abs_tol=1e-6),
                      f"{cost:.4f} vs {u.path_cost:.4f}"))
    feas = [r for r in enumerate_simple_paths(net)
            if r["feasible"] and not any((a, b) in set(map(tuple, u.excluded_edges))
                                         for a, b in zip(r["path"], r["path"][1:]))]
    best = min(r["cost"] for r in feas)
    out.append(_check("UCS", "optimal vs exhaustive enumeration", math.isclose(best, u.path_cost, abs_tol=1e-6),
                      f"UCS {u.path_cost:.2f} = best of {len(feas)} feasible routes {best:.2f}"))
    out.append(_check("UCS", "delivered power covers critical load", u.power_budget_kW > 0,
                      f"delivered {u.delivered_power_kW:.1f} kW - critical {u.critical_load_kW:.0f} kW = "
                      f"budget {u.power_budget_kW:.1f} kW"))
    return out
def validate_sa(ds: MissionDataset, u: UCSResult, s: SAResult) -> List[Dict]:
    prob = NanoAllocationProblem(ds.nano, u.power_budget_kW, s.power_fraction, u.total_thermal_C)
    x, out = s.allocation, []
    for c in ds.nano.categories:
        out.append(_check("SA", f"{c.id} within [{c.min_NU}, {c.max_NU}] NU",
                          c.min_NU <= x[c.id] <= c.max_NU, f"x_{c.id} = {x[c.id]}"))
    m = prob.material(x)
    out.append(_check("SA", "material <= reserve", m <= ds.nano.reserve_NU, f"{m} <= {ds.nano.reserve_NU} NU"))
    p = prob.power(x)
    out.append(_check("SA", "nano power <= rho * B", p <= prob.power_cap + 1e-9,
                      f"{p:.1f} <= {prob.power_cap:.1f} kW (rho={s.power_fraction:.2f})"))
    th = u.total_thermal_C + prob.heat(x)
    out.append(_check("SA", "armor thermal <= limit", th <= ds.nano.armor_thermal_limit_C + 1e-9,
                      f"{th:.2f} <= {ds.nano.armor_thermal_limit_C:.1f} C"))
    J = prob.objective(x)
    out.append(_check("SA", "reported objective = recomputed", math.isclose(J, s.objective, abs_tol=1e-9),
                      f"J = {J:.4f}"))
    out.append(_check("SA", "final J >= initial J", s.objective >= s.initial_objective - 1e-9,
                      f"{s.objective:.3f} >= {s.initial_objective:.3f}"))
    return out
def validate_csp(ds: MissionDataset, req: Requirements, c: CSPResult) -> List[Dict]:
    out = [_check("CSP", "schedule found", c.solved, c.failure_reason or f"makespan {c.makespan} slots")]
    if not c.solved:
        return out
    sch = c.schedule
    tasks = {t.id: t for t in req.tasks}
    out.append(_check("CSP", "every task scheduled", set(sch) == set(tasks), f"{len(sch)}/{len(tasks)} tasks"))
    bad = [f"{p}->{k}" for k, t in tasks.items() for p in t.after if sch[k]["start"] < sch[p]["end"]]
    out.append(_check("CSP", "C1 precedence", not bad, ", ".join(bad) or "all dependencies respected"))
    bad = []
    for s in ds.lab.separations:
        gap = sch[s["second"]]["start"] - sch[s["first"]]["end"]
        if gap < s["min_gap_slots"]:
            bad.append(f"{s['first']}->{s['second']} gap {gap}")
    out.append(_check("CSP", "C2 safety separation", not bad, ", ".join(bad) or "cool-down gaps respected"))

    def overlap(a, b):
        return sch[a]["start"] < sch[b]["end"] and sch[b]["start"] < sch[a]["end"]

    bad = [f"{n['a']}|{n['b']}" for n in ds.lab.no_overlap if overlap(n["a"], n["b"])]
    out.append(_check("CSP", "C3 safety no-overlap", not bad, ", ".join(bad) or "no forbidden overlaps"))
    bad, ids = [], list(sch)
    for i in range(len(ids)):
        for j in range(i + 1, len(ids)):
            a, b = ids[i], ids[j]
            shared = set(sch[a]["resources"].values()) & set(sch[b]["resources"].values())
            if shared and overlap(a, b):
                bad.append(f"{a}/{b} on {shared}")
    out.append(_check("CSP", "C4 resource exclusivity", not bad, ", ".join(bad) or "no unit double-booked"))
    bad = []
    units = {u.id: u for lst in ds.lab.resources.values() for u in lst}
```

```
    for k, v in sch.items():
        for rtype, uid in v["resources"].items():
            if tasks[k].needs[rtype] not in units[uid].capabilities:
                bad.append(f"{k} on {uid}")
    out.append(_check("CSP", "equipment compatibility", not bad, ", ".join(bad) or "all units capable"))
    H = req.horizon
    power = [sum(v["power_kW"] for v in sch.values() if v["start"] <= t < v["end"]) for t in range(H)]
    techs = [sum(v["technicians"] for v in sch.values() if v["start"] <= t < v["end"]) for t in range(H)]
    out.append(_check("CSP", "C5 power profile <= P_res", max(power) <= req.power_cap_kW + 1e-9,
                      f"peak {max(power):.0f} kW <= {req.power_cap_kW:.1f} kW"))
    out.append(_check("CSP", "C6 technicians", max(techs) <= req.technicians,
                      f"peak {max(techs)} <= {req.technicians}"))
    bad = [k for k, t in tasks.items() if (t.deadline is not None and sch[k]["end"] > t.deadline)
           or sch[k]["end"] > H]
    out.append(_check("CSP", "C7 deadlines / horizon", not bad, ", ".join(bad) or f"makespan {c.makespan} <= {H}"))
    durs = [k for k, t in tasks.items() if sch[k]["end"] - sch[k]["start"] != t.duration]
    out.append(_check("CSP", "durations match derived requirements", not durs, ", ".join(durs) or "ok"))
    return out
def validate_integrated(ds: MissionDataset, u: UCSResult, s: SAResult,
                        req: Requirements, c: CSPResult) -> List[Dict]:
    out = []
    peak_lab = 0.0
    if c.solved:
        peak_lab = max(sum(v["power_kW"] for v in c.schedule.values() if v["start"] <= t < v["end"])
                       for t in range(req.horizon))
    demand = u.critical_load_kW + s.nano_power_kW + peak_lab
    out.append(_check("INTEGRATED", "power balance  P_crit + P_nano + peak P_lab <= P_delivered",
                      demand <= u.delivered_power_kW + 1e-9,
                      f"{u.critical_load_kW:.0f} + {s.nano_power_kW:.1f} + {peak_lab:.0f} = {demand:.1f} "
                      f"<= {u.delivered_power_kW:.1f} kW"))
    out.append(_check("INTEGRATED", "reactor balance  P_delivered + losses <= P_avail",
                      u.delivered_power_kW + u.total_loss_kW <= ds.reactor["available_output_kW"] + 1e-9,
                      f"{u.delivered_power_kW:.0f} + {u.total_loss_kW:.0f} <= {ds.reactor['available_output_kW']} kW"))
    out.append(_check("INTEGRATED", "thermal  theta_route + theta_nano <= Theta_armor",
                      s.total_thermal_C <= ds.nano.armor_thermal_limit_C + 1e-9,
                      f"{u.total_thermal_C:.2f} + {s.nano_heat_C:.2f} = {s.total_thermal_C:.2f} "
                      f"<= {ds.nano.armor_thermal_limit_C} C"))
    sp_min = next(cat.min_NU for cat in ds.nano.categories if cat.id == "SP")
    out.append(_check("INTEGRATED", "mandatory structural protection",
                      s.allocation["SP"] >= sp_min, f"x_SP = {s.allocation['SP']} >= {sp_min} NU"))
    out.append(_check("INTEGRATED", "consumed nanomaterial is replenished",
                      c.solved and "NANO_REPL" in c.schedule,
                      f"{s.material_used_NU} NU replenished in slots "
                      f"{c.schedule['NANO_REPL']['start']}-{c.schedule['NANO_REPL']['end']}" if c.solved else "no schedule"))
    r = ds.rules
    deficit = r["armor_battery_capacity_kWh"] * (r["target_state_of_charge"] - r["armor_state_of_charge"])
    if c.solved:
        slots = c.schedule["CHARGE"]["duration"]
        energy = slots * r["charger_rating_kW"] * u.route_efficiency * ds.mission["slot_minutes"] / 60
    else:
        energy = 0.0
    out.append(_check("INTEGRATED", "armor recharged to target state of charge", energy >= deficit - 1e-9,
                      f"{energy:.1f} kWh delivered >= {deficit:.1f} kWh deficit"))
    out.append(_check("INTEGRATED", "mission completes before deadline",
                      c.solved and c.makespan <= ds.mission["horizon_slots"],
                      f"{c.makespan} slots x {ds.mission['slot_minutes']} min = "
                      f"{c.makespan * ds.mission['slot_minutes'] / 60:.1f} h <= "
                      f"{ds.mission['horizon_slots'] * ds.mission['slot_minutes'] / 60:.1f} h"))
    return out
```

A.10 jarvis/visualization.py

```
from __future__ import annotations

from pathlib import Path
from typing import Dict, List

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch, Patch
import networkx as nx

C = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300", "#4a3aa7", "#e34948"]
INK, INK2, MUTED, GRID, SURF = "#0b0b0b", "#52514e", "#8a8984", "#e4e3df", "#fcfcfb"
GOOD, BAD = "#1a7f37", "#c62828"
FOOT = "FICTIONAL ENGINEERING SIMULATION - not measurements of a real physical system"

plt.rcParams.update({
    "font.family": "DejaVu Sans", "font.size": 9, "text.color": INK,
    "axes.edgecolor": MUTED, "axes.labelcolor": INK, "axes.titleweight": "bold",
    "axes.titlesize": 11, "axes.titlecolor": INK, "xtick.color": INK2, "ytick.color": INK2,
    "axes.grid": False, "grid.color": GRID, "grid.linewidth": 0.6,
    "figure.facecolor": "white", "axes.facecolor": "white", "savefig.dpi": 160,
    "axes.spines.top": False, "axes.spines.right": False,
})
def _finish(fig, path: Path, title: str = None):
    if title:
        fig.suptitle(title, fontsize=13, fontweight="bold", color=INK)
    fig.text(0.995, 0.005, FOOT, ha="right", va="bottom", fontsize=6.5, color=MUTED, style="italic")
    fig.savefig(path, bbox_inches="tight")
```

```
    plt.close(fig)
    return str(path)
def _box(ax, x, y, w, h, text, fc="#f3f6fb", ec=INK2, fs=8.5, bold=False):
    ax.add_patch(FancyBboxPatch((x - w / 2, y - h / 2), w, h, boxstyle="round,pad=0.02,rounding_size=0.08",
                                fc=fc, ec=ec, lw=1.2))
    ax.text(x, y, text, ha="center", va="center", fontsize=fs, color=INK,
            fontweight="bold" if bold else "normal", wrap=True)
def _arrow(ax, p, q, text=None, color=INK2, style="-|>", ls="-", rad=0.0, fs=7.5, off=(0, 0.12)):
    ax.add_patch(FancyArrowPatch(p, q, arrowstyle=style, mutation_scale=12, color=color, lw=1.3,
                                 linestyle=ls, connectionstyle=f"arc3,rad={rad}"))
    if text:
        ax.text((p[0] + q[0]) / 2 + off[0], (p[1] + q[1]) / 2 + off[1], text, ha="center",
                va="center", fontsize=fs, color=INK2,
                bbox=dict(fc="white", ec="none", pad=0.5))
def architecture(path: Path):
    fig, ax = plt.subplots(figsize=(12, 7))
    ax.set_xlim(0, 12); ax.set_ylim(0, 7); ax.axis("off")
    _box(ax, 2.0, 6.2, 3.7, 0.8, "FRIDAY Fault Monitor\nreactor / converter / nanite telemetry", "#fdecea", bold=True)
    _box(ax, 6.0, 6.2, 3.4, 0.8, "Shared Mission Dataset (JSON)\nnetwork, reserve, lab, deadlines", "#eef4fb", bold=True)
    _box(ax, 10.0, 6.2, 3.0, 0.8, "Tony Stark\nactivates JARVIS recovery", "#f4f4f2")
    _box(ax, 6.0, 4.9, 9.6, 0.7, "JARVIS Orchestrator  -  sequencing, data hand-off, feasibility monitoring, controlled
replanning",
         "#e8f0fb", bold=True)
    _box(ax, 2.0, 3.4, 3.2, 1.3, "Stage 1  UCS\nPower Routing Engine\nmin-cost safe route\n-> power budget B, eta, theta",
"#ddeaf9")
    _box(ax, 6.0, 3.4, 3.2, 1.3, "Stage 2  Simulated Annealing\nNanomaterial Optimiser\nx* under rho*B, reserve, heat\n-> P_res,
repair needs", "#fde7dc")
    _box(ax, 10.0, 3.4, 3.2, 1.3, "Stage 3  CSP\nLab Scheduler\nMRV + degree + forward checking\n-> conflict-free schedule",
"#dcf3ea")
    _box(ax, 3.9, 1.6, 4.0, 0.9, "Validation Engine\nindependent re-check of every constraint", "#f4f4f2", bold=True)
    _box(ax, 8.6, 1.6, 4.2, 0.9, "Visualisation + JARVIS Recovery Dashboard\nfigures, Gantt chart, status report", "#f4f4f2",
bold=True)
    _box(ax, 6.0, 0.45, 5.0, 0.55, "Mark 85 armor: restored, safe, operational", "#e3f4e6", bold=True)
    _arrow(ax, (3.85, 6.2), (4.3, 6.2)); _arrow(ax, (8.5, 6.2), (7.7, 6.2))
    _arrow(ax, (6.0, 5.8), (6.0, 5.25))
    for x in (2.0, 6.0, 10.0):
        _arrow(ax, (x, 4.55), (x, 4.05))
    _arrow(ax, (3.6, 3.4), (4.4, 3.4), "B, theta", off=(0, 0.2))
    _arrow(ax, (7.6, 3.4), (8.4, 3.4), "x*, P_res", off=(0, 0.2))
    _arrow(ax, (8.4, 3.0), (7.6, 3.0), "replan: lower rho", color=C[7], ls="--", off=(0, -0.2))
    _arrow(ax, (4.4, 3.0), (3.6, 3.0), "replan: re-route", color=C[7], ls="--", off=(0, -0.2))
    _arrow(ax, (2.0, 2.75), (3.2, 2.05)); _arrow(ax, (6.0, 2.75), (4.6, 2.05)); _arrow(ax, (10.0, 2.75), (5.6, 2.0))
    _arrow(ax, (5.9, 1.6), (6.5, 1.6)); _arrow(ax, (8.6, 1.15), (6.8, 0.75))
    return _finish(fig, path, "Figure 1. JARVIS system architecture for the Mark 85 recovery mission")
NET_POS = {"ARC": (0, 0), "PC1": (1, 1.6), "PC2": (1, 0.55), "PC3": (1, -0.5), "BKP": (1, -1.6),
           "SW1": (2, 1.1), "SW2": (2, 0.0), "SW3": (2, -1.6), "BUS_A": (3, 1.6), "BUS_B": (3, 0.45),
           "BUS_C": (3, -1.0), "CLH": (4, 0.0), "FLC": (5, 1.5), "STB": (5, 0.5), "SEN": (5, -0.5),
           "LSC": (5, -1.5)}
def power_network(ds, ucs, path: Path):
    net = ds.network
    G = nx.DiGraph()
    for n in net.nodes.values():
        G.add_node(n.id)
    fig, ax = plt.subplots(figsize=(13, 7.2))
    route_edges = set(zip(ucs.path, ucs.path[1:]))
    w = net.cost_weights
    for e in net.edges:
        on = (e.u, e.v) in route_edges
        failed = net.nodes[e.u].failed or net.nodes[e.v].failed
        lowcap = e.capacity_kW < net.required_throughput_kW
        color = C[0] if on else (BAD if failed else (MUTED if lowcap else "#b9b8b3"))
        style = "dashed" if (failed or lowcap) else "solid"
        nx.draw_networkx_edges(G, NET_POS, edgelist=[(e.u, e.v)], ax=ax, edge_color=color,
                               width=3.4 if on else 1.3, style=style, arrows=True, arrowsize=14,
                               node_size=1500, connectionstyle="arc3,rad=0.0")
        (x1, y1), (x2, y2) = NET_POS[e.u], NET_POS[e.v]
        ax.text(x1 + (x2 - x1) * 0.5, y1 + (y2 - y1) * 0.5 + 0.09,
                f"c={e.cost(w):.2f}\nL{e.loss_kW:g} T{e.thermal_C:g} S{e.switch_ms:g}\ncap {e.capacity_kW:g}",
                fontsize=6.3, ha="center", va="center", color=INK if on else INK2,
                fontweight="bold" if on else "normal",
                bbox=dict(fc="white", ec="none", alpha=0.85, pad=0.4))
    nx.draw_networkx_edges(G, NET_POS, edgelist=net.load_edges, ax=ax, edge_color=C[2], width=1.6,
                           style="dotted", arrows=True, arrowsize=12, node_size=1500)
    colors = []
    for n in G.nodes:
        node = net.nodes[n]
        if node.failed:
            colors.append("#f6c9c4")
        elif n in ucs.path:
            colors.append("#cfe1f7")
        elif node.type == "load":
            colors.append("#d3f1e5")
        else:
            colors.append("#f1f0ec")
    nx.draw_networkx_nodes(G, NET_POS, ax=ax, node_color=colors, node_size=1500,
                           edgecolors=[BAD if net.nodes[n].failed else INK2 for n in G.nodes], linewidths=1.4)
    nx.draw_networkx_labels(G, NET_POS, ax=ax, font_size=8, font_weight="bold")
    for n, (x, y) in NET_POS.items():
        node = net.nodes[n]
        sub = node.label + (" [FAILED]" if node.failed else "")
        ax.text(x, y - 0.27, sub, ha="center", fontsize=6.5, color=BAD if node.failed else INK2)
    for s in ds.subsystems:
        if s["id"] in NET_POS:
            x, y = NET_POS[s["id"]]
```

```
            ax.text(x + 0.33, y, f"{s['critical_load_kW']} kW", fontsize=7, va="center", color=INK2)
    legend = [Line2D([0], [0], color=C[0], lw=3.4, label=f"UCS optimal route (cost {ucs.path_cost:.2f})"),
              Line2D([0], [0], color=BAD, lw=1.3, ls="--", label="touches FAILED converter PC1"),
              Line2D([0], [0], color=MUTED, lw=1.3, ls="--", label=f"capacity < {net.required_throughput_kW:g} kW"),
              Line2D([0], [0], color="#b9b8b3", lw=1.3, label="admissible, not selected"),
              Line2D([0], [0], color=C[2], lw=1.6, ls=":", label="critical load feeders")]
    ax.legend(handles=legend, loc="lower left", fontsize=7.5, frameon=False)
    ax.text(0.0, 2.25, f"c(e) = {w['loss']}*loss[kW] + {w['thermal']}*thermal[C] + {w['switching']}*switching[ms]   |   "
            f"route thermal limit {net.thermal_limit_C:g} C   |   P_avail = {ds.reactor['available_output_kW']} kW",
            fontsize=8, color=INK2)
    ax.set_ylim(-2.2, 2.45); ax.axis("off")
    return _finish(fig, path, "Figure 2. Weighted Arc Reactor power-distribution network and UCS optimal route")
def ucs_expansion(ucs, net, path: Path):
    fig, ax = plt.subplots(figsize=(11, 4.2))
    steps = ucs.trace
    xs = [s["step"] for s in steps]; gs = [s["g"] for s in steps]
    ax.plot(xs, gs, color=C[0], lw=2, marker="o", ms=8, markeredgecolor="white", markeredgewidth=2)
    for s in steps:
        lab = s["expand"] + (" (GOAL)" if s.get("goal") else "")
        up = s["step"] % 2 == 1
        ax.annotate(f"{lab}\ng={s['g']:.2f}\nth={s['theta']:.1f}", (s["step"], s["g"]),
                    textcoords="offset points", xytext=(0, 12 if up else -14), ha="center",
                    va="bottom" if up else "top", fontsize=6.8, color=INK)
    ax.set_xlabel("Expansion step (order in which UCS pops labels from the priority queue)")
    ax.set_ylabel("Cumulative path cost g(n)")
    ax.grid(axis="y")
    ax.set_xticks(xs)
    ax.set_ylim(-14, max(gs) * 1.3)
    ax.set_yticks(range(0, int(max(gs) * 1.3) + 1, 10))
    return _finish(fig, path, "Figure 4. UCS node-expansion order with cumulative cost g(n) (non-decreasing)")
def sa_convergence(sa, path: Path):
    h = sa.history
    it = [r["iter"] for r in h]
    fig, (a1, a2) = plt.subplots(2, 1, figsize=(11, 6.2), sharex=True,
                                 gridspec_kw={"height_ratios": [3, 1.4]})
    a1.plot(it, [r["cur_J"] for r in h], color=C[0], lw=1.0, alpha=0.75, label="current solution J(x)")
    a1.plot(it, [r["best_J"] for r in h], color=C[1], lw=2.0, label="best-so-far J(x*)")
    worse = [r for r in h if r["decision"].startswith("ACCEPT-WORSE")]
    a1.scatter([r["iter"] for r in worse], [r["cur_J"] for r in worse], s=9, color=C[6], zorder=3,
               label=f"accepted worse move ({len(worse)})")
    a1.set_ylabel("Readiness objective J")
    a1.grid(axis="y"); a1.legend(frameon=False, loc="lower right")
    a1.set_title(f"rho = {sa.power_fraction:.2f}, cap = {sa.power_cap_kW:.1f} kW  |  "
                 f"J: {sa.initial_objective:.2f} -> {sa.objective:.3f}  |  {sa.iterations} iterations", fontsize=9,
                 fontweight="normal", color=INK2)
    a2.plot(it, [r["T"] for r in h], color=C[3], lw=1.8)
    a2.set_yscale("log"); a2.set_ylabel("Temperature T (log)"); a2.set_xlabel("Iteration")
    a2.grid(axis="y")
    return _finish(fig, path, "Figure 6. Simulated Annealing convergence and geometric cooling schedule")
def nano_allocation(ds, attempts, final_sa, path: Path):
    cats = ds.nano.categories
    rejected = [a.sa for a in attempts if a.sa is not None and a.sa is not final_sa and a.sa.feasible]
    series = []
    if rejected:
        series.append((f"Attempt 1 (rho={rejected[0].power_fraction:.2f}) - rejected by CSP", rejected[0].allocation, C[1]))
    series.append((f"Final (rho={final_sa.power_fraction:.2f}) - committed", final_sa.allocation, C[0]))
    fig, (a1, a2) = plt.subplots(1, 2, figsize=(13, 4.8), gridspec_kw={"width_ratios": [1.6, 1]})
    n = len(series); width = 0.8 / n
    for k, (lab, alloc, col) in enumerate(series):
        xs = [i + (k - (n - 1) / 2) * width for i in range(len(cats))]
        bars = a1.bar(xs, [alloc[c.id] for c in cats], width=width - 0.03, color=col, label=lab)
        for b, c in zip(bars, cats):
            a1.text(b.get_x() + b.get_width() / 2, b.get_height() + 6, str(alloc[c.id]), ha="center",
                    fontsize=7.5, color=INK)
    for i, c in enumerate(cats):
        a1.hlines(c.min_NU, i - 0.45, i + 0.45, colors=INK, linestyles="--", lw=1.2)
    a1.plot([], [], ls="--", color=INK, label="mandatory minimum")
    a1.set_xticks(range(len(cats))); a1.set_xticklabels([f"{c.id}\n" + c.name.replace(" ", "\n") for c in cats], fontsize=7.5)
    a1.set_ylabel("Nanomaterial (NU)"); a1.grid(axis="y"); a1.legend(frameon=False, fontsize=8)
    a1.set_title("Allocation per category", fontsize=10)
    used = {c.id: final_sa.allocation[c.id] for c in cats}
    rows = [("Material\n% of reserve", {c.id: 100 * used[c.id] / ds.nano.reserve_NU for c in cats},
             f"{final_sa.material_used_NU}/{ds.nano.reserve_NU} NU"),
            ("Power\n% of cap rho*B", {c.id: 100 * c.power_kW_per_NU * used[c.id] / final_sa.power_cap_kW for c in cats},
             f"{final_sa.nano_power_kW:.1f}/{final_sa.power_cap_kW:.1f} kW")]
    for y, (lab, shares, txt) in enumerate(rows):
        left = 0
        for i, c in enumerate(cats):
            a2.barh([y], [shares[c.id]], left=left, color=C[i], edgecolor="white", linewidth=2,
                    height=0.55, label=c.id if y == 0 else None)
            left += shares[c.id]
        a2.barh([y], [100 - left], left=left, color="#e9e8e4", edgecolor="white", linewidth=2, height=0.55,
                label="unused" if y == 0 else None)
        a2.text(101, y, f"{left:.1f}%\n{txt}", va="center", fontsize=8)
    a2.set_yticks([0, 1]); a2.set_yticklabels([r[0] for r in rows])
    a2.set_xlim(0, 125); a2.set_ylim(-0.6, 2.0); a2.set_xlabel("Utilisation (%)")
    a2.legend(frameon=False, fontsize=7.5, ncol=6, loc="upper center", handlelength=1.2, columnspacing=0.8)
    a2.set_title("Reserve and power-cap utilisation (final)", fontsize=10)
    return _finish(fig, path, "Figure 5. Optimised nanomaterial allocation for the damaged Mark 85")
def csp_constraint_graph(model, path: Path):
    G = nx.Graph()
    G.add_nodes_from(model.order)
    kinds = {"precedence": C[0], "separation": C[1], "no-overlap": C[7]}
    pos = nx.circular_layout(model.order)
```

```
    fig, ax = plt.subplots(figsize=(10.5, 8))
    edges = model.constraint_edges()
    for a, b, kind in edges:
        if kind.startswith("resource"):
            nx.draw_networkx_edges(G, pos, edgelist=[(a, b)], ax=ax, edge_color=MUTED, style="dashed", width=1.0)
            (x1, y1), (x2, y2) = pos[a], pos[b]
            ax.text((x1 + x2) / 2, (y1 + y2) / 2, kind.split(":")[1], fontsize=6.3, color=INK2, ha="center",
                    bbox=dict(fc="white", ec="none", alpha=0.8, pad=0.2))
    for a, b, kind in edges:
        if not kind.startswith("resource"):
            nx.draw_networkx_edges(nx.DiGraph([(a, b)]), pos, edgelist=[(a, b)], ax=ax, edge_color=kinds[kind],
                                   width=2.4, arrows=True, arrowstyle="-|>" if kind != "no-overlap" else "-",
                                   arrowsize=16, node_size=2600)
    nx.draw_networkx_nodes(G, pos, ax=ax, node_size=2600, node_color="#eef4fb", edgecolors=INK2)
    nx.draw_networkx_labels(G, pos, ax=ax, font_size=7, font_weight="bold")
    for k, (x, y) in pos.items():
        t = model.tasks[k]
        ax.text(x, y - 0.17, f"d={t.duration}, {t.power_kW:g} kW, tech {t.technicians}", ha="center",
                fontsize=6.5, color=INK2)
    legend = [Line2D([0], [0], color=C[0], lw=2.4, label="C1 precedence  s_j >= s_i + d_i"),
              Line2D([0], [0], color=C[1], lw=2.4, label="C2 safety separation (cool-down gap)"),
              Line2D([0], [0], color=C[7], lw=2.4, label="C3 safety no-overlap"),
              Line2D([0], [0], color=MUTED, lw=1, ls="--", label="C4 potential shared resource unit"),
              Patch(fc="white", ec="white", label=f"C5/C6 global: power <= {model.cap:.1f} kW, technicians <= {model.ntech}")]
    ax.legend(handles=legend, loc="upper center", bbox_to_anchor=(0.5, -0.01), ncol=2, frameon=False, fontsize=8)
    ax.axis("off")
    return _finish(fig, path, "Figure 7. CSP constraint graph for the laboratory recovery schedule")
def gantt(csp, req, ds, path: Path, title="Figure 8. Stark Laboratory recovery schedule (Gantt chart)"):
    sch = csp.schedule
    order = sorted(sch, key=lambda k: (sch[k]["start"], sch[k]["end"]))
    H = req.horizon
    fig, (a1, a2, a3) = plt.subplots(3, 1, figsize=(12.5, 8.4), sharex=True,
                                     gridspec_kw={"height_ratios": [4, 1.5, 1.1]})
    for i, k in enumerate(order):
        v = sch[k]
        a1.barh(i, v["duration"], left=v["start"], height=0.62, color=C[0], edgecolor="white", linewidth=2)
        a1.text(v["end"] + 0.1, i, f"[{v['start']},{v['end']})  " + " / ".join(v["resources"].values()),
                va="center", fontsize=7, color=INK2)
    a1.set_yticks(range(len(order))); a1.set_yticklabels([f"{k}\n{sch[k]['name']}" for k in order], fontsize=7.2)
    a1.invert_yaxis(); a1.grid(axis="x")
    a1.axvline(H, color=BAD, ls="--", lw=1.2); a1.text(H - 0.1, -0.7, f"deadline H={H}", color=BAD, ha="right", fontsize=8)
    a1.axvline(csp.makespan, color=INK, ls=":", lw=1.2)
    a1.text(csp.makespan + 0.1, len(order) - 0.3, f"makespan {csp.makespan}", fontsize=8, color=INK)
    slots = list(range(H))
    pw = [sum(v["power_kW"] for v in sch.values() if v["start"] <= t < v["end"]) for t in slots]
    tc = [sum(v["technicians"] for v in sch.values() if v["start"] <= t < v["end"]) for t in slots]
    a2.bar([t + 0.5 for t in slots], pw, width=0.94, color=C[2])
    a2.axhline(req.power_cap_kW, color=BAD, ls="--", lw=1.2)
    a2.text(0.1, req.power_cap_kW * 1.04, f"P_res = {req.power_cap_kW:.1f} kW", fontsize=7.5, color=BAD, va="bottom")
    a2.set_ylim(0, req.power_cap_kW * 1.25)
    a2.set_ylabel("Lab power\n(kW)"); a2.grid(axis="y")
    a3.bar([t + 0.5 for t in slots], tc, width=0.94, color=C[6])
    a3.axhline(req.technicians, color=BAD, ls="--", lw=1.2)
    a3.set_ylabel("Technicians"); a3.set_yticks(range(req.technicians + 1)); a3.grid(axis="y")
    a3.set_xlabel(f"Time slot ({ds.mission['slot_minutes']} min each)")
    a3.set_xticks(range(H + 1)); a3.set_xlim(0, H + 4.2)
    return _finish(fig, path, title)
def workflow(out, path: Path):
    u, s, r, c = out.ucs, out.sa, out.req, out.csp
    fig, ax = plt.subplots(figsize=(13, 6.6))
    ax.set_xlim(0, 13); ax.set_ylim(0, 6.6); ax.axis("off")
    _box(ax, 1.3, 5.6, 2.3, 1.2, "Shared dataset\nreactor 1000 kW\nreserve 1200 NU\n9 tasks, H=16", "#eef4fb", bold=True)
    _box(ax, 4.2, 5.6, 2.6, 1.2, f"STAGE 1  UCS\n{len(u.path)}-node route\ncost {u.path_cost:.2f}", "#ddeaf9", bold=True)
    _box(ax, 7.6, 5.6, 2.8, 1.2, f"STAGE 2  SA\nJ* = {s.objective:.3f}\nrho = {s.power_fraction:.2f}", "#fde7dc", bold=True)
    _box(ax, 11.3, 5.6, 2.6, 1.2, f"STAGE 3  CSP\nmakespan {c.makespan} slots\n{c.nodes} search nodes", "#dcf3ea", bold=True)
    _box(ax, 4.2, 3.4, 2.9, 1.7,
         f"UCS -> SA\nB = {u.power_budget_kW:.1f} kW\ntheta_route = {u.total_thermal_C:.1f} C\n"
         f"eta = {u.route_efficiency:.3f}\nlosses = {u.total_loss_kW:.0f} kW", "#f7f7f5")
    _box(ax, 7.6, 3.4, 3.4, 1.7,
         "SA -> CSP\n" + ", ".join(f"{k}={v}" for k, v in s.allocation.items()) +
         f"\nP_res = {s.residual_power_kW:.1f} kW\ntheta_total = {s.total_thermal_C:.2f} C\nM = {s.material_used_NU} NU",
         "#f7f7f5", fs=7.8)
    durs = "\n".join(f"{t.id}: d={t.duration}" for t in r.tasks)
    _box(ax, 11.3, 3.3, 2.7, 2.2, f"Derived CSP inputs\nP_res cap {r.power_cap_kW:.1f} kW\n" + durs,
         "#f7f7f5", fs=6.6)
    passed = sum(ch["passed"] for ch in out.checks)
    _box(ax, 6.5, 1.0, 8.4, 1.0, f"Validation: {passed}/{len(out.checks)} checks passed  ->  {out.status}",
         "#e3f4e6" if out.success else "#fdecea", bold=True)
    _arrow(ax, (2.45, 5.6), (2.9, 5.6)); _arrow(ax, (5.5, 5.6), (6.2, 5.6)); _arrow(ax, (9.0, 5.6), (10.0, 5.6))
    _arrow(ax, (4.2, 5.0), (4.2, 4.25)); _arrow(ax, (5.65, 4.0), (6.6, 5.0), "B, theta, eta", off=(-0.35, 0.05))
    _arrow(ax, (7.6, 5.0), (7.6, 4.25)); _arrow(ax, (9.3, 3.4), (9.95, 3.4)); _arrow(ax, (11.3, 4.25), (11.3, 5.0))
    _arrow(ax, (10.0, 6.25), (9.0, 6.25), "CSP infeasible -> lower rho", color=C[7], ls="--", rad=-0.0, off=(0, 0.2))
    _arrow(ax, (6.3, 6.35), (5.5, 6.35), "SA infeasible -> re-route", color=C[7], ls="--", off=(0, 0.17))
    _arrow(ax, (11.3, 2.55), (8.8, 1.5)); _arrow(ax, (4.2, 2.55), (5.0, 1.5)); _arrow(ax, (7.6, 2.55), (7.0, 1.5))
    return _finish(fig, path, "Figure 3. End-to-end workflow and data exchanged between UCS, SA and CSP")
def dashboard(ds, out, path: Path):
    u, s, r, c = out.ucs, out.sa, out.req, out.csp
    fig = plt.figure(figsize=(15, 9.6))
    gs = fig.add_gridspec(3, 3, height_ratios=[0.5, 1.6, 1.7], hspace=0.55, wspace=0.32)
    banner = fig.add_subplot(gs[0, :]); banner.axis("off")
    banner.add_patch(FancyBboxPatch((0, 0.05), 1, 0.9, boxstyle="round,pad=0.01", transform=banner.transAxes,
                                    fc="#e3f4e6" if out.success else "#fdecea", ec=GOOD if out.success else BAD, lw=2))
    passed = sum(ch["passed"] for ch in out.checks)
    banner.text(0.5, 0.62, f"JARVIS RECOVERY STATUS: {out.status}", ha="center", va="center", fontsize=15,
```

```
                fontweight="bold", transform=banner.transAxes, color=INK)
    banner.text(0.5, 0.25, f"Mission {ds.mission['id']}  |  route {' > '.join(u.path)}  |  power cost {u.path_cost:.2f}  |  "
                f"{passed}/{len(out.checks)} constraint checks PASS  |  replanning attempts {len(out.attempts)}  |  "
                f"completion {c.makespan * ds.mission['slot_minutes'] / 60:.1f} h",
                ha="center", va="center", fontsize=9.5, transform=banner.transAxes, color=INK2)

    a = fig.add_subplot(gs[1, 0])
    P = ds.reactor["available_output_kW"]
    labels = ["Reactor available", "Route losses", "Bottleneck curtailment", "Delivered to CLH", "Critical loads",
              "Nanomaterial (SA)", "Peak lab ops (CSP)", "Unused margin"]
    peak = max(sum(v["power_kW"] for v in c.schedule.values() if v["start"] <= t < v["end"]) for t in range(r.horizon))
    vals = [P, u.total_loss_kW, P - u.total_loss_kW - u.delivered_power_kW, u.delivered_power_kW,
            u.critical_load_kW, s.nano_power_kW, peak,
            u.delivered_power_kW - u.critical_load_kW - s.nano_power_kW - peak]
    cols = [C[0], C[7], C[3], C[0], C[2], C[1], C[6], "#bdbcb7"]
    a.barh(range(len(vals)), vals, color=cols, edgecolor="white", linewidth=2)
    for i, v in enumerate(vals):
        a.text(v + 8, i, f"{v:.1f} kW", va="center", fontsize=7.5)
    a.set_yticks(range(len(vals))); a.set_yticklabels(labels, fontsize=8); a.invert_yaxis()
    a.set_xlim(0, P * 1.3); a.set_title("Power budget chain (kW)", fontsize=10); a.grid(axis="x")

    a = fig.add_subplot(gs[1, 1])
    cats = ds.nano.categories
    a.bar([cc.id for cc in cats], [s.allocation[cc.id] for cc in cats], color=C[0], width=0.6)
    for i, cc in enumerate(cats):
        a.hlines(cc.min_NU, i - 0.35, i + 0.35, colors=INK, linestyles="--", lw=1)
        a.text(i, s.allocation[cc.id] + 8, str(s.allocation[cc.id]), ha="center", fontsize=8)
    a.set_title(f"Nanomaterial allocation ({s.material_used_NU}/{ds.nano.reserve_NU} NU, dashed = minimum)", fontsize=10)
    a.grid(axis="y")

    a = fig.add_subplot(gs[1, 2])
    subs = ds.subsystems
    xs = range(len(subs))
    a.bar([i - 0.2 for i in xs], [q["health_pct"] for q in subs], width=0.38, color=C[1], label="before (FRIDAY)")
    a.bar([i + 0.2 for i in xs], [out.health_after[q["id"]] for q in subs], width=0.38, color=C[0],
          label="after (model estimate)")
    a.set_xticks(list(xs)); a.set_xticklabels([q["id"] for q in subs]); a.set_ylim(0, 115)
    a.set_title("Subsystem health (%)", fontsize=10); a.legend(frameon=False, fontsize=7.5, ncol=2, loc="upper left")
    a.grid(axis="y")

    a = fig.add_subplot(gs[2, :2])
    sch = c.schedule
    order = sorted(sch, key=lambda k: (sch[k]["start"], sch[k]["end"]))
    for i, k in enumerate(order):
        a.barh(i, sch[k]["duration"], left=sch[k]["start"], color=C[0], edgecolor="white", linewidth=2, height=0.7)
        a.text(sch[k]["start"] + 0.08, i, k, va="center", fontsize=7, color="white", fontweight="bold")
    a.set_yticks([]); a.invert_yaxis(); a.set_xlim(0, r.horizon); a.set_xticks(range(r.horizon + 1))
    a.axvline(r.horizon, color=BAD, ls="--"); a.grid(axis="x")
    a.set_title(f"Repair schedule (makespan {c.makespan} of {r.horizon} slots)", fontsize=10)

    a = fig.add_subplot(gs[2, 2]); a.axis("off")
    stages = ["UCS", "SA", "CSP", "INTEGRATED"]
    y = 0.98
    a.text(0, y, "Constraint validation", fontsize=10, fontweight="bold", transform=a.transAxes)
    for st in stages:
        cs = [ch for ch in out.checks if ch["stage"] == st]
        ok = sum(ch["passed"] for ch in cs)
        y -= 0.1
        a.text(0, y, f"{st:<11} {ok}/{len(cs)} PASS" if ok == len(cs) else f"{st:<11} {ok}/{len(cs)} - FAIL",
               fontsize=9.5, family="monospace", color=GOOD if ok == len(cs) else BAD, transform=a.transAxes,
               fontweight="bold")
    y -= 0.12
    for line in [f"Route cost      {u.path_cost:.2f}", f"Delivered       {u.delivered_power_kW:.1f} kW",
                 f"Budget B        {u.power_budget_kW:.1f} kW", f"Nano power      {s.nano_power_kW:.1f} kW",
                 f"Residual P_res  {s.residual_power_kW:.1f} kW", f"Thermal total   {s.total_thermal_C:.2f} C",
                 f"Objective J*    {s.objective:.3f}"]:
        a.text(0, y, line, fontsize=8.5, family="monospace", transform=a.transAxes); y -= 0.075
    return _finish(fig, path, "Figure 9. JARVIS recovery dashboard - Mark 85")
def performance(perf: dict, path: Path):
    fig, (a1, a2, a3) = plt.subplots(1, 3, figsize=(15, 4.6))
    sa, hc = perf["sa_vs_hc"]["sa"], perf["sa_vs_hc"]["hc"]
    bp = a1.boxplot([sa, hc], tick_labels=["Simulated\nAnnealing", "Hill climbing\n(T = 0)"], widths=0.5,
                    patch_artist=True, medianprops=dict(color=INK, lw=1.6))
    for patch, col in zip(bp["boxes"], [C[0], C[1]]):
        patch.set_facecolor(col); patch.set_alpha(0.85)
    a1.set_ylabel("Final objective J"); a1.grid(axis="y")
    a1.set_title(f"SA vs hill climbing, {len(sa)} seeds", fontsize=10)
    rows = perf["csp_variants"]
    names = [r["variant"] for r in rows]
    a2.barh(names, [r["nodes"] for r in rows], color=C[2])
    a2.set_xscale("log"); a2.set_xlabel("Search nodes (log scale)"); a2.invert_yaxis()
    for i, r in enumerate(rows):
        a2.text(r["nodes"] * 1.15, i, f"{r['nodes']} nodes, {r['backtracks']} bt", va="center", fontsize=7.5)
    a2.set_title("CSP search effort by heuristic set", fontsize=10); a2.grid(axis="x")
    a2.set_xlim(1, max(r["nodes"] for r in rows) * 40)
    rows = perf["route_methods"]
    a3.bar([r["method"] for r in rows], [r["cost"] for r in rows], color=[C[0] if r["feasible"] else "#bdbcb7" for r in rows])
    for i, r in enumerate(rows):
        a3.text(i, r["cost"] + 0.8, f"{r['cost']:.2f}\n{'feasible' if r['feasible'] else 'VIOLATES'}", ha="center",
                fontsize=7.5)
    a3.set_ylabel("Route cost"); a3.set_title("Route found by each search strategy", fontsize=10)
    a3.tick_params(axis="x", labelsize=7.5); a3.grid(axis="y")
    a3.set_ylim(0, max(r["cost"] for r in rows) * 1.25)
    return _finish(fig, path, "Figure 10. Performance analysis of the three stages")
```

A.11 jarvis/experiments.py

```
from __future__ import annotations

import statistics
import time
from collections import deque

from .annealing import NanoAllocationProblem, simulated_annealing
from .csp import solve_recovery_csp
from .ucs import uniform_cost_search
def _bfs_route(net):
    q = deque([[net.source]])
    seen = {net.source}
    while q:
        p = q.popleft()
        if p[-1] == net.goal:
            return p
        for e in net.out_edges(p[-1]):
            if e.v in seen or net.nodes[e.v].failed or e.capacity_kW < net.required_throughput_kW:
                continue
            seen.add(e.v)
            q.append(p + [e.v])
    return None
def _route_stats(net, path):
    edges = [net.edge(a, b) for a, b in zip(path, path[1:])]
    th = sum(e.thermal_C for e in edges)
    feasible = (not any(net.nodes[n].failed for n in path)
                and min(e.capacity_kW for e in edges) >= net.required_throughput_kW
                and th <= net.thermal_limit_C + 1e-9)
    return {"path": path, "cost": round(sum(e.cost(net.cost_weights) for e in edges), 2),
            "thermal": th, "feasible": feasible}
def run_experiments(ds, outcome, n_seeds: int = 30) -> dict:
    u, s = outcome.ucs, outcome.sa
    net = ds.network
    P, crit = ds.reactor["available_output_kW"], ds.critical_load_kW

    prob = NanoAllocationProblem(ds.nano, u.power_budget_kW, s.power_fraction, u.total_thermal_C)
    sa_vals, hc_vals, sa_iters, sa_times = [], [], [], []
    for seed in range(n_seeds):
        t0 = time.perf_counter()
        r = simulated_annealing(prob, ds.annealing, seed)
        sa_times.append(time.perf_counter() - t0)
        sa_vals.append(round(r.objective, 4)); sa_iters.append(r.iterations)
        hc_vals.append(round(simulated_annealing(prob, ds.annealing, seed, greedy=True).objective, 4))
    sa_summary = {
        "sa": sa_vals, "hc": hc_vals,
        "sa_mean": statistics.mean(sa_vals), "sa_std": statistics.pstdev(sa_vals),
        "sa_best": max(sa_vals), "sa_worst": min(sa_vals),
        "hc_mean": statistics.mean(hc_vals), "hc_std": statistics.pstdev(hc_vals),
        "hc_best": max(hc_vals), "hc_worst": min(hc_vals),
        "sa_wins": sum(a > b + 1e-9 for a, b in zip(sa_vals, hc_vals)),
        "sa_mean_iters": statistics.mean(sa_iters), "sa_mean_time_s": statistics.mean(sa_times),
    }

    variants = [("MRV + degree + FC", True, True, True), ("MRV + FC", True, False, True),
                ("FC only (static order)", False, False, True), ("MRV + degree, no FC", True, True, False),
                ("Plain backtracking", False, False, False)]
    csp_rows = []
    for name, mrv, deg, fc in variants:
        t0 = time.perf_counter()
        res, _, _ = solve_recovery_csp(outcome.req, ds.lab, use_mrv=mrv, use_degree=deg, use_fc=fc,
                                       node_limit=2_000_000, trace_limit=0)
        csp_rows.append({"variant": name, "solved": res.solved, "makespan": res.makespan,
                         "nodes": res.nodes, "backtracks": res.backtracks,
                         "checks": res.constraint_checks, "wipeouts": res.wipeouts,
                         "time_ms": round((time.perf_counter() - t0) * 1000, 2)})

    routes = []
    routes.append({"method": "UCS\n(all constraints)", **_route_stats(net, u.path)})
    r = uniform_cost_search(net, P, crit, enforce_thermal=False, record_trace=False)
    routes.append({"method": "UCS\nthermal-blind", **_route_stats(net, r.path)})
    r = uniform_cost_search(net, P, crit, enforce_thermal=False, enforce_capacity=False, record_trace=False)
    routes.append({"method": "UCS thermal- &\ncapacity-blind", **_route_stats(net, r.path)})
    bfs = _bfs_route(net)
    routes.append({"method": "BFS\n(fewest hops)", **_route_stats(net, bfs)})

    reps = 50
    t0 = time.perf_counter()
    for _ in range(reps):
        uniform_cost_search(net, P, crit, record_trace=False)
    ucs_ms = (time.perf_counter() - t0) / reps * 1000
    return {"sa_vs_hc": sa_summary, "csp_variants": csp_rows, "route_methods": routes,
            "timing_ms": {"ucs": round(ucs_ms, 3), "sa": round(sa_summary["sa_mean_time_s"] * 1000, 1),
                          "csp": csp_rows[0]["time_ms"]}}
```

A.12 jarvis/reporting.py

```
from __future__ import annotations

import csv
import json
```

```
from pathlib import Path

from .data_models import to_jsonable
def rule(title="", ch="="):
    return f"\n{ch * 78}\n{title}\n{ch * 78}" if title else ch * 78
def mission_header(ds) -> str:
    lines = [rule("JARVIS :: MARK 85 INTELLIGENT RECOVERY  (fictional engineering simulation)"),
             f"Mission          : {ds.mission['id']} at {ds.mission['location']}",
             f"Trigger          : {ds.mission['trigger']}",
             f"Arc Reactor      : {ds.reactor['available_output_kW']} kW available of "
             f"{ds.reactor['nominal_output_kW']} kW nominal ({ds.reactor['status']})",
             f"Critical load    : {ds.critical_load_kW:.0f} kW  (FLC+STB+SEN+LSC)",
             f"Nanomaterial     : {ds.nano.reserve_NU} NU reserve, armor thermal limit "
             f"{ds.nano.armor_thermal_limit_C} C",
             f"Laboratory       : {len(ds.lab.tasks)} tasks, {ds.lab.technicians} technicians, horizon "
             f"{ds.mission['horizon_slots']} x {ds.mission['slot_minutes']} min", "",
             "Subsystem status reported by FRIDAY:"]
    for s in ds.subsystems:
        lines.append(f"  {s['id']:<4} {s['name']:<26} health {s['health_pct']:>3}%  load "
                     f"{s['critical_load_kW']:>4} kW  {s['status']}")
    return "\n".join(lines)
def ucs_report(ds, u) -> str:
    net = ds.network
    L = [rule("STAGE 1 - UNIFORM COST SEARCH: Arc Reactor power routing"),
         f"Edge cost c(e) = {net.cost_weights['loss']}*loss + {net.cost_weights['thermal']}*thermal + "
         f"{net.cost_weights['switching']}*switching ; P_req = {net.required_throughput_kW:g} kW ; "
         f"Theta_max = {net.thermal_limit_C:g} C", "",
         f"{'edge':<14}{'loss kW':>8}{'therm C':>9}{'sw ms':>7}{'cap kW':>8}{'cost':>8}"]
    for e in net.edges:
        L.append(f"{e.u + '->' + e.v:<14}{e.loss_kW:>8g}{e.thermal_C:>9g}{e.switch_ms:>7g}"
                 f"{e.capacity_kW:>8g}{e.cost(net.cost_weights):>8.2f}")
    L.append("\nExpansion trace (frontier shown as node:g/theta, ordered by g):")
    for s in u.trace:
        L.append(f" step {s['step']:>2}: expand {s['expand']:<6} g={s['g']:>6.2f} theta={s['theta']:>5.1f} "
                 f"path={'>'.join(s['path'])}{'   <-- GOAL' if s.get('goal') else ''}")
        for p in s["pushed"]:
            L.append(f"          push {p['to']:<6} g={p['g']:>6.2f} theta={p['theta']:>5.1f} (edge {p['edge_cost']})")
        for r in s["rejected"]:
            L.append(f"          reject {r['to']:<6} {r['reason']}")
        L.append("          frontier: " + ", ".join(f"{f['node']}:{f['g']}/{f['theta']}" for f in s["frontier_after"]))
    L += ["", f"OPTIMAL ROUTE   : {' -> '.join(u.path)}",
          f"path cost g*    : {u.path_cost:.2f}",
          f"losses / thermal / switching : {u.total_loss_kW:g} kW / {u.total_thermal_C:g} C / {u.total_switch_ms:g} ms",
          f"bottleneck cap  : {u.bottleneck_capacity_kW:g} kW",
          f"delivered power : min({ds.reactor['available_output_kW']} - {u.total_loss_kW:g}, "
          f"{u.bottleneck_capacity_kW:g}) = {u.delivered_power_kW:g} kW",
          f"power budget B  : {u.delivered_power_kW:g} - {u.critical_load_kW:g} = {u.power_budget_kW:g} kW  -> Stage 2",
          f"route efficiency eta = {u.route_efficiency:.3f}  -> Stage 3 charging rate",
          f"expanded {u.expansions} labels, generated {u.generated}"]
    return "\n".join(L)
def sa_report(ds, s, show=25) -> str:
    L = [rule("STAGE 2 - SIMULATED ANNEALING: nanomaterial allocation"),
         f"power cap rho*B = {s.power_fraction:.2f} x {s.power_cap_kW / s.power_fraction:.1f} = {s.power_cap_kW:.1f} kW",
         f"initial (mandatory minimums) {s.initial_allocation}  J0 = {s.initial_objective:.3f}", "",
         f"{'iter':>5} {'T':>8} {'move':<24}{'cand J':>9}{'dE':>9}{'P(acc)':>8}  decision"]
    rows = s.history[1:show + 1] + [None] + [r for r in s.history if r["iter"] % 400 == 0 and r["iter"] > show]
    for r in rows:
        if r is None:
            L.append("  ...  (every 400th iteration below)")
            continue
        cj = f"{r['cand_J']:.3f}" if r["cand_J"] is not None else "-"
        de = f"{r['dE']:+.3f}" if r["dE"] is not None else "-"
        L.append(f"{r['iter']:>5} {r['T']:>8.4f} {r['move']:<24}{cj:>9}{de:>9}{r['p_accept']:>8.3f}  {r['decision']}")
    L += ["", f"termination     : {s.termination}",
          f"iterations {s.iterations}, accepted-worse {s.accepted_worse}, rejected-infeasible neighbours "
          f"{s.rejected_infeasible}, best improvements {s.improvements}",
          f"BEST ALLOCATION : {s.allocation}",
          f"objective J*    : {s.objective:.4f}  (from {s.initial_objective:.4f})",
          f"material used   : {s.material_used_NU} / {ds.nano.reserve_NU} NU",
          f"nano power      : {s.nano_power_kW:.1f} kW (cap {s.power_cap_kW:.1f})",
          f"residual P_res  : {s.residual_power_kW:.1f} kW  -> Stage 3 power constraint",
          f"total thermal   : {s.total_thermal_C:.2f} C (limit {ds.nano.armor_thermal_limit_C})"]
    return "\n".join(L)
def csp_report(out, trace_lines=60) -> str:
    r, c = out.req, out.csp
    L = [rule("STAGE 3 - CSP: Stark Laboratory recovery scheduling"), "Derived inputs:"]
    L += ["  " + n for n in r.notes]
    L.append(f"\n{'task':<14}{'dur':>4}{'kW':>6}{'tech':>5}  needs / after / derivation")
    for t in r.tasks:
        L.append(f"{t.id:<14}{t.duration:>4}{t.power_kW:>6g}{t.technicians:>5}  "
                 f"{t.needs} after={t.after or '-'} dl={t.deadline}  [{t.derivation}]")
    L.append("\nInitial domain sizes (after unary + bound propagation):")
    L.append("  " + ", ".join(f"{k}:{len(v)}" for k, v in out.csp_model.domains.items()))
    for b in out.bounds_log:
        L.append("  " + b)
    L.append("\nSearch trace (first solve):")
    L += ["  " + x for x in out.csp_first.trace[:trace_lines]]
    L.append(f"\nfirst solve: nodes={out.csp_first.nodes} backtracks={out.csp_first.backtracks} "
             f"checks={out.csp_first.constraint_checks} FC-prunes={out.csp_first.fc_prunes} "
             f"wipeouts={out.csp_first.wipeouts} makespan={out.csp_first.makespan}")
    L.append("Makespan tightening:")
    for m in out.makespan_search:
        L.append(f"  H={m['horizon']:>2}: {'solved' if m['solved'] else 'infeasible'} ({m['reason']})")
    L.append(f"\nFINAL SCHEDULE (makespan {c.makespan} slots):")
```

```
    L.append(f"{'task':<14}{'start':>6}{'end':>5}  resources")
    for k, v in sorted(c.schedule.items(), key=lambda kv: (kv[1]["start"], kv[0])):
        L.append(f"{k:<14}{v['start']:>6}{v['end']:>5}  {v['resources']}")
    return "\n".join(L)
def attempts_report(out) -> str:
    L = [rule("CONTROLLED REPLANNING LOG")]
    for a in out.attempts:
        L.append(f"attempt {a.number}: rho={a.power_fraction:.2f} excluded={a.excluded_edges or '-'}")
        if a.ucs and a.ucs.found:
            L.append(f"   UCS route {' > '.join(a.ucs.path)}  B={a.ucs.power_budget_kW:.1f} kW")
        if a.sa and a.sa.feasible:
            L.append(f"   SA  {a.sa.allocation} J={a.sa.objective:.3f} P_res={a.sa.residual_power_kW:.1f} kW")
        L.append(f"   outcome: {a.outcome}")
        L.append(f"   action : {a.action}")
    return "\n".join(L)
def validation_report(out) -> str:
    L = [rule("INTEGRATED VALIDATION")]
    for ch in out.checks:
        L.append(f"[{'PASS' if ch['passed'] else 'FAIL'}] {ch['stage']:<10} {ch['check']:<55} {ch['detail']}")
    passed = sum(c["passed"] for c in out.checks)
    L += ["", f"{passed}/{len(out.checks)} checks passed", "Estimated post-recovery health (model): " +
          ", ".join(f"{k} {v}%" for k, v in out.health_after.items()),
          rule(f"FINAL ARMOR STATUS: {out.status}")]
    return "\n".join(L)
def export(out, ds, perf, outdir: Path):
    outdir.mkdir(parents=True, exist_ok=True)
    s = out.sa
    with open(outdir / "sa_iterations.csv", "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["iter", "T", "move", "cand_J", "dE", "p_accept", "decision", "cur_J", "best_J"] +
                   [c.id for c in ds.nano.categories])
        for r in s.history:
            w.writerow([r["iter"], f"{r['T']:.6f}", r["move"], r["cand_J"], r["dE"], f"{r['p_accept']:.5f}",
                        r["decision"], f"{r['cur_J']:.5f}", f"{r['best_J']:.5f}"] +
                       [r["alloc"][c.id] for c in ds.nano.categories])
    with open(outdir / "schedule.csv", "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["task", "name", "start", "end", "duration", "power_kW", "technicians", "resources"])
        for k, v in sorted(out.csp.schedule.items(), key=lambda kv: kv[1]["start"]):
            w.writerow([k, v["name"], v["start"], v["end"], v["duration"], v["power_kW"], v["technicians"],
                        "; ".join(f"{a}={b}" for a, b in v["resources"].items())])
    with open(outdir / "validation.csv", "w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=["stage", "check", "passed", "detail"])
        w.writeheader(); w.writerows(out.checks)
    (outdir / "csp_trace.txt").write_text("\n".join(out.csp_first.trace), encoding="utf-8")
    summary = {
        "status": out.status, "success": out.success,
        "attempts": [{"number": a.number, "rho": a.power_fraction, "excluded": a.excluded_edges,
                      "outcome": a.outcome, "action": a.action,
                      "route": a.ucs.path if a.ucs else None,
                      "allocation": a.sa.allocation if a.sa else None,
                      "objective": a.sa.objective if a.sa else None,
                      "residual_power_kW": a.sa.residual_power_kW if a.sa else None,
                      "durations": {t.id: t.duration for t in a.req.tasks} if a.req else None}
                     for a in out.attempts],
        "ucs": {k: v for k, v in to_jsonable(out.ucs).items() if k not in ("trace",)},
        "ucs_trace": out.ucs.trace,
        "sa": {k: v for k, v in to_jsonable(out.sa).items() if k != "history"},
        "requirements": to_jsonable(out.req),
        "csp_first": {k: v for k, v in to_jsonable(out.csp_first).items() if k != "trace"},
        "csp_final": {k: v for k, v in to_jsonable(out.csp).items() if k != "trace"},
        "csp_domains": {k: len(v) for k, v in out.csp_model.domains.items()},
        "csp_bounds_log": out.bounds_log,
        "csp_constraint_edges": out.csp_model.constraint_edges(),
        "makespan_search": out.makespan_search,
        "checks": out.checks, "health_after": out.health_after,
        "performance": perf,
    }
    (outdir / "results.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")
```

A.13 jarvis/api.py

```
from __future__ import annotations

import math
import time
from dataclasses import replace
from pathlib import Path
from typing import Any, Dict, List, Optional

from .annealing import NanoAllocationProblem, simulated_annealing
from .csp import solve_recovery_csp
from .data_models import MissionDataset, load_dataset, to_jsonable
from .experiments import run_experiments
from .pipeline import estimate_health, run_mission
from .requirements import derive_requirements
from .ucs import enumerate_simple_paths, uniform_cost_search
from .validation import validate_csp, validate_integrated, validate_sa, validate_ucs
from .visualization import NET_POS

DEFAULT_DATASET = Path(__file__).resolve().parent.parent / "data" / "mission_dataset.json"
MODES = ("integrated", "ucs", "sa", "csp")
```

```
class ApiError(ValueError):
    pass
def clean(obj):
    if isinstance(obj, float):
        return obj if math.isfinite(obj) else None
    if isinstance(obj, dict):
        return {str(k): clean(v) for k, v in obj.items()}
    if isinstance(obj, (list, tuple, set)):
        return [clean(v) for v in obj]
    return obj
def _num(section: dict, key: str, lo: float, hi: float, cast=float):
    if key not in section or section[key] in (None, ""):
        return None
    try:
        v = cast(section[key])
    except (TypeError, ValueError):
        raise ApiError(f"{key} must be a number")
    if not (lo <= v <= hi):
        raise ApiError(f"{key} must be between {lo:g} and {hi:g}")
    return v
def build_dataset(params: dict, path: Path = DEFAULT_DATASET) -> MissionDataset:
    ds = load_dataset(path)
    m = params.get("mission", {}) or {}
    u = params.get("ucs", {}) or {}
    s = params.get("sa", {}) or {}
    c = params.get("csp", {}) or {}

    if "failed_nodes" in m:
        failed = set(m["failed_nodes"] or [])
        for nid in failed:
            if nid not in ds.network.nodes:
                raise ApiError(f"unknown component {nid}")
            if nid in (ds.network.source, ds.network.goal):
                raise ApiError(f"{nid} cannot be failed (source / goal)")
        for nid, node in ds.network.nodes.items():
            if node.type in ("source", "load"):
                continue
            ds.network.nodes[nid] = replace(node, status="FAILED" if nid in failed
                                            else ("OK" if node.failed else node.status))
    v = _num(m, "available_kW", 100, 1500)
    if v is not None:
        ds.reactor["available_output_kW"] = v
    v = _num(m, "seed", 0, 10**9, int)
    if v is not None:
        ds.mission["seed"] = v

    for key, name in (("w_loss", "loss"), ("w_thermal", "thermal"), ("w_switching", "switching")):
        v = _num(u, key, 0, 10)
        if v is not None:
            ds.network.cost_weights = {**ds.network.cost_weights, name: v}
    v = _num(u, "thermal_limit", 1, 50)
    if v is not None:
        ds.network.thermal_limit_C = v
    v = _num(u, "required_kW", 0, 1500)
    if v is not None:
        ds.network.required_throughput_kW = v

    v = _num(s, "rho", 0.05, 1.0)
    if v is not None:
        ds.nano.initial_power_fraction = v
    for key, attr, lo, hi, cast in (("T0", "initial_temperature", 0.01, 100, float),
                                    ("alpha", "cooling_rate", 0.5, 0.999, float),
                                    ("max_iter", "max_iterations", 50, 50_000, int)):
        v = _num(s, key, lo, hi, cast)
        if v is not None:
            setattr(ds.annealing, attr, v)

    v = _num(c, "horizon", 4, 40, int)
    if v is not None:
        ds.mission["horizon_slots"] = v
    return ds
def dataset_info(path: Path = DEFAULT_DATASET) -> dict:
    ds = load_dataset(path)
    return clean({
        "mission": ds.mission, "reactor": ds.reactor, "subsystems": ds.subsystems,
        "network": _network_json(ds),
        "nano": {"reserve_NU": ds.nano.reserve_NU, "thermal_limit_C": ds.nano.armor_thermal_limit_C,
                 "rho": ds.nano.initial_power_fraction,
                 "categories": [to_jsonable(cat) for cat in ds.nano.categories]},
        "annealing": to_jsonable(ds.annealing),
        "rules": ds.rules,
        "lab": {"technicians": ds.lab.technicians,
                "tasks": [{"id": t.id, "name": t.name, "after": t.after} for t in ds.lab.tasks]},
        "defaults": {
            "mission": {"seed": ds.mission["seed"], "available_kW": ds.reactor["available_output_kW"],
                        "failed_nodes": [n.id for n in ds.network.nodes.values() if n.failed]},
            "ucs": {"w_loss": ds.network.cost_weights["loss"], "w_thermal": ds.network.cost_weights["thermal"],
                    "w_switching": ds.network.cost_weights["switching"],
                    "thermal_limit": ds.network.thermal_limit_C,
                    "required_kW": ds.network.required_throughput_kW},
            "sa": {"rho": ds.nano.initial_power_fraction, "T0": ds.annealing.initial_temperature,
                   "alpha": ds.annealing.cooling_rate, "max_iter": ds.annealing.max_iterations,
                   "budget_kW": None, "compare_hc": True},
            "csp": {"horizon": ds.mission["horizon_slots"], "mrv": True, "degree": True, "fc": True,
```

```
                    "minimize": True, "power_cap_kW": None},
        },
    })
def _network_json(ds: MissionDataset) -> dict:
    net = ds.network
    return {
        "source": net.source, "goal": net.goal,
        "required_kW": net.required_throughput_kW, "thermal_limit_C": net.thermal_limit_C,
        "weights": net.cost_weights,
        "nodes": [{"id": n.id, "label": n.label, "type": n.type, "status": n.status,
                   "x": NET_POS[n.id][0], "y": NET_POS[n.id][1]} for n in net.nodes.values()],
        "edges": [{"u": e.u, "v": e.v, "loss": e.loss_kW, "thermal": e.thermal_C, "switch": e.switch_ms,
                   "capacity": e.capacity_kW, "cost": round(e.cost(net.cost_weights), 3)} for e in net.edges],
        "load_edges": net.load_edges,
    }
def _ucs_json(ds: MissionDataset, u) -> dict:
    routes = enumerate_simple_paths(ds.network)
    return {
        "found": u.found, "path": u.path, "cost": u.path_cost, "loss_kW": u.total_loss_kW,
        "thermal_C": u.total_thermal_C, "switch_ms": u.total_switch_ms,
        "bottleneck_kW": u.bottleneck_capacity_kW, "delivered_kW": u.delivered_power_kW,
        "critical_kW": u.critical_load_kW, "budget_kW": u.power_budget_kW,
        "efficiency": u.route_efficiency, "expansions": u.expansions, "generated": u.generated,
        "pruned": len(u.pruned), "excluded_edges": u.excluded_edges, "trace": u.trace,
        "routes": routes[:40], "routes_total": len(routes),
        "routes_feasible": sum(r["feasible"] for r in routes),
        "available_kW": ds.reactor["available_output_kW"],
    }
def _downsample(history: List[dict], n: int = 700) -> List[dict]:
    if len(history) <= n:
        return history
    step = len(history) / n
    idx = sorted({int(i * step) for i in range(n)} | {len(history) - 1})
    return [history[i] for i in idx]
def _sa_json(ds: MissionDataset, s, problem: NanoAllocationProblem, hc=None) -> dict:
    cats = ds.nano.categories
    hist = [{"i": r["iter"], "T": r["T"], "cur": r["cur_J"], "best": r["best_J"], "cand": r["cand_J"],
             "d": r["decision"]} for r in s.history]
    out = {
        "feasible": s.feasible, "allocation": s.allocation, "initial": s.initial_allocation,
        "objective": s.objective, "initial_objective": s.initial_objective,
        "nano_power_kW": s.nano_power_kW, "nano_heat_C": s.nano_heat_C,
        "material_NU": s.material_used_NU, "reserve_NU": ds.nano.reserve_NU,
        "power_cap_kW": s.power_cap_kW, "rho": s.power_fraction, "budget_kW": problem.B,
        "residual_kW": s.residual_power_kW, "thermal_C": s.total_thermal_C,
        "thermal_limit_C": ds.nano.armor_thermal_limit_C, "route_thermal_C": problem.route_thermal,
        "iterations": s.iterations, "accepted_worse": s.accepted_worse,
        "rejected_infeasible": s.rejected_infeasible, "improvements": s.improvements,
        "termination": s.termination,
        "categories": [{"id": c.id, "name": c.name, "min": c.min_NU, "max": c.max_NU,
                        "power": c.power_kW_per_NU, "packet": c.packet_NU} for c in cats],
        "benefit": problem.benefit_terms(s.allocation) if s.feasible else {},
        "history": _downsample(hist),
        "first_moves": [{"i": r["iter"], "T": r["T"], "move": r["move"], "cand": r["cand_J"], "dE": r["dE"],
                         "p": r["p_accept"], "d": r["decision"]} for r in s.history[1:41]],
    }
    if hc is not None:
        out["hill_climb"] = {"objective": hc.objective, "allocation": hc.allocation,
                             "iterations": hc.iterations}
    return out
def _csp_json(ds: MissionDataset, req, first, best, model, bounds_log, ms_log) -> dict:
    H = req.horizon
    sched = best.schedule if best.solved else {}
    power = [sum(v["power_kW"] for v in sched.values() if v["start"] <= t < v["end"]) for t in range(H)]
    techs = [sum(v["technicians"] for v in sched.values() if v["start"] <= t < v["end"]) for t in range(H)]
    return {
        "solved": best.solved, "makespan": best.makespan, "first_makespan": first.makespan,
        "schedule": sched, "failure_reason": first.failure_reason,
        "nodes": first.nodes, "backtracks": first.backtracks, "checks": first.constraint_checks,
        "prunes": first.fc_prunes, "wipeouts": first.wipeouts, "elapsed_ms": round(first.elapsed_s * 1000, 2),
        "power_cap_kW": req.power_cap_kW, "horizon": H, "technicians": req.technicians,
        "slot_minutes": ds.mission["slot_minutes"],
        "tasks": [to_jsonable(t) for t in req.tasks], "notes": req.notes,
        "domains": {k: len(v) for k, v in model.domains.items()} if model else {},
        "bounds_log": bounds_log, "makespan_search": ms_log,
        "constraint_edges": model.constraint_edges() if model else [],
        "power_profile": power, "tech_profile": techs,
        "trace": first.trace[:400], "trace_total": len(first.trace),
    }
def _summary(checks: List[dict]) -> dict:
    return {"passed": sum(c["passed"] for c in checks), "total": len(checks),
            "all_passed": all(c["passed"] for c in checks) if checks else False}
def _stage_ucs(ds):
    return uniform_cost_search(ds.network, ds.reactor["available_output_kW"], ds.critical_load_kW)
def _stage_sa(ds, u, sa_params: dict):
    budget = _num(sa_params, "budget_kW", 1, 5000)
    if budget is not None:
        u = replace(u, power_budget_kW=budget)
    elif not u.found:
        raise ApiError("UCS found no admissible power route, so there is no power budget B. "
                       "Repair a component or enter a manual budget for SA.")
    elif u.power_budget_kW <= 0:
        raise ApiError(f"The UCS route delivers {u.delivered_power_kW:.0f} kW, which does not even cover the "
```

```
                      f"{u.critical_load_kW:.0f} kW critical load (B = {u.power_budget_kW:.0f} kW). "
                      "Raise the reactor output or enter a manual budget for SA.")
    problem = NanoAllocationProblem(ds.nano, u.power_budget_kW, ds.nano.initial_power_fraction,
                                    u.total_thermal_C)
    s = simulated_annealing(problem, ds.annealing, ds.mission["seed"])
    return u, problem, s
def mode_ucs(ds, p) -> dict:
    u = _stage_ucs(ds)
    checks = validate_ucs(ds, u)
    return {"ucs": _ucs_json(ds, u), "checks": checks, "summary": _summary(checks),
            "success": u.found and all(c["passed"] for c in checks),
            "status": (f"ROUTE FOUND: {' > '.join(u.path)}" if u.found
                       else "NO ADMISSIBLE ROUTE - every path violates a hard constraint")}
def mode_sa(ds, p) -> dict:
    sp = p.get("sa", {}) or {}
    u = _stage_ucs(ds)
    u2, problem, s = _stage_sa(ds, u, sp)
    hc = None
    if sp.get("compare_hc", True) and s.feasible:
        hc = simulated_annealing(problem, ds.annealing, ds.mission["seed"], greedy=True)
    checks = validate_sa(ds, u2, s) if s.feasible else [
        {"stage": "SA", "check": "feasible start", "passed": False, "detail": s.termination}]
    out = {"sa": _sa_json(ds, s, problem, hc), "checks": checks, "summary": _summary(checks),
           "success": s.feasible and all(c["passed"] for c in checks),
           "upstream": {"budget_source": "manual" if u2 is not u else "UCS route",
                        "route": u.path, "budget_kW": u2.power_budget_kW, "route_thermal_C": u2.total_thermal_C},
           "status": (f"ALLOCATION FOUND: J* = {s.objective:.3f}" if s.feasible
                      else s.termination)}
    if u.found:
        out["ucs"] = _ucs_json(ds, u)
    return out
def _solve_csp(req, ds, cp: dict):
    kw = dict(use_mrv=bool(cp.get("mrv", True)), use_degree=bool(cp.get("degree", True)),
              use_fc=bool(cp.get("fc", True)))
    first, model, bounds = solve_recovery_csp(req, ds.lab, **kw)
    best, ms_log = first, []
    if first.solved and cp.get("minimize", True):
        H = first.makespan - 1
        while H > 0:
            res, _, _ = solve_recovery_csp(req, ds.lab, horizon=H, node_limit=200_000, trace_limit=0, **kw)
            ms_log.append({"horizon": H, "solved": res.solved, "nodes": res.nodes,
                           "reason": res.failure_reason if not res.solved else f"makespan {res.makespan}"})
            if not res.solved:
                break
            best, H = res, res.makespan - 1
    return first, best, model, bounds, ms_log
def mode_csp(ds, p) -> dict:
    cp = p.get("csp", {}) or {}
    sp = p.get("sa", {}) or {}
    cap = _num(cp, "power_cap_kW", 1, 5000)
    u = _stage_ucs(ds)
    auto_rho = sp.get("rho") in (None, "") and cap is None
    rho = ds.nano.initial_power_fraction
    rho_log = []
    while True:
        ds.nano.initial_power_fraction = rho
        u2, problem, s = _stage_sa(ds, u, sp)
        if not s.feasible:
            raise ApiError("SA could not produce a feasible allocation, so CSP task durations cannot be "
                           "derived: " + s.termination)
        req = derive_requirements(ds, u2, s)
        if cap is not None:
            req = replace(req, power_cap_kW=cap,
                          notes=req.notes + [f"P_res manually overridden to {cap:.1f} kW"])
        first, best, model, bounds, ms_log = _solve_csp(req, ds, cp)
        rho_log.append({"rho": round(rho, 4), "residual_kW": s.residual_power_kW, "solved": first.solved,
                        "reason": first.failure_reason})
        nxt = round(rho - ds.rules["replan_power_fraction_step"], 4)
        if first.solved or not auto_rho or nxt < ds.rules["min_power_fraction"] - 1e-9:
            break
        rho = nxt
    checks = validate_csp(ds, req, best)
    return {"csp": _csp_json(ds, req, first, best, model, bounds, ms_log),
            "checks": checks, "summary": _summary(checks),
            "success": best.solved and all(c["passed"] for c in checks),
            "upstream": {"route": u.path, "budget_kW": u2.power_budget_kW, "allocation": s.allocation,
                         "nano_power_kW": s.nano_power_kW, "residual_kW": s.residual_power_kW,
                         "power_cap_kW": req.power_cap_kW, "rho": s.power_fraction, "rho_auto": auto_rho,
                         "rho_log": rho_log,
                         "cap_source": "manual" if cap is not None else "SA residual"},
            "status": (f"SCHEDULE FOUND: makespan {best.makespan} slots "
                       f"({best.makespan * ds.mission['slot_minutes'] / 60:.1f} h)" if best.solved
                       else "NO SCHEDULE: " + first.failure_reason)}
def mode_integrated(ds, p) -> dict:
    out = run_mission(ds, verbose=False)
    attempts = [{"number": a.number, "rho": a.power_fraction, "excluded": a.excluded_edges,
                 "outcome": a.outcome, "action": a.action,
                 "route": a.ucs.path if a.ucs and a.ucs.found else None,
                 "budget_kW": a.ucs.power_budget_kW if a.ucs and a.ucs.found else None,
                 "allocation": a.sa.allocation if a.sa and a.sa.feasible else None,
                 "objective": a.sa.objective if a.sa and a.sa.feasible else None,
                 "residual_kW": a.sa.residual_power_kW if a.sa and a.sa.feasible else None,
                 "makespan": a.csp.makespan if a.csp and a.csp.solved else None}
                for a in out.attempts]
```

```
    res: Dict[str, Any] = {"attempts": attempts, "status": out.status, "success": out.success,
                           "checks": out.checks, "summary": _summary(out.checks)}
    if out.ucs is not None:
        res["ucs"] = _ucs_json(ds, out.ucs)
    if out.sa is not None and out.ucs is not None and out.ucs.found:
        problem = NanoAllocationProblem(ds.nano, out.ucs.power_budget_kW, out.sa.power_fraction,
                                        out.ucs.total_thermal_C)
        res["sa"] = _sa_json(ds, out.sa, problem)
    if out.csp is not None and out.req is not None and out.csp_first is not None:
        res["csp"] = _csp_json(ds, out.req, out.csp_first, out.csp, out.csp_model,
                               out.bounds_log, out.makespan_search)
    if out.health_after:
        res["health"] = [{"id": s["id"], "name": s["name"], "before": s["health_pct"],
                          "after": out.health_after[s["id"]]} for s in ds.subsystems]
    if not out.checks:
        res["checks"] = [{"stage": "MISSION", "check": "recovery plan produced", "passed": False,
                          "detail": out.status}]
        res["summary"] = _summary(res["checks"])
    return res
_HANDLERS = {"integrated": mode_integrated, "ucs": mode_ucs, "sa": mode_sa, "csp": mode_csp}
def run(params: Optional[dict] = None, dataset_path: Path = DEFAULT_DATASET) -> dict:
    params = params or {}
    mode = params.get("mode", "integrated")
    t0 = time.perf_counter()
    try:
        if mode not in _HANDLERS:
            raise ApiError(f"unknown mode '{mode}' (choose one of {', '.join(MODES)})")
        ds = build_dataset(params, dataset_path)
        body = _HANDLERS[mode](ds, params)
        body.update(ok=True, error=None)
    except ApiError as e:
        body = {"ok": False, "error": str(e), "success": False, "status": "INPUT / FEASIBILITY ERROR"}
        ds = None
    body["mode"] = mode
    body["elapsed_ms"] = round((time.perf_counter() - t0) * 1000, 1)
    if ds is not None:
        body["network"] = _network_json(ds)
        body["subsystems"] = ds.subsystems
    return clean(body)
_PERF_CACHE: Dict[str, dict] = {}
def performance(dataset_path: Path = DEFAULT_DATASET) -> dict:
    if "v" in _PERF_CACHE:
        return _PERF_CACHE["v"]
    ds = load_dataset(dataset_path)
    t0 = time.perf_counter()
    out = run_mission(ds, verbose=False)
    mission_ms = (time.perf_counter() - t0) * 1000
    perf = run_experiments(ds, out)
    u, s, c = out.ucs, out.sa, out.csp
    for r in perf["route_methods"]:
        r["method"] = r["method"].replace(chr(10), " ")
    body = {
        "mission_ms": round(mission_ms, 1),
        "status": out.status, "success": out.success,
        "attempts": [{"number": a.number, "rho": a.power_fraction, "outcome": a.outcome, "action": a.action}
                     for a in out.attempts],
        "checks_passed": sum(ch["passed"] for ch in out.checks), "checks_total": len(out.checks),
        "ucs": {"path": u.path, "cost": u.path_cost, "loss_kW": u.total_loss_kW, "thermal_C": u.total_thermal_C,
                "switch_ms": u.total_switch_ms, "delivered_kW": u.delivered_power_kW, "budget_kW": u.power_budget_kW,
                "efficiency": u.route_efficiency, "expansions": u.expansions, "generated": u.generated,
                "pruned": len(u.pruned)},
        "sa": {"allocation": s.allocation, "objective": s.objective, "initial_objective": s.initial_objective,
               "material_NU": s.material_used_NU, "nano_power_kW": s.nano_power_kW, "power_cap_kW": s.power_cap_kW,
               "residual_kW": s.residual_power_kW, "thermal_C": s.total_thermal_C, "iterations": s.iterations,
               "accepted_worse": s.accepted_worse, "rejected": s.rejected_infeasible, "improvements": s.improvements,
               "rho": s.power_fraction},
        "csp": {"makespan": c.makespan, "slot_minutes": ds.mission["slot_minutes"], "horizon": ds.mission["horizon_slots"],
                "nodes": out.csp_first.nodes, "backtracks": out.csp_first.backtracks,
                "checks": out.csp_first.constraint_checks, "prunes": out.csp_first.fc_prunes,
                "makespan_search": out.makespan_search},
        "performance": {k: v for k, v in perf.items()},
    }
    _PERF_CACHE["v"] = clean(body)
    return _PERF_CACHE["v"]
```

A.14 server.py

```
from __future__ import annotations

import argparse
import json
import mimetypes
import sys
import threading
import webbrowser
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path

from jarvis import api

ROOT = Path(__file__).resolve().parent
WEB = ROOT / "web"
```

```
MAX_BODY = 64 * 1024
mimetypes.add_type("text/javascript", ".js")
class Handler(BaseHTTPRequestHandler):
    server_version = "JARVIS/1.0"

    def log_message(self, fmt, *args):
        sys.stderr.write("[jarvis] " + (fmt % args) + "\n")

    def _send(self, code: int, body: bytes, ctype: str):
        self.send_response(code)
        self.send_header("Content-Type", ctype)
        self.send_header("Content-Length", str(len(body)))
        self.send_header("Cache-Control", "no-store")
        self.end_headers()
        self.wfile.write(body)

    def _json(self, code: int, obj):
        self._send(code, json.dumps(obj, allow_nan=False).encode("utf-8"), "application/json; charset=utf-8")

    def do_GET(self):
        path = self.path.split("?", 1)[0]
        if path == "/api/dataset":
            return self._json(200, api.dataset_info())
        if path == "/api/performance":
            return self._json(200, api.performance())
        if path == "/docs/case_study_source.md":
            src = ROOT / "docs" / "case_study_source.md"
            if src.is_file():
                return self._send(200, src.read_bytes(), "text/markdown; charset=utf-8")
        if path == "/":
            path = "/index.html"
        target = (WEB / path.lstrip("/")).resolve()
        if WEB.resolve() not in target.parents or not target.is_file():
            return self._json(404, {"ok": False, "error": "not found"})
        ctype = mimetypes.guess_type(target.name)[0] or "application/octet-stream"
        if ctype.startswith("text/") or ctype.endswith("javascript"):
            ctype += "; charset=utf-8"
        self._send(200, target.read_bytes(), ctype)

    def do_POST(self):
        if self.path.split("?", 1)[0] != "/api/run":
            return self._json(404, {"ok": False, "error": "not found"})
        length = int(self.headers.get("Content-Length") or 0)
        if length > MAX_BODY:
            return self._json(413, {"ok": False, "error": "request too large"})
        try:
            params = json.loads(self.rfile.read(length) or b"{}")
            if not isinstance(params, dict):
                raise ValueError("body must be a JSON object")
        except ValueError as e:
            return self._json(400, {"ok": False, "error": f"invalid JSON: {e}"})
        try:
            result = api.run(params)
        except Exception as e:
            return self._json(500, {"ok": False, "error": f"{type(e).__name__}: {e}"})
        self._json(200, result)
def main(argv=None) -> int:
    ap = argparse.ArgumentParser(description="JARVIS Mission Console (web frontend)")
    ap.add_argument("--host", default="127.0.0.1")
    ap.add_argument("--port", type=int, default=8085)
    ap.add_argument("--no-browser", action="store_true")
    args = ap.parse_args(argv)
    httpd = ThreadingHTTPServer((args.host, args.port), Handler)
    url = f"http://{args.host}:{args.port}/"
    print(f"JARVIS Mission Console running at {url}  (Ctrl+C to stop)")
    if not args.no_browser:
        threading.Timer(0.6, lambda: webbrowser.open(url)).start()
    try:
        httpd.serve_forever()
    except KeyboardInterrupt:
        print("\nshutting down")
    finally:
        httpd.server_close()
    return 0
if __name__ == "__main__":
    sys.exit(main())
```

A.15 tests/test_mission.py

```
import copy
import sys
import unittest
from dataclasses import replace
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from jarvis.annealing import NanoAllocationProblem, simulated_annealing
from jarvis.csp import solve_recovery_csp
from jarvis.data_models import Node, load_dataset
from jarvis.pipeline import run_mission
from jarvis.requirements import derive_requirements
```

```
from jarvis.ucs import enumerate_simple_paths, uniform_cost_search
from jarvis.validation import validate_csp, validate_sa, validate_ucs

DATA = ROOT / "data" / "mission_dataset.json"
class TestUCS(unittest.TestCase):
    def setUp(self):
        self.ds = load_dataset(DATA)
        self.P = self.ds.reactor["available_output_kW"]

    def test_optimal_against_enumeration(self):
        u = uniform_cost_search(self.ds.network, self.P, self.ds.critical_load_kW)
        best = min(r["cost"] for r in enumerate_simple_paths(self.ds.network) if r["feasible"])
        self.assertTrue(u.found)
        self.assertAlmostEqual(u.path_cost, best, places=6)
        self.assertNotIn("PC1", u.path)
        self.assertTrue(all(c["passed"] for c in validate_ucs(self.ds, u)))

    def test_costs_popped_in_nondecreasing_order(self):
        u = uniform_cost_search(self.ds.network, self.P, self.ds.critical_load_kW)
        gs = [s["g"] for s in u.trace]
        self.assertEqual(gs, sorted(gs))

    def test_excluding_edge_gives_next_route(self):
        u1 = uniform_cost_search(self.ds.network, self.P, self.ds.critical_load_kW)
        u2 = uniform_cost_search(self.ds.network, self.P, self.ds.critical_load_kW,
                                 excluded_edges=[("SW1", "SW2")])
        self.assertTrue(u2.found)
        self.assertGreater(u2.path_cost, u1.path_cost)
        self.assertNotIn(("SW1", "SW2"), list(zip(u2.path, u2.path[1:])))

    def test_thermal_constraint_changes_answer(self):
        blind = uniform_cost_search(self.ds.network, self.P, self.ds.critical_load_kW, enforce_thermal=False)
        safe = uniform_cost_search(self.ds.network, self.P, self.ds.critical_load_kW)
        self.assertGreater(blind.total_thermal_C, self.ds.network.thermal_limit_C)
        self.assertLessEqual(safe.total_thermal_C, self.ds.network.thermal_limit_C)
class TestSA(unittest.TestCase):
    def setUp(self):
        self.ds = load_dataset(DATA)
        self.u = uniform_cost_search(self.ds.network, 1000, self.ds.critical_load_kW)

    def test_feasible_and_reproducible(self):
        prob = NanoAllocationProblem(self.ds.nano, self.u.power_budget_kW, 0.7, self.u.total_thermal_C)
        a = simulated_annealing(prob, self.ds.annealing, seed=85)
        b = simulated_annealing(prob, self.ds.annealing, seed=85)
        self.assertEqual(a.allocation, b.allocation)
        self.assertTrue(all(c["passed"] for c in validate_sa(self.ds, self.u, a)))
        self.assertGreater(a.accepted_worse, 0)

    def test_infeasible_budget_detected(self):
        prob = NanoAllocationProblem(self.ds.nano, 200.0, 0.8, self.u.total_thermal_C)
        r = simulated_annealing(prob, self.ds.annealing, seed=1)
        self.assertFalse(r.feasible)
        self.assertIn("INFEASIBLE", r.termination)
class TestCSP(unittest.TestCase):
    def setUp(self):
        self.ds = load_dataset(DATA)
        u = uniform_cost_search(self.ds.network, 1000, self.ds.critical_load_kW)
        prob = NanoAllocationProblem(self.ds.nano, u.power_budget_kW, 0.7, u.total_thermal_C)
        sa = simulated_annealing(prob, self.ds.annealing, seed=85)
        self.req = derive_requirements(self.ds, u, sa)

    def test_heuristic_and_plain_agree_on_feasibility(self):
        h, _, _ = solve_recovery_csp(self.req, self.ds.lab)
        p, _, _ = solve_recovery_csp(self.req, self.ds.lab, use_mrv=False, use_degree=False, use_fc=False)
        self.assertTrue(h.solved and p.solved)
        self.assertLessEqual(h.nodes, p.nodes)
        self.assertTrue(all(c["passed"] for c in validate_csp(self.ds, self.req, h)))
        self.assertTrue(all(c["passed"] for c in validate_csp(self.ds, self.req, p)))

    def test_tight_horizon_is_infeasible(self):
        r, _, _ = solve_recovery_csp(self.req, self.ds.lab, horizon=10)
        self.assertFalse(r.solved)
class TestIntegratedMission(unittest.TestCase):
    def test_nominal_mission_replans_then_succeeds(self):
        ds = load_dataset(DATA)
        out = run_mission(ds, verbose=False)
        self.assertTrue(out.success)
        self.assertGreaterEqual(len(out.attempts), 2)
        self.assertIn("CSP infeasible", out.attempts[0].outcome)
        self.assertTrue(all(c["passed"] for c in out.checks))

    def test_stress_failed_secondary_converter(self):
        ds = load_dataset(DATA)
        ds.network.nodes["PC2"] = replace(ds.network.nodes["PC2"], status="FAILED")
        out = run_mission(ds, verbose=False)
        if out.success:
            self.assertNotIn("PC2", out.ucs.path)
            self.assertTrue(all(c["passed"] for c in out.checks))
        else:
            self.assertTrue(any("exclude" in a.action or "ABORT" in a.action for a in out.attempts))
if __name__ == "__main__":
    unittest.main()
```

A.16 tests/test_api.py

```
import json
import sys
import threading
import unittest
import urllib.request
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from jarvis import api
def strict_json(obj):
    return json.loads(json.dumps(obj, allow_nan=False))
class TestApiModes(unittest.TestCase):
    def test_integrated_reference_run(self):
        r = strict_json(api.run({"mode": "integrated"}))
        self.assertTrue(r["ok"] and r["success"])
        self.assertEqual(r["ucs"]["path"], ["ARC", "PC2", "SW1", "SW2", "BUS_B", "CLH"])
        self.assertEqual(len(r["attempts"]), 2)
        self.assertEqual(r["csp"]["makespan"], 15)
        self.assertEqual(r["summary"], {"passed": 36, "total": 36, "all_passed": True})

    def test_single_methods(self):
        for mode, key in (("ucs", "ucs"), ("sa", "sa"), ("csp", "csp")):
            r = strict_json(api.run({"mode": mode}))
            self.assertTrue(r["ok"] and r["success"], (mode, r.get("status")))
            self.assertIn(key, r)

    def test_csp_auto_rho_matches_replanner(self):
        r = api.run({"mode": "csp"})
        self.assertTrue(r["upstream"]["rho_auto"])
        self.assertAlmostEqual(r["upstream"]["rho"], 0.7)
        fixed = api.run({"mode": "csp", "sa": {"rho": 0.8}})
        self.assertFalse(fixed["success"])

    def test_sa_manual_budget_and_hill_climb(self):
        r = api.run({"mode": "sa", "sa": {"budget_kW": 600, "compare_hc": True}})
        self.assertEqual(r["upstream"]["budget_source"], "manual")
        self.assertAlmostEqual(r["sa"]["power_cap_kW"], 0.8 * 600)
        self.assertIn("hill_climb", r["sa"])

    def test_stress_scenario_aborts_safely(self):
        r = strict_json(api.run({"mode": "integrated", "mission": {"failed_nodes": ["PC1", "PC2"]}}))
        self.assertTrue(r["ok"])
        self.assertFalse(r["success"])
        self.assertTrue(r["status"].startswith("RECOVERY"))

    def test_bad_input_is_reported_not_raised(self):
        for params in ({"mode": "nope"}, {"mission": {"failed_nodes": ["ARC"]}},
                       {"sa": {"rho": 5}}, {"ucs": {"w_loss": "abc"}}):
            r = api.run(params)
            self.assertFalse(r["ok"], params)
            self.assertTrue(r["error"])

    def test_dataset_info(self):
        info = strict_json(api.dataset_info())
        self.assertEqual(len(info["network"]["nodes"]), 16)
        self.assertEqual(info["defaults"]["mission"]["failed_nodes"], ["PC1"])
class TestServer(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        from http.server import ThreadingHTTPServer
        import server
        cls.httpd = ThreadingHTTPServer(("127.0.0.1", 0), server.Handler)
        cls.base = f"http://127.0.0.1:{cls.httpd.server_address[1]}"
        threading.Thread(target=cls.httpd.serve_forever, daemon=True).start()

    @classmethod
    def tearDownClass(cls):
        cls.httpd.shutdown()
        cls.httpd.server_close()

    def test_index_and_run(self):
        with urllib.request.urlopen(self.base + "/") as resp:
            self.assertIn(b"JARVIS Case Study", resp.read())
        req = urllib.request.Request(self.base + "/api/run", data=json.dumps({"mode": "ucs"}).encode(),
                                     headers={"Content-Type": "application/json"})
        with urllib.request.urlopen(req) as resp:
            self.assertTrue(json.load(resp)["success"])

    def test_no_path_traversal(self):
        with self.assertRaises(urllib.error.HTTPError) as cm:
            urllib.request.urlopen(self.base + "/../server.py")
        self.assertEqual(cm.exception.code, 404)
if __name__ == "__main__":
    unittest.main()
```

A.17 requirements.txt

```
matplotlib>=3.7
networkx>=3.0
python-docx>=1.1
```

Appendix B. Shared Mission Dataset (JSON)


```
{
  "_notice": "FICTIONAL ENGINEERING SIMULATION DATA. All values are invented for an academic AI case study and are not measurements
of any real physical system.",
  "mission": {
    "id": "MK85-REC-001",
    "armor": "Iron Man Mark 85",
    "location": "Stark Tower - Armor Testing Laboratory",
    "trigger": "FRIDAY fault report: Arc Reactor distribution fault, primary converter PC1 offline, nanomaterial reserve depleted,
subsystem damage",
    "slot_minutes": 30,
    "horizon_slots": 16,
    "seed": 85,
    "max_replanning_attempts": 4
  },
  "reactor": {
    "nominal_output_kW": 1500,
    "available_output_kW": 1000,
    "status": "DEGRADED"
  },
  "subsystems": [
    {"id": "FLC", "name": "Flight Controller",        "health_pct": 62, "critical_load_kW": 120, "status": "DEGRADED"},
    {"id": "STB", "name": "Stabilization System",     "health_pct": 55, "critical_load_kW": 110, "status": "DEGRADED"},
    {"id": "SEN", "name": "Sensor Array",             "health_pct": 71, "critical_load_kW": 60,  "status": "MISCALIBRATED"},
    {"id": "LSC", "name": "Life-Support Controller",  "health_pct": 88, "critical_load_kW": 130, "status": "NOMINAL-ON-BACKUP"},
    {"id": "STR", "name": "Structural Shell",         "health_pct": 47, "critical_load_kW": 0,   "status": "BREACHED"}
  ],
  "power_network": {
    "source": "ARC",
    "goal": "CLH",
    "required_throughput_kW": 600,
    "thermal_limit_C": 14.0,
    "cost_weights": {"loss": 1.0, "thermal": 0.5, "switching": 0.4},
    "nodes": [
      {"id": "ARC",   "label": "Arc Reactor",            "type": "source",    "status": "DEGRADED"},
      {"id": "PC1",   "label": "Primary Converter",      "type": "converter", "status": "FAILED"},
      {"id": "PC2",   "label": "Secondary Converter",    "type": "converter", "status": "OK"},
      {"id": "PC3",   "label": "Aux Micro-Converter",    "type": "converter", "status": "OK"},
      {"id": "BKP",   "label": "Backup Storage",         "type": "storage",   "status": "OK"},
      {"id": "SW1",   "label": "Switching Unit 1",       "type": "switch",    "status": "OK"},
      {"id": "SW2",   "label": "Switching Unit 2",       "type": "switch",    "status": "OK"},
      {"id": "SW3",   "label": "Switching Unit 3",       "type": "switch",    "status": "OK"},
      {"id": "BUS_A", "label": "Distribution Bus A",     "type": "bus",       "status": "OK"},
      {"id": "BUS_B", "label": "Distribution Bus B",     "type": "bus",       "status": "OK"},
      {"id": "BUS_C", "label": "Distribution Bus C",     "type": "bus",       "status": "OK"},
      {"id": "CLH",   "label": "Critical Load Hub",      "type": "hub",       "status": "OK"},
      {"id": "FLC",   "label": "Flight Controller",      "type": "load",      "status": "DEGRADED"},
      {"id": "STB",   "label": "Stabilization System",   "type": "load",      "status": "DEGRADED"},
      {"id": "SEN",   "label": "Sensor Array",           "type": "load",      "status": "MISCALIBRATED"},
      {"id": "LSC",   "label": "Life-Support Controller","type": "load",      "status": "OK"}
    ],
    "edges": [
      {"u": "ARC",   "v": "PC1",   "loss_kW": 6,  "thermal_C": 1.5, "switch_ms": 1, "capacity_kW": 1200},
      {"u": "ARC",   "v": "PC2",   "loss_kW": 14, "thermal_C": 3.0, "switch_ms": 2, "capacity_kW": 950},
      {"u": "ARC",   "v": "PC3",   "loss_kW": 8,  "thermal_C": 2.0, "switch_ms": 1, "capacity_kW": 520},
      {"u": "ARC",   "v": "BKP",   "loss_kW": 22, "thermal_C": 1.0, "switch_ms": 4, "capacity_kW": 700},
      {"u": "PC1",   "v": "SW1",   "loss_kW": 3,  "thermal_C": 1.0, "switch_ms": 1, "capacity_kW": 1200},
      {"u": "PC2",   "v": "SW1",   "loss_kW": 9,  "thermal_C": 2.5, "switch_ms": 3, "capacity_kW": 960},
      {"u": "PC2",   "v": "SW2",   "loss_kW": 5,  "thermal_C": 4.0, "switch_ms": 2, "capacity_kW": 850},
      {"u": "PC3",   "v": "SW2",   "loss_kW": 4,  "thermal_C": 2.0, "switch_ms": 2, "capacity_kW": 520},
      {"u": "BKP",   "v": "SW3",   "loss_kW": 10, "thermal_C": 1.0, "switch_ms": 3, "capacity_kW": 700},
      {"u": "SW1",   "v": "SW2",   "loss_kW": 2,  "thermal_C": 1.0, "switch_ms": 2, "capacity_kW": 960},
      {"u": "SW1",   "v": "BUS_A", "loss_kW": 7,  "thermal_C": 2.0, "switch_ms": 1, "capacity_kW": 800},
      {"u": "SW1",   "v": "BUS_B", "loss_kW": 10, "thermal_C": 1.5, "switch_ms": 1, "capacity_kW": 780},
      {"u": "SW2",   "v": "BUS_B", "loss_kW": 4,  "thermal_C": 3.5, "switch_ms": 1, "capacity_kW": 980},
      {"u": "SW2",   "v": "BUS_C", "loss_kW": 9,  "thermal_C": 1.0, "switch_ms": 1, "capacity_kW": 750},
      {"u": "SW3",   "v": "BUS_C", "loss_kW": 5,  "thermal_C": 1.0, "switch_ms": 1, "capacity_kW": 700},
      {"u": "BUS_A", "v": "CLH",   "loss_kW": 8,  "thermal_C": 4.5, "switch_ms": 0, "capacity_kW": 900},
      {"u": "BUS_B", "v": "CLH",   "loss_kW": 5,  "thermal_C": 4.0, "switch_ms": 0, "capacity_kW": 1000},
      {"u": "BUS_C", "v": "CLH",   "loss_kW": 12, "thermal_C": 2.0, "switch_ms": 0, "capacity_kW": 760}
    ],
    "load_edges": [
      {"u": "CLH", "v": "FLC"}, {"u": "CLH", "v": "STB"}, {"u": "CLH", "v": "SEN"}, {"u": "CLH", "v": "LSC"}
    ]
  },
  "nanomaterial": {
    "reserve_NU": 1200,
    "armor_thermal_limit_C": 30.0,
    "initial_power_fraction": 0.80,
    "objective_weights": {"energy": 12.0, "material": 6.0, "packet_waste": 1.2},
    "categories": [
      {"id": "SP", "name": "Structural Protection",  "min_NU": 300, "max_NU": 700, "power_kW_per_NU": 0.30, "heat_C_per_NU": 0.010,
"benefit_weight": 0.30, "saturation_NU": 260, "packet_NU": 40},
      {"id": "ES", "name": "Emergency Shielding",    "min_NU": 80,  "max_NU": 400, "power_kW_per_NU": 0.60, "heat_C_per_NU": 0.018,
"benefit_weight": 0.20, "saturation_NU": 150, "packet_NU": 30},
      {"id": "FS", "name": "Flight Stabilization",   "min_NU": 60,  "max_NU": 350, "power_kW_per_NU": 0.70, "heat_C_per_NU": 0.020,
"benefit_weight": 0.20, "saturation_NU": 120, "packet_NU": 25},
      {"id": "SR", "name": "Structural Repair",      "min_NU": 100, "max_NU": 500, "power_kW_per_NU": 0.45, "heat_C_per_NU": 0.012,
"benefit_weight": 0.20, "saturation_NU": 200, "packet_NU": 50},
      {"id": "SN", "name": "Sensor Protection",      "min_NU": 40,  "max_NU": 250, "power_kW_per_NU": 0.35, "heat_C_per_NU": 0.008,
"benefit_weight": 0.10, "saturation_NU": 80,  "packet_NU": 20}
    ]
```

```
  },
  "annealing": {
    "initial_temperature": 4.0,
    "cooling_rate": 0.92,
    "min_temperature": 0.005,
    "iterations_per_temperature": 40,
    "max_iterations": 4000,
    "stall_epochs": 25,
    "step_min_NU": 1,
    "step_max_NU": 15
  },
  "requirement_rules": {
    "struct_restoration_NU_per_slot": 80,
    "replenishment_NU_per_slot": 400,
    "sensor_calibration_NU_per_slot": 100,
    "flight_sync_NU_per_slot": 150,
    "thermal_inspection_C_per_slot": 15,
    "armor_battery_capacity_kWh": 400,
    "armor_state_of_charge": 0.35,
    "target_state_of_charge": 0.90,
    "charger_rating_kW": 120,
    "converter_repair_slots_per_failed_unit": 3,
    "replan_power_fraction_step": 0.10,
    "min_power_fraction": 0.40
  },
  "laboratory": {
    "resources": {
      "ARM":     [{"id": "ARM-1", "capabilities": ["nano", "standard"]}, {"id": "ARM-2", "capabilities": ["precision",
"standard"]}],
      "STATION": [{"id": "RS-E",  "capabilities": ["electrical"]},       {"id": "RS-N",  "capabilities": ["nanofab"]}],
      "BAY":     [{"id": "BAY-1", "capabilities": ["standard", "charging-dock"]}, {"id": "BAY-2", "capabilities": ["standard",
"thermal-shielded"]}],
      "CHARGER": [{"id": "CHG-1", "capabilities": ["armor-charging"]}],
      "TESTRIG": [{"id": "TEST-RIG", "capabilities": ["diagnostics"]}]
    },
    "technicians": 2,
    "tasks": [
      {"id": "ARC_DIAG",     "name": "Arc Reactor Diagnostics",       "duration_rule": "fixed:2",            "power_kW": 40,
"technicians": 1,
       "needs": {"BAY": "standard", "TESTRIG": "diagnostics"}, "after": [], "deadline": null},
      {"id": "CONV_REPAIR",  "name": "Power-Converter Repair",        "duration_rule": "converter_repair",   "power_kW": 60,
"technicians": 1,
       "needs": {"ARM": "standard", "STATION": "electrical"}, "after": ["ARC_DIAG"], "deadline": 8},
      {"id": "NANO_REPL",    "name": "Nanomaterial Replenishment",    "duration_rule": "replenishment",      "power_kW": 70,
"technicians": 0,
       "needs": {"ARM": "nano", "STATION": "nanofab"}, "after": ["STRUCT_REST"], "deadline": null},
      {"id": "STRUCT_REST",  "name": "Structural Restoration",        "duration_rule": "struct_restoration", "power_kW": 90,
"technicians": 1,
       "needs": {"ARM": "nano", "BAY": "standard"}, "after": ["ARC_DIAG"], "deadline": null},
      {"id": "SENSOR_CAL",   "name": "Sensor Calibration",            "duration_rule": "sensor_calibration", "power_kW": 30,
"technicians": 0,
       "needs": {"ARM": "precision", "BAY": "standard", "TESTRIG": "diagnostics"}, "after": ["STRUCT_REST"], "deadline": null},
      {"id": "FLC_SYNC",     "name": "Flight-Controller Synchronization", "duration_rule": "flight_sync",    "power_kW": 35,
"technicians": 1,
       "needs": {"TESTRIG": "diagnostics"}, "after": ["CONV_REPAIR", "SENSOR_CAL"], "deadline": null},
      {"id": "CHARGE",       "name": "Armor Charging",                "duration_rule": "charging",           "power_kW": 120,
"technicians": 0,
       "needs": {"BAY": "charging-dock", "CHARGER": "armor-charging"}, "after": ["CONV_REPAIR"], "deadline": null},
      {"id": "THERMAL_INSP", "name": "Thermal Inspection",            "duration_rule": "thermal_inspection", "power_kW": 20,
"technicians": 1,
       "needs": {"BAY": "thermal-shielded"}, "after": ["CHARGE"], "deadline": null},
      {"id": "SAFETY_TEST",  "name": "Final Safety Testing",          "duration_rule": "fixed:2",            "power_kW": 100,
"technicians": 2,
       "needs": {"BAY": "thermal-shielded", "TESTRIG": "diagnostics"}, "after": ["FLC_SYNC", "THERMAL_INSP", "NANO_REPL"],
"deadline": 16}
    ],
    "separations": [
      {"first": "CHARGE", "second": "THERMAL_INSP", "min_gap_slots": 1, "reason": "post-charge cool-down before thermal imaging"}
    ],
    "no_overlap": [
      {"a": "STRUCT_REST", "b": "CHARGE", "reason": "no nanite fabrication during high-current charging"},
      {"a": "ARC_DIAG",    "b": "CHARGE", "reason": "reactor diagnostics require isolated reactor"}
    ]
  }
}
```

Appendix C. Console Output of the Executed Mission

Unedited output of python main.py (outputs/console_log.txt).

```
================================================================================
JARVIS :: MARK 85 INTELLIGENT RECOVERY  (fictional engineering simulation)
================================================================================
Mission          : MK85-REC-001 at Stark Tower - Armor Testing Laboratory
Trigger          : FRIDAY fault report: Arc Reactor distribution fault, primary converter PC1 offline, nanomaterial reserve
depleted, subsystem damage
Arc Reactor      : 1000 kW available of 1500 kW nominal (DEGRADED)
Critical load    : 420 kW  (FLC+STB+SEN+LSC)
Nanomaterial     : 1200 NU reserve, armor thermal limit 30.0 C
Laboratory       : 9 tasks, 2 technicians, horizon 16 x 30 min

Subsystem status reported by FRIDAY:
  FLC  Flight Controller          health  62%  load  120 kW  DEGRADED
  STB  Stabilization System       health  55%  load  110 kW  DEGRADED
  SEN  Sensor Array               health  71%  load   60 kW  MISCALIBRATED
  LSC  Life-Support Controller    health  88%  load  130 kW  NOMINAL-ON-BACKUP
  STR  Structural Shell           health  47%  load    0 kW  BREACHED

=== JARVIS planning attempt 1: rho=0.80, excluded edges=none ===
[UCS] route ARC -> PC2 -> SW1 -> SW2 -> BUS_B -> CLH cost=44.20 delivered=950.0 kW budget B=530.0 kW
[SA ] allocation {'SP': 360, 'ES': 151, 'FS': 125, 'SR': 200, 'SN': 120} J=54.246 P_nano=418.1 kW P_res=111.9 kW
[CSP] CSP infeasible: CHARGE draws 120 kW > available P_res 111.9 kW; empty domain for CHARGE -> reduce nanomaterial power fraction
rho 0.80 -> 0.70 to free laboratory power; re-run SA and CSP

=== JARVIS planning attempt 2: rho=0.70, excluded edges=none ===
[UCS] route ARC -> PC2 -> SW1 -> SW2 -> BUS_B -> CLH cost=44.20 delivered=950.0 kW budget B=530.0 kW
[SA ] allocation {'SP': 320, 'ES': 122, 'FS': 125, 'SR': 174, 'SN': 100} J=50.831 P_nano=370.0 kW P_res=160.0 kW
[CSP] schedule found: makespan=15 slots, nodes=10, backtracks=0

================================================================================
STAGE 1 - UNIFORM COST SEARCH: Arc Reactor power routing
================================================================================
Edge cost c(e) = 1.0*loss + 0.5*thermal + 0.4*switching ; P_req = 600 kW ; Theta_max = 14 C

edge           loss kW  therm C  sw ms  cap kW    cost
ARC->PC1             6      1.5      1    1200    7.15
ARC->PC2            14        3      2     950   16.30
ARC->PC3             8        2      1     520    9.40
ARC->BKP            22        1      4     700   24.10
PC1->SW1             3        1      1    1200    3.90
PC2->SW1             9      2.5      3     960   11.45
PC2->SW2             5        4      2     850    7.80
PC3->SW2             4        2      2     520    5.80
BKP->SW3            10        1      3     700   11.70
SW1->SW2             2        1      2     960    3.30
SW1->BUS_A           7        2      1     800    8.40
SW1->BUS_B          10      1.5      1     780   11.15
SW2->BUS_B           4      3.5      1     980    6.15
SW2->BUS_C           9        1      1     750    9.90
SW3->BUS_C           5        1      1     700    5.90
BUS_A->CLH           8      4.5      0     900   10.25
BUS_B->CLH           5        4      0    1000    7.00
BUS_C->CLH          12        2      0     760   13.00

Expansion trace (frontier shown as node:g/theta, ordered by g):
 step  1: expand ARC    g=  0.00 theta=  0.0 path=ARC
          push PC2    g= 16.30 theta=  3.0 (edge 16.3)
          push BKP    g= 24.10 theta=  1.0 (edge 24.1)
          reject PC1    PC1 FAILED (FRIDAY fault)
          reject PC3    capacity 520 < required 600 kW
          frontier: PC2:16.3/3.0, BKP:24.1/1.0
 step  2: expand PC2    g= 16.30 theta=  3.0 path=ARC>PC2
          push SW1    g= 27.75 theta=  5.5 (edge 11.45)
          push SW2    g= 24.10 theta=  7.0 (edge 7.8)
          frontier: BKP:24.1/1.0, SW2:24.1/7.0, SW1:27.75/5.5
 step  3: expand BKP    g= 24.10 theta=  1.0 path=ARC>BKP
          push SW3    g= 35.80 theta=  2.0 (edge 11.7)
          frontier: SW2:24.1/7.0, SW1:27.75/5.5, SW3:35.8/2.0
 step  4: expand SW2    g= 24.10 theta=  7.0 path=ARC>PC2>SW2
          push BUS_B  g= 30.25 theta= 10.5 (edge 6.15)
          push BUS_C  g= 34.00 theta=  8.0 (edge 9.9)
          frontier: SW1:27.75/5.5, BUS_B:30.25/10.5, BUS_C:34.0/8.0, SW3:35.8/2.0
 step  5: expand SW1    g= 27.75 theta=  5.5 path=ARC>PC2>SW1
          push SW2    g= 31.05 theta=  6.5 (edge 3.3)
          push BUS_A  g= 36.15 theta=  7.5 (edge 8.4)
          push BUS_B  g= 38.90 theta=  7.0 (edge 11.15)
          frontier: BUS_B:30.25/10.5, SW2:31.05/6.5, BUS_C:34.0/8.0, SW3:35.8/2.0, BUS_A:36.15/7.5, BUS_B:38.9/7.0
 step  6: expand BUS_B  g= 30.25 theta= 10.5 path=ARC>PC2>SW2>BUS_B
          reject CLH    thermal 14.5 > limit 14.0 C
          frontier: SW2:31.05/6.5, BUS_C:34.0/8.0, SW3:35.8/2.0, BUS_A:36.15/7.5, BUS_B:38.9/7.0
 step  7: expand SW2    g= 31.05 theta=  6.5 path=ARC>PC2>SW1>SW2
          push BUS_B  g= 37.20 theta= 10.0 (edge 6.15)
          push BUS_C  g= 40.95 theta=  7.5 (edge 9.9)
          frontier: BUS_C:34.0/8.0, SW3:35.8/2.0, BUS_A:36.15/7.5, BUS_B:37.2/10.0, BUS_B:38.9/7.0, BUS_C:40.95/7.5
 step  8: expand BUS_C  g= 34.00 theta=  8.0 path=ARC>PC2>SW2>BUS_C
          push CLH    g= 47.00 theta= 10.0 (edge 13.0)
          frontier: SW3:35.8/2.0, BUS_A:36.15/7.5, BUS_B:37.2/10.0, BUS_B:38.9/7.0, BUS_C:40.95/7.5, CLH:47.0/10.0
 step  9: expand SW3    g= 35.80 theta=  2.0 path=ARC>BKP>SW3
```

```
          push BUS_C  g= 41.70 theta=  3.0 (edge 5.9)
          frontier: BUS_A:36.15/7.5, BUS_B:37.2/10.0, BUS_B:38.9/7.0, BUS_C:40.95/7.5, BUS_C:41.7/3.0, CLH:47.0/10.0
 step 10: expand BUS_A  g= 36.15 theta=  7.5 path=ARC>PC2>SW1>BUS_A
          push CLH    g= 46.40 theta= 12.0 (edge 10.25)
          frontier: BUS_B:37.2/10.0, BUS_B:38.9/7.0, BUS_C:40.95/7.5, BUS_C:41.7/3.0, CLH:46.4/12.0, CLH:47.0/10.0
 step 11: expand BUS_B  g= 37.20 theta= 10.0 path=ARC>PC2>SW1>SW2>BUS_B
          push CLH    g= 44.20 theta= 14.0 (edge 7.0)
          frontier: BUS_B:38.9/7.0, BUS_C:40.95/7.5, BUS_C:41.7/3.0, CLH:44.2/14.0, CLH:46.4/12.0, CLH:47.0/10.0
 step 12: expand BUS_B  g= 38.90 theta=  7.0 path=ARC>PC2>SW1>BUS_B
          push CLH    g= 45.90 theta= 11.0 (edge 7.0)
          frontier: BUS_C:40.95/7.5, BUS_C:41.7/3.0, CLH:44.2/14.0, CLH:45.9/11.0, CLH:46.4/12.0, CLH:47.0/10.0
 step 13: expand BUS_C  g= 40.95 theta=  7.5 path=ARC>PC2>SW1>SW2>BUS_C
          push CLH    g= 53.95 theta=  9.5 (edge 13.0)
          frontier: BUS_C:41.7/3.0, CLH:44.2/14.0, CLH:45.9/11.0, CLH:46.4/12.0, CLH:47.0/10.0, CLH:53.95/9.5
 step 14: expand BUS_C  g= 41.70 theta=  3.0 path=ARC>BKP>SW3>BUS_C
          push CLH    g= 54.70 theta=  5.0 (edge 13.0)
          frontier: CLH:44.2/14.0, CLH:45.9/11.0, CLH:46.4/12.0, CLH:47.0/10.0, CLH:53.95/9.5, CLH:54.7/5.0
 step 15: expand CLH    g= 44.20 theta= 14.0 path=ARC>PC2>SW1>SW2>BUS_B>CLH   <-- GOAL
          frontier: CLH:45.9/11.0, CLH:46.4/12.0, CLH:47.0/10.0, CLH:53.95/9.5, CLH:54.7/5.0

OPTIMAL ROUTE   : ARC -> PC2 -> SW1 -> SW2 -> BUS_B -> CLH
path cost g*    : 44.20
losses / thermal / switching : 34 kW / 14 C / 8 ms
bottleneck cap  : 950 kW
delivered power : min(1000 - 34, 950) = 950 kW
power budget B  : 950 - 420 = 530 kW  -> Stage 2
route efficiency eta = 0.950  -> Stage 3 charging rate
expanded 15 labels, generated 19

================================================================================
STAGE 2 - SIMULATED ANNEALING: nanomaterial allocation
================================================================================
power cap rho*B = 0.70 x 530.0 = 371.0 kW
initial (mandatory minimums) {'SP': 300, 'ES': 80, 'FS': 60, 'SR': 100, 'SN': 40}  J0 = 38.287

 iter        T move                       cand J       dE  P(acc)  decision
    1   4.0000 add 4 NU to SP             38.264   +0.023   0.994  ACCEPT-WORSE
    2   4.0000 add 13 NU to SN            38.226   +0.038   0.990  ACCEPT-WORSE
    3   4.0000 add 7 NU to SP             38.181   +0.045   0.989  ACCEPT-WORSE
    4   4.0000 move 3 NU SN->ES           38.259   -0.078   1.000  ACCEPT-BETTER
    5   4.0000 remove 9 NU from SN        38.278   -0.019   1.000  ACCEPT-BETTER
    6   4.0000 add 15 NU to SP            39.359   -1.082   1.000  ACCEPT-BETTER (new best)
    7   4.0000 move 3 NU SP->FS           39.478   -0.119   1.000  ACCEPT-BETTER (new best)
    8   4.0000 add 1 NU to FS             39.508   -0.029   1.000  ACCEPT-BETTER (new best)
    9   4.0000 remove 4 NU from SP        38.341   +1.167   0.747  ACCEPT-WORSE
   10   4.0000 add 9 NU to ES             39.683   -1.342   1.000  ACCEPT-BETTER (new best)
   11   4.0000 remove 3 NU from ES        38.440   +1.243   0.733  REJECT
   12   4.0000 add 14 NU to SN            39.624   +0.059   0.985  ACCEPT-WORSE
   13   4.0000 move 11 NU ES->FS          39.715   -0.091   1.000  ACCEPT-BETTER (new best)
   14   4.0000 remove 2 NU from SN        39.733   -0.019   1.000  ACCEPT-BETTER (new best)
   15   4.0000 move 4 NU SP->ES           39.836   -0.103   1.000  ACCEPT-BETTER (new best)
   16   4.0000 remove 10 NU from SP       39.904   -0.068   1.000  ACCEPT-BETTER (new best)
   17   4.0000 move 11 NU SN->SR          40.164   -0.260   1.000  ACCEPT-BETTER (new best)
   18   4.0000 move 2 NU ES->SN           40.129   +0.035   0.991  ACCEPT-WORSE
   19   4.0000 move 3 NU SP->SR           40.200   -0.071   1.000  ACCEPT-BETTER (new best)
   20   4.0000 move 3 NU ES->SN           40.137   +0.063   0.984  ACCEPT-WORSE
   21   4.0000 remove 6 NU from FS        38.802   +1.336   0.716  ACCEPT-WORSE
   22   4.0000 remove 2 NU from FS        38.750   +0.051   0.987  ACCEPT-WORSE
   23   4.0000 move 10 NU SR->SN          38.486   +0.264   0.936  ACCEPT-WORSE
   24   4.0000 move 11 NU SN->SR          38.770   -0.285   1.000  ACCEPT-BETTER
   25   4.0000 add 15 NU to SN            39.839   -1.068   1.000  ACCEPT-BETTER
  ...  (every 400th iteration below)
  400   1.8886 move 7 NU SP->ES           46.178   -0.083   1.000  ACCEPT-BETTER
  800   0.8204 move 1 NU ES->SN           47.145   +0.023   0.973  ACCEPT-WORSE
 1200   0.3564 remove 2 NU from SN        49.268   -0.039   1.000  ACCEPT-BETTER
 1600   0.1548 remove 5 NU from SR        49.441   +1.243   0.000  REJECT
 2000   0.0672 move 14 NU SR->SP          50.466   +0.230   0.033  REJECT
 2400   0.0292 remove 1 NU from ES        49.537   +1.202   0.000  REJECT
 2800   0.0127 move 5 NU ES->SR           49.617   +1.196   0.000  REJECT
 3200   0.0055 remove 4 NU from ES        49.576   +1.204   0.000  REJECT

termination     : temperature 0.0047 < T_min 0.005
iterations 3240, accepted-worse 672, rejected-infeasible neighbours 890, best improvements 44
BEST ALLOCATION : {'SP': 320, 'ES': 122, 'FS': 125, 'SR': 174, 'SN': 100}
objective J*    : 50.8307  (from 38.2866)
material used   : 841 / 1200 NU
nano power      : 370.0 kW (cap 371.0)
residual P_res  : 160.0 kW  -> Stage 3 power constraint
total thermal   : 24.78 C (limit 30.0)

================================================================================
STAGE 3 - CSP: Stark Laboratory recovery scheduling
================================================================================
Derived inputs:
  Power cap for concurrent lab operations P_res = B - P_nano = 530.0 - 370.0 = 160.0 kW
  Route efficiency eta = 0.950 scales the armor charging rate
  Combined thermal load 24.78 C sets the thermal-inspection length

task           dur    kW tech  needs / after / derivation
ARC_DIAG         2    40    1  {'BAY': 'standard', 'TESTRIG': 'diagnostics'} after=- dl=None  [fixed procedure length]
CONV_REPAIR      3    60    1  {'ARM': 'standard', 'STATION': 'electrical'} after=['ARC_DIAG'] dl=8  [3 slots x 1 failed unit(s)
['PC1']]
NANO_REPL        3    70    0  {'ARM': 'nano', 'STATION': 'nanofab'} after=['STRUCT_REST'] dl=None  [ceil(M=841 NU consumed / 400
NU per slot)]
STRUCT_REST      3    90    1  {'ARM': 'nano', 'BAY': 'standard'} after=['ARC_DIAG'] dl=None  [ceil(x_SR=174 / 80 NU per slot)]
```

```
SENSOR_CAL       2    30    0  {'ARM': 'precision', 'BAY': 'standard', 'TESTRIG': 'diagnostics'} after=['STRUCT_REST'] dl=None  [1
+ ceil(x_SN=100 / 100)]
FLC_SYNC         2    35    1  {'TESTRIG': 'diagnostics'} after=['CONV_REPAIR', 'SENSOR_CAL'] dl=None  [1 + ceil(x_FS=125 / 150)]
CHARGE           4   120    0  {'BAY': 'charging-dock', 'CHARGER': 'armor-charging'} after=['CONV_REPAIR'] dl=None  [ceil(deficit
220 kWh / (120 kW x eta=0.950 x 0.5 h))]
THERMAL_INSP     3    20    1  {'BAY': 'thermal-shielded'} after=['CHARGE'] dl=None  [1 + ceil(theta_total=24.78 C / 15)]
SAFETY_TEST      2   100    2  {'BAY': 'thermal-shielded', 'TESTRIG': 'diagnostics'} after=['FLC_SYNC', 'THERMAL_INSP',
'NANO_REPL'] dl=16  [fixed procedure length]

Initial domain sizes (after unary + bound propagation):
  ARC_DIAG:4, CONV_REPAIR:4, NANO_REPL:7, STRUCT_REST:12, SENSOR_CAL:12, FLC_SYNC:6, CHARGE:2, THERMAL_INSP:2, SAFETY_TEST:2
  bound: CONV_REPAIR start >= 2 (after ARC_DIAG) |D| 12->8
  bound: ARC_DIAG start <= 3 (before CONV_REPAIR) |D| 30->8
  bound: NANO_REPL start >= 3 (after STRUCT_REST) |D| 14->11
  bound: STRUCT_REST start <= 10 (before NANO_REPL) |D| 28->22
  bound: STRUCT_REST start >= 2 (after ARC_DIAG) |D| 22->18
  bound: SENSOR_CAL start >= 5 (after STRUCT_REST) |D| 30->20
  bound: FLC_SYNC start >= 5 (after CONV_REPAIR) |D| 15->10
  bound: FLC_SYNC start >= 7 (after SENSOR_CAL) |D| 10->8
  bound: SENSOR_CAL start <= 12 (before FLC_SYNC) |D| 20->16
  bound: CHARGE start >= 5 (after CONV_REPAIR) |D| 13->8
  bound: THERMAL_INSP start >= 10 (after CHARGE) |D| 14->4
  bound: CHARGE start <= 8 (before THERMAL_INSP) |D| 8->4
  bound: SAFETY_TEST start >= 9 (after FLC_SYNC) |D| 15->6
  bound: FLC_SYNC start <= 12 (before SAFETY_TEST) |D| 8->6
  bound: SAFETY_TEST start >= 13 (after THERMAL_INSP) |D| 6->2
  bound: THERMAL_INSP start <= 11 (before SAFETY_TEST) |D| 4->2
  bound: NANO_REPL start <= 11 (before SAFETY_TEST) |D| 11->9
  bound: NANO_REPL start >= 5 (after STRUCT_REST) |D| 9->7
  bound: STRUCT_REST start <= 8 (before NANO_REPL) |D| 18->14
  bound: SENSOR_CAL start <= 10 (before FLC_SYNC) |D| 16->12
  bound: CHARGE start <= 6 (before THERMAL_INSP) |D| 4->2
  bound: STRUCT_REST start <= 7 (before SENSOR_CAL) |D| 14->12
  bound: CONV_REPAIR start <= 3 (before CHARGE) |D| 8->4
  bound: ARC_DIAG start <= 1 (before CONV_REPAIR) |D| 8->4

Search trace (first solve):
  SELECT SAFETY_TEST (MRV |D|=2, degree=6)
  ASSIGN SAFETY_TEST=(start 13, end 15, BAY-2/TEST-RIG)
    FC: NANO_REPL 7->6, FLC_SYNC 6->5, THERMAL_INSP 2->1
    SELECT THERMAL_INSP (MRV |D|=1, degree=4)
    ASSIGN THERMAL_INSP=(start 10, end 13, BAY-2)
      FC: SENSOR_CAL 12->10, CHARGE 2->1
      SELECT CHARGE (MRV |D|=1, degree=4)
      ASSIGN CHARGE=(start 5, end 9, BAY-1/CHG-1)
        FC: CONV_REPAIR 4->2, NANO_REPL 6->2, STRUCT_REST 12->2, SENSOR_CAL 10->6
        SELECT CONV_REPAIR (MRV |D|=2, degree=5)
        ASSIGN CONV_REPAIR=(start 2, end 5, ARM-1/RS-E)
          FC WIPEOUT: domain of STRUCT_REST empty -> undo CONV_REPAIR
        ASSIGN CONV_REPAIR=(start 2, end 5, ARM-2/RS-E)
          FC: ARC_DIAG 4->2
          SELECT ARC_DIAG (MRV |D|=2, degree=3)
          ASSIGN ARC_DIAG=(start 0, end 2, BAY-1/TEST-RIG)
            SELECT STRUCT_REST (MRV |D|=2, degree=2)
            ASSIGN STRUCT_REST=(start 2, end 5, ARM-1/BAY-1)
              SELECT NANO_REPL (MRV |D|=2, degree=0)
              ASSIGN NANO_REPL=(start 9, end 12, ARM-1/RS-N)
                SELECT FLC_SYNC (MRV |D|=5, degree=1)
                ASSIGN FLC_SYNC=(start 7, end 9, TEST-RIG)
                  FC: SENSOR_CAL 6->1
                  SELECT SENSOR_CAL (MRV |D|=1, degree=0)
                  ASSIGN SENSOR_CAL=(start 5, end 7, ARM-2/BAY-2/TEST-RIG)

first solve: nodes=10 backtracks=0 checks=424 FC-prunes=37 wipeouts=1 makespan=15
Makespan tightening:
  H=14: infeasible (critical path forces SAFETY_TEST to end at slot 15 > limit 14; empty domain for THERMAL_INSP; empty domain for
SAFETY_TEST)

FINAL SCHEDULE (makespan 15 slots):
task           start  end  resources
ARC_DIAG           0    2  {'BAY': 'BAY-1', 'TESTRIG': 'TEST-RIG'}
CONV_REPAIR        2    5  {'ARM': 'ARM-2', 'STATION': 'RS-E'}
STRUCT_REST        2    5  {'ARM': 'ARM-1', 'BAY': 'BAY-1'}
CHARGE             5    9  {'BAY': 'BAY-1', 'CHARGER': 'CHG-1'}
SENSOR_CAL         5    7  {'ARM': 'ARM-2', 'BAY': 'BAY-2', 'TESTRIG': 'TEST-RIG'}
FLC_SYNC           7    9  {'TESTRIG': 'TEST-RIG'}
NANO_REPL          9   12  {'ARM': 'ARM-1', 'STATION': 'RS-N'}
THERMAL_INSP      10   13  {'BAY': 'BAY-2'}
SAFETY_TEST       13   15  {'BAY': 'BAY-2', 'TESTRIG': 'TEST-RIG'}

============================================================================
CONTROLLED REPLANNING LOG
============================================================================
attempt 1: rho=0.80 excluded=-
   UCS route ARC > PC2 > SW1 > SW2 > BUS_B > CLH  B=530.0 kW
   SA  {'SP': 360, 'ES': 151, 'FS': 125, 'SR': 200, 'SN': 120} J=54.246 P_res=111.9 kW
   outcome: CSP infeasible: CHARGE draws 120 kW > available P_res 111.9 kW; empty domain for CHARGE
   action : reduce nanomaterial power fraction rho 0.80 -> 0.70 to free laboratory power; re-run SA and CSP
attempt 2: rho=0.70 excluded=-
   UCS route ARC > PC2 > SW1 > SW2 > BUS_B > CLH  B=530.0 kW
   SA  {'SP': 320, 'ES': 122, 'FS': 125, 'SR': 174, 'SN': 100} J=50.831 P_res=160.0 kW
   outcome: feasible
   action : validate and commit

============================================================================
```

```
INTEGRATED VALIDATION
================================================================================
[PASS] UCS         route found                                          ARC -> PC2 -> SW1 -> SW2 -> BUS_B -> CLH
[PASS] UCS         starts at source / ends at goal                      ARC ... CLH
[PASS] UCS         avoids FAILED components                             failed on route: none
[PASS] UCS         capacity >= required throughput                      bottleneck 950 kW >= 600 kW
[PASS] UCS         route thermal <= limit                               14.00 C <= 14.00 C
[PASS] UCS         reported cost = recomputed cost                      44.2000 vs 44.2000
[PASS] UCS         optimal vs exhaustive enumeration                    UCS 44.20 = best of 6 feasible routes 44.20
[PASS] UCS         delivered power covers critical load                 delivered 950.0 kW - critical 420 kW = budget 530.0 kW
[PASS] SA          SP within [300, 700] NU                              x_SP = 320
[PASS] SA          ES within [80, 400] NU                               x_ES = 122
[PASS] SA          FS within [60, 350] NU                               x_FS = 125
[PASS] SA          SR within [100, 500] NU                              x_SR = 174
[PASS] SA          SN within [40, 250] NU                               x_SN = 100
[PASS] SA          material <= reserve                                  841 <= 1200 NU
[PASS] SA          nano power <= rho * B                                370.0 <= 371.0 kW (rho=0.70)
[PASS] SA          armor thermal <= limit                               24.78 <= 30.0 C
[PASS] SA          reported objective = recomputed                      J = 50.8307
[PASS] SA          final J >= initial J                                 50.831 >= 38.287
[PASS] CSP         schedule found                                       makespan 15 slots
[PASS] CSP         every task scheduled                                 9/9 tasks
[PASS] CSP         C1 precedence                                        all dependencies respected
[PASS] CSP         C2 safety separation                                 cool-down gaps respected
[PASS] CSP         C3 safety no-overlap                                 no forbidden overlaps
[PASS] CSP         C4 resource exclusivity                              no unit double-booked
[PASS] CSP         equipment compatibility                              all units capable
[PASS] CSP         C5 power profile <= P_res                            peak 155 kW <= 160.0 kW
[PASS] CSP         C6 technicians                                       peak 2 <= 2
[PASS] CSP         C7 deadlines / horizon                               makespan 15 <= 16
[PASS] CSP         durations match derived requirements                 ok
[PASS] INTEGRATED power balance  P_crit + P_nano + peak P_lab <= P_delivered 420 + 370.0 + 155 = 945.0 <= 950.0 kW
[PASS] INTEGRATED reactor balance  P_delivered + losses <= P_avail      950 + 34 <= 1000 kW
[PASS] INTEGRATED thermal  theta_route + theta_nano <= Theta_armor      14.00 + 10.78 = 24.78 <= 30.0 C
[PASS] INTEGRATED mandatory structural protection                       x_SP = 320 >= 300 NU
[PASS] INTEGRATED consumed nanomaterial is replenished                  841 NU replenished in slots 9-12
[PASS] INTEGRATED armor recharged to target state of charge             228.0 kWh delivered >= 220.0 kWh deficit
[PASS] INTEGRATED mission completes before deadline                     15 slots x 30 min = 7.5 h <= 8.0 h

36/36 checks passed
Estimated post-recovery health (model): FLC 86.6%, STB 84.1%, SEN 91.7%, LSC 94.7%, STR 77.8%

================================================================================
FINAL ARMOR STATUS: ARMOR RESTORED - OPERATIONAL AND SAFE
================================================================================

================================================================================
PERFORMANCE STUDY
================================================================================
SA  over 30 seeds: mean J=50.772 sd=0.287 best=51.199 worst=50.128
HC  over 30 seeds: mean J=47.482 sd=0.194 best=48.525 worst=47.446   SA better in 30 seeds
CSP MRV + degree + FC        solved=True nodes=    10 backtracks=     0 checks=    424 time=1.01 ms
CSP MRV + FC                 solved=True nodes=    10 backtracks=     0 checks=    370 time=0.78 ms
CSP FC only (static order)   solved=True nodes=    38 backtracks=     4 checks=   1636 time=1.6 ms
CSP MRV + degree, no FC      solved=True nodes=   341 backtracks=    33 checks=   1149 time=2.13 ms
CSP Plain backtracking       solved=True nodes=  3828 backtracks=  1097 checks=  19697 time=19.48 ms
ROUTE UCS (all constraints)          ARC > PC2 > SW1 > SW2 > BUS_B > CLH cost=44.2 thermal=14.0 feasible=True
ROUTE UCS thermal-blind              ARC > PC2 > SW2 > BUS_B > CLH cost=37.25 thermal=14.5 feasible=False
ROUTE UCS thermal- & capacity-blind  ARC > PC3 > SW2 > BUS_B > CLH cost=28.35 thermal=11.5 feasible=False
ROUTE BFS (fewest hops)              ARC > PC2 > SW1 > BUS_A > CLH cost=46.4 thermal=12.0 feasible=True
timing (ms): {'ucs': 0.118, 'sa': 54.1, 'csp': 1.01}

================================================================================
FIGURES
================================================================================
  outputs/figures/fig01_architecture.png
  outputs/figures/fig02_power_network.png
  outputs/figures/fig03_ucs_expansion.png
  outputs/figures/fig04_nano_allocation.png
  outputs/figures/fig05_sa_convergence.png
  outputs/figures/fig06_csp_constraint_graph.png
  outputs/figures/fig07_gantt.png
  outputs/figures/fig08_workflow.png
  outputs/figures/fig09_dashboard.png
  outputs/figures/fig10_performance.png

Exports written to outputs
```